Chương 11

Suy luận

Sau khi đã tiền huấn luyện và tinh chỉnh một LLM, ta có thể dùng nó để đưa ra dự đoán trên dữ liệu mới. Quá trình này được gọi là suy luận (*inference*): LLM tính xác suất của các đầu ra có thể có khi biết một đầu vào, rồi chọn đầu ra có xác suất lớn nhất. Bài toán suy luận thường được biểu diễn dưới dạng sau:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}), \tag{11.1}$$

trong đó $\mathbf{x}$ là chuỗi đầu vào, $\mathbf{y}$ là một chuỗi đầu ra có thể có, và $\hat{\mathbf{y}}$ là chuỗi đầu ra tốt nhất.

Đây có lẽ là một trong những công thức được dùng rộng rãi nhất trong NLP, có nguồn gốc từ những ngày đầu của các hệ thống nhận dạng tiếng nói (*speech recognition*) và dịch máy (*machine translation*) dựa trên mô hình xác suất. Dù với một số ứng dụng, chẳng hạn dự đoán một token duy nhất bằng một mô hình ngôn ngữ rất nhỏ, việc giải bài toán tối ưu này có vẻ tầm thường, thì trong phần lớn trường hợp, thách thức tính toán nảy sinh từ cả việc tính $\Pr(\mathbf{y}|\mathbf{x})$ lẫn việc thực hiện phép arg max. Vì vậy, những vấn đề chúng ta muốn giải quyết trong chương này là: 1) tính xác suất dự đoán một cách hiệu quả khi đã có một LLM được huấn luyện, và 2) thiết kế một phương pháp tìm kiếm xấp xỉ hiệu quả để tìm $\hat{\mathbf{y}}$.

Chú giải của người dịch: Đọc công thức (11.1)

Ký hiệu $\arg\max_{\mathbf{y}} f(\mathbf{y})$ nghĩa là "giá trị của $\mathbf{y}$ làm cho $f(\mathbf{y})$ lớn nhất" (chứ không phải bản thân giá trị lớn nhất đó). Ví dụ, nếu mô hình cho

$\Pr(\text{“Xin chào”}|\mathbf{x}) = 0.6$, $\Pr(\text{“Chào bạn”}|\mathbf{x}) = 0.3$ và $\Pr(\text{“Tạm biệt”}|\mathbf{x}) = 0.1$ thì $\hat{\mathbf{y}} =$ “Xin chào”. Cái khó là trong thực tế số chuỗi đầu ra có thể có là khổng lồ: với bộ từ vựng 50 000 token và chuỗi dài 20 token đã có $50\,000^{20}$ khả năng, nên không thể thử hết từng chuỗi.

Ở mức tổng quát, đây là những vấn đề nền tảng của trí tuệ nhân tạo và đã được nghiên cứu sâu rộng. Do đó, nhiều kỹ thuật đã được kiểm chứng có thể áp dụng trực tiếp; chẳng hạn, ta có thể dùng các thuật toán tìm kiếm tham lam (*greedy search*) để xây dựng một hệ thống suy luận hiệu quả. Mặt khác, có thể xét thêm các tối ưu đặc thù cho mô hình, như các mô hình chú ý hiệu quả cho Transformer, để cải thiện hiệu suất hơn nữa. Nhưng trong nhiều ứng dụng thực tế, ta vẫn phải cân bằng giữa độ chính xác và hiệu suất bằng cách kết hợp khéo léo nhiều kỹ thuật khác nhau.

Tầm quan trọng của bài toán suy luận trong LLM còn nằm ở chỗ nhiều kịch bản ứng dụng đòi hỏi xử lý những chuỗi cực dài. Các nghiên cứu gần đây cho thấy việc đưa thêm prompt và thông tin ngữ cảnh trong lúc suy luận, chẳng hạn các prompt chuỗi suy nghĩ (*chain-of-thought*) dài, có thể cải thiện đáng kể hiệu năng của LLM. Điều này mở ra một hướng mới để co giãn LLM: có thể đạt kết quả tốt hơn bằng cách tăng lượng tính toán tại thời điểm suy luận. Ví dụ, nhờ co giãn tại thời điểm suy luận (*inference-time scaling*), các hệ thống o1 của OpenAI [2024] và R1 của DeepSeek [2025] đã đạt hiệu năng ấn tượng trên các nhiệm vụ lập luận (*reasoning*) phức tạp và lập trình. Đến lượt mình, điều này lại thúc đẩy giới NLP tập trung nhiều hơn vào vấn đề suy luận hiệu quả.

Chú giải của người dịch: “Suy luận” và “lập luận”

Trong chương này, *suy luận* (*inference*) chỉ việc *chạy* một mô hình đã huấn luyện xong để sinh ra kết quả, giống như một học sinh đã ôn bài xong và giờ đi làm bài thi. Còn *lập luận* (*reasoning*) chỉ *năng lực* suy nghĩ nhiều bước (giải toán, chứng minh, viết chương trình). Hai khái niệm liên quan với nhau: để “lập luận” tốt hơn, mô hình có thể được cho “suy luận” lâu hơn, sinh ra nhiều bước trung gian hơn — đó chính là ý tưởng co giãn tại thời điểm suy luận.

Trong chương này, chúng tôi sẽ giới thiệu các khái niệm và thuật toán cơ bản của suy luận LLM, bao gồm khung điền trước–giải mã (*prefill-decode*), các thuật toán tìm kiếm (giải mã), và các thước đo đánh giá hiệu

năng suy luận. Sau đó, chúng tôi trình bày các phương pháp nâng cao hiệu suất suy luận LLM, bao gồm nhiều kỹ thuật tăng tốc hệ thống và nén mô hình. Cuối cùng, chúng tôi bàn về co giãn tại thời điểm suy luận, được xem là một ứng dụng quan trọng của tối ưu hóa suy luận.

11.1 Điền trước và giải mã

Trong mục này, chúng tôi trình bày khung điền trước–giải mã, cách tiếp cận phổ biến nhất để mô tả và cài đặt quá trình suy luận của LLM. Trước hết chúng tôi giới thiệu ký hiệu và kiến thức nền, rồi mô tả chi tiết khung này, chẳng hạn các thuật toán giải mã dùng trong suy luận LLM.

11.1.1 Kiến thức chuẩn bị

Dù đã mô tả LLM nhiều lần trong cuốn sách này, chúng tôi vẫn bắt đầu bằng việc định nghĩa ngắn gọn các ký hiệu để thuận tiện cho phần thảo luận tiếp theo, và để chương này tự đầy đủ.

- $\mathbf{x}$: Chuỗi token đầu vào. Có thể xem nó là "prompt", gồm các chỉ dẫn, đầu vào của người dùng và mọi ngữ cảnh bổ sung được dùng làm đầu vào cho LLM. $\mathbf{x}$ gồm $m+1$ token, ký hiệu là $x_0 \cdots x_m$, trong đó x_0 là ký hiệu bắt đầu $\langle \text{SOS} \rangle$.
- $\mathbf{y}$: Chuỗi token đầu ra, còn gọi là *phản hồi* cho đầu vào. $\mathbf{y}$ gồm n token, ký hiệu là $y_1 \cdots y_n$.
- $\mathbf{y}_{<i}$: Các token đầu ra đứng trước vị trí i, tức là $\mathbf{y}_{<i} = y_1 \cdots y_{i-1}$.
- $\Pr(\mathbf{y}|\mathbf{x})$: Xác suất LLM sinh ra $\mathbf{y}$ khi biết $\mathbf{x}$. Nếu LLM được tham số hóa bởi θ, ta có thể viết là $\Pr_\theta(\mathbf{y}|\mathbf{x})$.
- $[\mathbf{x}, \mathbf{y}]$: Chuỗi token ghép nối của $\mathbf{x}$ và $\mathbf{y}$, tức là $[\mathbf{x}, \mathbf{y}] = x_0 \cdots x_m y_1 \cdots y_n$. Đôi khi chúng tôi dùng ký hiệu $\text{seq}_{\mathbf{x},\mathbf{y}}$ để biểu diễn $[\mathbf{x}, \mathbf{y}]$.
- $\Pr([\mathbf{x}, \mathbf{y}])$: Xác suất LLM sinh ra chuỗi token $[\mathbf{x}, \mathbf{y}]$.

Như mô tả trong Phương trình (11.1), mục tiêu của suy luận LLM là cực đại hóa $\Pr(\mathbf{y}|\mathbf{x})$. Mô hình hóa xác suất có điều kiện này là việc thường

gặp trong NLP. Thoạt nhìn, nó có vẻ là một bài toán mô hình hóa chuỗi-sang-chuỗi (*sequence-to-sequence*), trong đó ta biến đổi một chuỗi thành chuỗi khác bằng các mô hình bộ mã hóa (*encoder*)–bộ giải mã (*decoder*). Tuy nhiên, ở đây chúng ta không bàn về bài toán chuỗi-sang-chuỗi hay kiến trúc bộ mã hóa–bộ giải mã. Thay vào đó, như đã thảo luận ở các chương trước, bài toán mô hình hóa này có thể được giải quyết bằng các mô hình chỉ có bộ giải mã. Để làm vậy, ta có thể diễn giải xác suất ở thang logarit $\log \Pr(\mathbf{y}|\mathbf{x})$ là hiệu của $\log \Pr([\mathbf{x}, \mathbf{y}])$ và $\log \Pr(\mathbf{x})$:

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \log \Pr([\mathbf{x}, \mathbf{y}]) - \log \Pr(\mathbf{x}), \tag{11.2}$$

trong đó $\log \Pr([\mathbf{x}, \mathbf{y}])$ và $\log \Pr(\mathbf{x})$ có thể thu được bằng cách chạy LLM lần lượt trên các chuỗi $[\mathbf{x}, \mathbf{y}]$ và $\mathbf{x}$. Ví dụ, ta có thể tính xác suất sinh ra $\mathbf{x}$ bằng quy tắc chuỗi:

$$\begin{aligned}
\log \Pr(\mathbf{x}) &= \log \Pr(x_0 \cdots x_m) \\
&= \log \left[\Pr(x_0) \Pr(x_1|x_0) \cdots \Pr(x_m|x_0 \cdots x_{m-1}) \right] \\
&= \underbrace{\log \Pr(x_0)}_{=0} + \sum_{j=1}^{m} \log \Pr(x_j|\mathbf{x}_{<j}) \\
&= \sum_{j=1}^{m} \log \Pr(x_j|\mathbf{x}_{<j}).
\end{aligned} \tag{11.3}$$

Nói cách khác, ta tính log-xác suất dự đoán token tại mỗi vị trí của $\mathbf{x}$ rồi cộng tất cả các log-xác suất này lại. Lưu ý rằng vì x_0 là ký hiệu bắt đầu được định nghĩa sẵn, ta bỏ qua đóng góp của nó ở đây.

Chú giải của người dịch: Vì sao dùng logarit và vì sao $\log \Pr(x_0) = 0$?

Theo quy tắc chuỗi, xác suất của cả chuỗi là *tích* các xác suất từng token. Nhờ tính chất $\log(ab) = \log a + \log b$ đã học ở lớp 11, tích trở thành *tổng*, dễ tính và tránh việc nhân nhiều số rất nhỏ (ví dụ $0.01^{100} = 10^{-200}$) khiến máy tính làm tròn thành 0. Ví dụ: nếu ba token có xác suất 0.5, 0.2, 0.1 thì $\Pr = 0.01$, còn $\log_{10} \Pr = -0.30 - 0.70 - 1 = -2$. Ký hiệu bắt đầu x_0 *luôn luôn* có mặt nên $\Pr(x_0) = 1$, do đó $\log \Pr(x_0) = \log 1 = 0$. Phương trình (11.2) chỉ là quy tắc $\Pr(\mathbf{y}|\mathbf{x}) = \Pr([\mathbf{x}, \mathbf{y}]) / \Pr(\mathbf{x})$ viết dưới dạng logarit.

Tuy nhiên, trong các cài đặt LLM thông thường, ta không cần tính log-xác suất của chuỗi đầu vào, mà dùng LLM để trực tiếp tính log-xác

suất của chuỗi đầu ra dưới dạng sau:

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i}), \tag{11.4}$$

trong đó $\mathbf{x}$ và $\mathbf{y}_{<i}$ cùng tạo thành ngữ cảnh để dự đoán y_i. Theo ký hiệu thông dụng trong các tài liệu, ta viết $\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ như cách viết tắt của $\Pr(y_i|[\mathbf{x}, \mathbf{y}_{<i}])$.

Như vậy, để giải quyết bài toán suy luận mô tả trong Phương trình (11.1), ta có hai bài toán con:

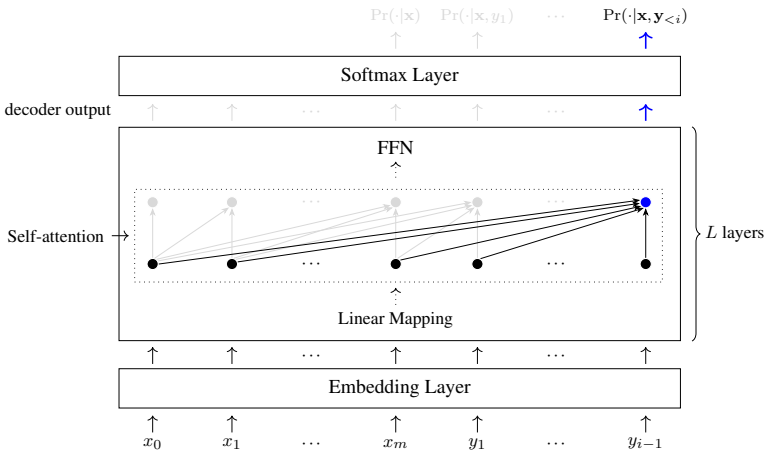
- **Tính toán mô hình**: ta mô hình hóa $\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ và tính nó một cách hiệu quả.
- **Tìm kiếm**: ta tìm chuỗi đầu ra tối ưu (hoặc gần tối ưu) theo $\log \Pr(\mathbf{y}|\mathbf{x})$.

Bài toán con thứ hai là một vấn đề kinh điển trong NLP. Trong Mục 11.1.3, chúng tôi sẽ cho thấy có một số thuật toán đã được nghiên cứu kỹ có thể áp dụng để tìm kiếm hiệu quả trong không gian các chuỗi đầu ra có thể có. Bài toán con thứ nhất đòi hỏi một mô hình ngôn ngữ tạo ra một phân phối trên bộ từ vựng (*vocabulary*) V khi biết một chuỗi token ngữ cảnh. Ta có thể làm điều này bằng cách huấn luyện một bộ giải mã Transformer, cho ra phân phối:

$$\Pr(\cdot|\mathbf{x}, \mathbf{y}_{<i}) = \text{Softmax}(\mathbf{H}\mathbf{W}^o)_{m+i} \tag{11.5}$$

$$\mathbf{H} = \text{Dec}([\mathbf{x}, \mathbf{y}_{<i}]). \tag{11.6}$$

Ở đây $\text{Dec}(\cdot)$ tạo ra một chuỗi biểu diễn, mỗi biểu diễn ứng với một vị trí của chuỗi đầu vào. Vì vậy, nếu đưa $[\mathbf{x}, \mathbf{y}_{<i}]$ vào LLM thì $\mathbf{H}$ là một ma trận $i' \times d$, trong đó d là số chiều của mỗi biểu diễn, và $i' = m + i$ là số token ngữ cảnh. Sau đó ta có thể dùng một tầng Softmax để biến các biểu diễn này thành các phân phối trên token. $\mathbf{W}^o \in \mathbb{R}^{d \times |V|}$ là ma trận ánh xạ tuyến tính của tầng Softmax, và $\mathbf{H}\mathbf{W}^o$ biến các biểu diễn d chiều trong $\mathbf{H}$ thành các biểu diễn $|V|$ chiều. Chỉ số dưới $m + i$ cho biết phép Softmax chỉ được áp dụng cho biểu diễn tại vị trí $m + i$. Xem Hình 11.1 để thấy minh họa của kiến trúc này.



Hình 11.1: Kiến trúc chỉ có bộ giải mã của LLM. Bộ giải mã gồm một tầng nhúng (*Embedding Layer*) và một chồng các tầng Transformer. Trong mỗi tầng Transformer, đầu vào đi qua một mô-đun tự chú ý (*Self-attention*) rồi đến một FFN. Đầu ra của bộ giải mã (*decoder output*) là một chuỗi biểu diễn được đưa vào tầng Softmax (*Softmax Layer*), tầng này sinh ra một phân phối trên các token cho mỗi vị trí. (*Linear Mapping*: ánh xạ tuyến tính; *L layers*: L tầng.)

Chú giải của người dịch: Tầng Softmax làm gì?

Biểu diễn cuối cùng tại vị trí $m+i$ là một vectơ d số (ví dụ $d = 4096$). Nhân nó với ma trận $\mathbf{W}^o$ cho ra $|V|$ "điểm số", mỗi điểm ứng với một token trong bộ từ vựng. Softmax biến các điểm số này thành xác suất dương có tổng bằng 1: $\text{Softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$. Ví dụ, với 3 token có điểm $(2, 1, 0)$ ta được xác suất khoảng $(0.67,\ 0.24,\ 0.09)$. Token có điểm cao nhất sẽ có xác suất lớn nhất được chọn làm token tiếp theo.

Dec$(\cdot)$ là một mạng giải mã Transformer gồm một mạng nhúng và một số mạng tự chú ý và FFN xếp chồng. Chúng tôi sẽ không bàn chi tiết về Transformer ở đây, vì bạn đọc có thể dễ dàng tìm hiểu các mô hình này từ tài liệu. Tuy nhiên, đáng chỉ ra rằng khó khăn của suy luận một phần đến từ việc dùng cơ chế tự chú ý (*self-attention*) trong Transformer. Nhắc lại rằng dạng tổng quát của tự chú ý một đầu là:

$$\text{Att}_{\text{qkv}}(\mathbf{q}_{i'}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\left(\frac{\mathbf{q}_{i'}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \tag{11.7}$$

trong đó $\mathbf{q}_{i'} \in \mathbb{R}^d$ là truy vấn (*query*) tại vị trí i' (tức là vị trí của y_i), còn $\mathbf{K}$ và $\mathbf{V} \in \mathbb{R}^{i' \times d}$ lần lượt là các khóa (*key*) và giá trị (*value*) tính đến vị trí i'.

Tại mỗi bước suy luận, ta gọi hàm tự chú ý $\text{Att}_{\text{qkv}}(\cdot)$, tiếp theo là một FFN, để tạo ra một biểu diễn d chiều tích hợp thông tin từ cả token hiện tại và ngữ cảnh bên trái của nó. Quá trình này được lặp lại qua L tầng tự chú ý và FFN, tạo thành một chồng tầng Transformer. Đầu ra của tầng thứ L trong chồng này là biểu diễn cuối cùng.

Mỗi lần như vậy, mô hình cho vị trí i' chú ý tới mọi vị trí trước đó, dẫn đến $2i'$ phép nhân vectơ (i' lần cho $\mathbf{q}_{i'}\mathbf{K}^\top$ và i' lần cho tích của $\text{Softmax}(\frac{\mathbf{q}_{i'}\mathbf{K}^\top}{\sqrt{d}})$ với $\mathbf{V}$). Do đó, sinh một chuỗi độ dài len có độ phức tạp thời gian $O(L \times d \times \text{len}^2)$ đối với mạng tự chú ý. Rõ ràng, suy luận của mô hình này chậm với các chuỗi dài vì độ phức tạp thời gian bậc hai theo độ dài chuỗi. Vì thế, nhiều cải tiến cho Transformer và các mô hình thay thế đã tập trung vào những phương pháp hiệu quả nhanh hơn độ phức tạp bậc hai này, chẳng hạn các cơ chế chú ý thưa (*sparse attention*) và các mô hình thời gian tuyến tính. Bàn luận chi tiết về các Transformer hiệu quả có thể tìm thấy ở các chương trước, và mục này sẽ tập trung vào kiến trúc Transformer chuẩn.

Chú giải của người dịch: Độ phức tạp bậc hai nghĩa là gì?

Ký hiệu $O(\cdot)$ mô tả khối lượng tính toán tăng thế nào khi đầu vào lớn lên. Khi sinh token thứ i', mô hình phải "nhìn lại" cả i' token trước đó. Tổng số lần nhìn lại khi sinh cả chuỗi độ dài len là $1 + 2 + \cdots + \text{len} = \frac{\text{len}(\text{len}+1)}{2}$, tức tỉ lệ với len^2 (nhân thêm d phép tính cho mỗi tích vô hướng và L tầng). Hệ quả: chuỗi dài gấp đôi thì tốn gấp khoảng *bốn* lần thời gian; chuỗi dài gấp 10 thì tốn gấp khoảng 100 lần.

Lưu ý rằng trong tự chú ý, các truy vấn, khóa và giá trị của một tầng đều là các ánh xạ tuyến tính từ cùng một đầu vào (tức là đầu ra của tầng trước). Một khi một cặp khóa–giá trị mới được sinh ra, nó sẽ được dùng lặp lại trong các bước suy luận tiếp theo. Thay vì sinh lại các cặp khóa–giá trị này trong lúc suy luận, cách hợp lý hơn là lưu chúng trong một cấu trúc gọi là **bộ nhớ đệm khóa–giá trị** (*key-value cache*), hay **bộ nhớ đệm KV** (*KV cache*). Do đó, $(\mathbf{K}, \mathbf{V})$ có thể được xem một cách tự nhiên là một bộ nhớ đệm KV. Bộ nhớ đệm này được cập nhật như sau:

$$\mathbf{K} = \text{Append}(\mathbf{K}, \mathbf{k}_{i'}), \tag{11.8}$$

$$\mathbf{V} = \text{Append}(\mathbf{V}, \mathbf{v}_{i'}), \tag{11.9}$$

trong đó $(\mathbf{k}_{i'}, \mathbf{v}_{i'})$ là cặp khóa–giá trị mới được sinh tại vị trí i', và $\text{Append}(\mathbf{a}, \mathbf{b})$ là hàm nối thêm vectơ hàng $\mathbf{b}$ vào ma trận $\mathbf{a}$. Hình 11.2 cho thấy cách một bộ giải mã Transformer hoạt động với bộ nhớ đệm KV.

Cuối cùng, quá trình tính $\log \Pr(\mathbf{y}|\mathbf{x})$ được tóm tắt như sau:

1. Ta ghép $\mathbf{x}$ và $\mathbf{y}$ thành một chuỗi $[\mathbf{x}, \mathbf{y}]$. Với mỗi vị trí i' của chuỗi này, ta thực hiện các bước sau.

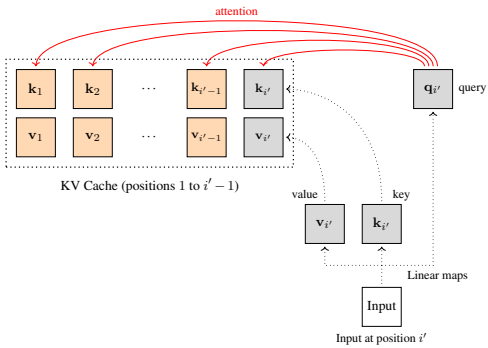
   (a) Tính vectơ nhúng của token tại vị trí i', rồi đưa vectơ nhúng thu được làm biểu diễn ban đầu vào chồng tầng Transformer.

   (b) Trong mỗi tầng Transformer, cho biểu diễn đầu vào đi qua mạng tự chú ý trước rồi qua một FFN. Trong mạng tự chú ý, biểu diễn đầu vào được biến đổi thành $\mathbf{q}_{i'}$, $\mathbf{k}_{i'}$ và $\mathbf{v}_{i'}$. Sau đó, ta cập nhật bộ nhớ đệm KV $(\mathbf{K}, \mathbf{V})$ bằng $\mathbf{k}_{i'}$ và $\mathbf{v}_{i'}$ (xem Phương trình (11.8–11.9)). Rồi ta tính đầu ra của mô hình chú ý bằng cách cho $\mathbf{q}_{i'}$ chú ý tới $(\mathbf{K}, \mathbf{V})$ (xem Phương trình (11.7)).

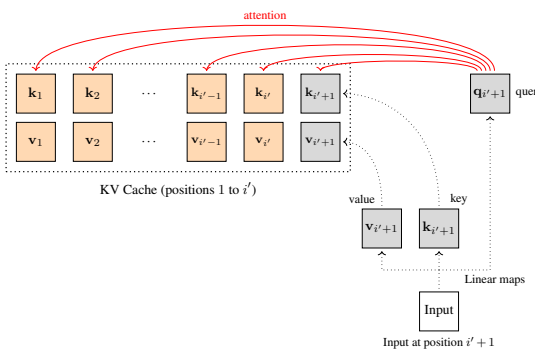
   (c) Nếu $i' > m$ (tức là $i = i' - m \geq 1$), ta lấy đầu ra của chồng Transformer và tính xác suất dự đoán token $\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ qua

tầng Softmax (xem Phương trình (11.5)).

2. Khi đến cuối chuỗi, ta thu được $\log \Pr(\mathbf{y}|\mathbf{x})$ bằng cách cộng $\log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})$ trên $i \in [1, n]$ (xem Phương trình (11.4)).



(a) Updating the KV Cache at Position i'



(b) Updating the KV Cache at Position $i'+1$

Hình 11.2: Minh họa bộ nhớ đệm KV. Ta cập nhật bộ nhớ đệm KV tại một vị trí, thực hiện phép chú ý, rồi chuyển sang vị trí tiếp theo để lặp lại quá trình. (a) Cập nhật bộ nhớ đệm KV tại vị trí i'; (b) cập nhật bộ nhớ đệm KV tại vị trí $i'+1$. (*KV Cache (positions 1 to* $i'-1$*)*: bộ nhớ đệm KV chứa các vị trí từ 1 đến $i'-1$; *Input at position* i': đầu vào tại vị trí i'; *Linear maps*: các ánh xạ tuyến tính; *query/key/value*: truy vấn/khóa/giá trị; *attention*: phép chú ý.)

Chú giải của người dịch: Bộ nhớ đệm KV: "ghi chép lại để khỏi tính lại"

Hãy hình dung bạn đọc một cuốn truyện dài và mỗi chương lại phải đọc lại từ đầu để nhớ cốt truyện: rất phí công. Cách thông minh hơn là ghi tóm tắt từng chương vào sổ tay, chương mới chỉ cần tra sổ. Bộ nhớ đệm KV chính là "sổ tay" đó: khóa $\mathbf{k}$ và giá trị $\mathbf{v}$ của các token cũ không đổi khi có token mới, nên ta tính chúng *một lần*, cất đi, và mỗi bước chỉ thêm một hàng mới (Phương trình (11.8–11.9)). Cái giá phải trả là bộ nhớ: sổ tay dày lên theo độ dài chuỗi, số tầng L và số chiều d.

11.1.2 Khung hai giai đoạn

Như đã thấy, mô hình hóa ngôn ngữ là một quá trình tự hồi quy (*autoregressive*) chuẩn, trong đó mỗi token được sinh ra lần lượt từng cái một, có điều kiện theo các token trước đó. Với Transformer, điều này đòi hỏi mô hình duy trì một bộ nhớ đệm KV lưu các biểu diễn quá khứ, và cho phép biểu diễn mới sinh ra chú ý tới bộ nhớ đệm KV này. Khi nhìn từ góc độ tính toán bộ nhớ đệm KV, suy luận LLM tự nhiên chia thành hai giai đoạn:

- **Điền trước** (*prefilling*). Giai đoạn điền trước (*prefilling*) khởi tạo bộ nhớ đệm KV cho chuỗi đầu vào $\mathbf{x}$. Nó được gọi là điền trước vì mô hình chuẩn bị và lưu sẵn các cặp khóa–giá trị cho mỗi token trong đầu vào trước khi việc sinh token thực sự bắt đầu. Quá trình điền trước trong một LLM có thể được biểu diễn như sau:

$$\text{cache} \;=\; \text{Dec}_{\text{kv}}(\mathbf{x}), \tag{11.10}$$

  trong đó $\text{Dec}_{\text{kv}}(\cdot)$ là mạng giải mã (tức là giống $\text{Dec}(\cdot)$), nhưng nó trả về bộ nhớ đệm KV trong tự chú ý thay vì các biểu diễn đầu ra. cache là một danh sách, cho bởi:

$$\text{cache} \;=\; \{\text{cache}^1, \cdots, \text{cache}^L\}, \tag{11.11}$$

  trong đó cache^l biểu diễn các cặp khóa–giá trị của tầng thứ l.

- **Giải mã** (*decoding*). Giai đoạn giải mã (*decoding*) tiếp tục sinh token dựa trên bộ nhớ đệm KV, như minh họa trong Hình 11.2. Khi một token mới được đưa vào bộ giải mã, ta cập nhật bộ nhớ đệm KV

ở mỗi tầng bằng cách thêm cặp khóa–giá trị mới. Bộ nhớ đệm đã cập nhật sau đó được dùng cho tính toán tự chú ý. Việc sinh token dừng lại khi một tiêu chí dừng nào đó được thỏa mãn, chẳng hạn khi token được sinh ra là ký hiệu kết thúc. Mục tiêu của giải mã là tìm chuỗi dự đoán tốt nhất, cho bởi:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\text{cache}). \tag{11.12}$$

Dù mô hình xác suất vẫn có điều kiện theo $\mathbf{x}$, quá trình giải mã hoạt động trên bộ nhớ đệm KV được tính từ $\mathbf{x}$. Vì vậy ta dùng $\Pr(\mathbf{y}|\text{cache})$ thay cho $\Pr(\mathbf{y}|\mathbf{x})$ để nhấn mạnh sự phụ thuộc vào bộ nhớ đệm KV.

Các quá trình điền trước và giải mã được minh họa trong Hình 11.3. Lưu ý rằng cả hai quá trình dùng chung một mô hình tự hồi quy nền tảng. Tuy nhiên, như trình bày trong Bảng 11.1, chúng khác nhau ở một số khía cạnh, dẫn đến những cách cài đặt rất khác nhau trong thực tế.

	Điền trước	Giải mã
Mục tiêu	Thiết lập ngữ cảnh ban đầu $\mathbf{x}$.	Tiếp tục sinh các token $\mathbf{y}$ sau đầu vào ban đầu.
Khả năng thấy token	Các token trong $\mathbf{x}$ được đưa vào cùng một lúc.	Các token trong $\mathbf{y}$ được sinh tuần tự, tức là việc dự đoán một token phụ thuộc vào các token đã sinh trước đó.
Cách dùng ngữ cảnh	Xây dựng ngữ cảnh, hay biểu diễn đã mã hóa của đầu vào.	Dùng các cặp khóa–giá trị đã lưu (từ điền trước) để sinh tiếp token.
Giới hạn tài nguyên	Bị giới hạn bởi tính toán	Bị giới hạn bởi bộ nhớ
Chi phí tính toán	Cao	Rất cao

Bảng 11.1: So sánh điền trước và giải mã.

Về bản chất, dù điền trước vẫn dựa trên dự đoán token tự hồi quy, nó có thể được xem là một quá trình mã hóa. Lý do là mục tiêu của ta không phải là sinh token, mà là xây dựng một biểu diễn ngữ cảnh (tức là bộ nhớ đệm KV) cho các bước tiếp theo trong giai đoạn giải mã. Theo nghĩa này, nó có một số điểm tương đồng với các mô hình dựa trên bộ mã hóa như BERT, trong đó ta mã hóa chuỗi đầu vào thành một chuỗi biểu diễn token theo ngữ cảnh. Mặt khác, khác với BERT sinh ra biểu diễn chuỗi hai chiều,

điền trước dựa trên nhiệm vụ mô hình hóa ngôn ngữ chuẩn, nên nó là một chiều. Lưu ý rằng vì toàn bộ chuỗi $\mathbf{x}$ được đưa vào mô hình cùng một lúc, mọi truy vấn có thể được gộp lại và phép tự chú ý được thực hiện song song trên $\mathbf{x}$. Gọi $\mathbf{Q}$ là các truy vấn được gộp vào một ma trận. Mô hình tự chú ý trong điền trước có thể được định nghĩa là:

$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{Mask}\Big)\mathbf{V}, \tag{11.13}$$

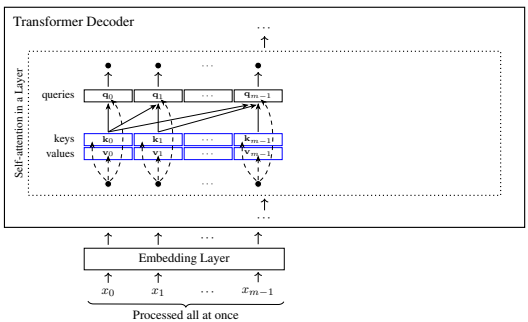
trong đó $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{(m+1)\times d}$. $\mathbf{Mask} \in \mathbb{R}^{(m+1)\times(m+1)}$ là một mặt nạ (*mask*) bảo đảm mỗi token chỉ chú ý tới chính nó và các token đứng trước nó trong chuỗi. Nó được biểu diễn bằng cách đặt các giá trị trong mặt nạ ứng với các token tương lai bằng một số âm rất lớn; chẳng hạn, với truy vấn $\mathbf{q}_i$ và khóa $\mathbf{k}_j$, ta đặt giá trị của phần tử (i, j) là $-\infty$ nếu $i < j$. Một ưu điểm của việc xử lý chuỗi bằng một phép tính tự chú ý duy nhất là ta có thể tận dụng tốt hơn khả năng tính toán song song của GPU hiện đại, và nhờ đó tăng tốc điền trước. Nói chung, quá trình điền trước được xem là bị giới hạn bởi tính toán (*compute-bound*). Lý do là việc gộp nhiều phép tính thành một phép tính làm giảm số lần truyền dữ liệu, và nút thắt hiệu năng thường đến từ năng lực tính toán chứ không phải băng thông bộ nhớ.

Chú giải của người dịch: Mặt nạ và "giới hạn bởi tính toán / bộ nhớ"

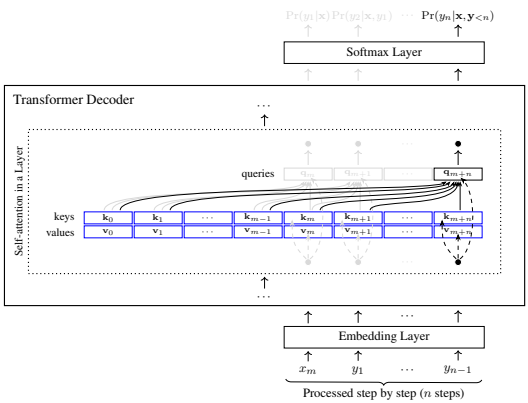
Mặt nạ: vì $e^{-\infty} = 0$, cộng $-\infty$ vào điểm số trước Softmax làm trọng số chú ý tới token tương lai bằng đúng 0. Ví dụ với 3 token, các ô $(1,2)$, $(1,3)$, $(2,3)$ của mặt nạ là $-\infty$, các ô còn lại là 0: token 1 chỉ nhìn chính nó, token 2 nhìn token 1–2, token 3 nhìn cả ba.
Giới hạn bởi tính toán nghĩa là phần cứng bận *tính* (như một đầu bếp có sẵn mọi nguyên liệu trên bàn, chỉ chậm vì nấu); *giới hạn bởi bộ nhớ* nghĩa là phần cứng chủ yếu chờ *chuyển dữ liệu* từ bộ nhớ đến bộ xử lý (đầu bếp nấu nhanh nhưng phải liên tục chạy ra kho lấy nguyên liệu). Khi giải mã, mỗi bước chỉ tính cho một token nhưng phải đọc lại toàn bộ bộ nhớ đệm KV, nên thời gian chủ yếu dành cho việc đọc bộ nhớ.

Giải mã (*decoding*) là một quá trình sinh văn bản từ trái sang phải chuẩn. Chuỗi token được sinh ra một cách tự hồi quy bằng cách dự đoán từng token một dựa trên bộ nhớ đệm KV. Mỗi token mới sinh ra cần chú ý tới mọi token trước đó, theo Phương trình (11.7). Vì vậy, quá trình giải mã bị giới hạn bởi bộ nhớ (*memory-bound*) do phải truy cập thường xuyên

vào bộ nhớ đệm KV. Chi phí giải mã tăng đáng kể khi càng nhiều token được sinh ra. Trong hầu hết trường hợp, giải mã tốn kém về tính toán hơn điền trước. Lưu ý rằng điều này không chỉ vì trong giải mã, LLM sinh token từng cái một và cập nhật bộ nhớ đệm KV lặp đi lặp lại. Như sẽ thấy ở tiểu mục sau, ta có thể cần khám phá nhiều chuỗi token khác nhau trong lúc giải mã, khiến bài toán phức tạp hơn và chi phí còn tăng thêm.



(a) Prefilling



(b) Decoding (at the n-th step)

Hình 11.3: Minh họa các quá trình điền trước và giải mã. Khi điền trước, toàn bộ chuỗi đầu vào được xử lý cùng nhau và bộ nhớ đệm KV được điền đầy. Khi giải mã, LLM sinh chuỗi đầu ra từng bước dựa trên bộ nhớ đệm KV đã được điền trước. (a) Điền trước; (b) Giải mã (tại bước thứ n). (*Processed all at once*: xử lý cùng một lúc; *Processed step by step (n steps)*: xử lý từng bước (n bước); *Self-attention in a Layer*: tự chú ý trong một tầng; *queries/keys/values*: truy vấn/khóa/giá trị.)

11.1.3 Các thuật toán giải mã

Đến đây, phần thảo luận của chúng ta về suy luận LLM chủ yếu tập trung vào bài toán tính toán mô hình, tức là cách tính $\Pr(\mathbf{y}|\mathbf{x})$. Bây giờ ta chuyển sang bài toán tìm kiếm. Bài toán có thể phát biểu như sau: cho một LLM $\Pr(\mathbf{y}|\mathbf{x})$, làm thế nào để sinh hiệu quả một chuỗi đầu ra chất lượng cao $\hat{\mathbf{y}}$ khi biết chuỗi đầu vào $\mathbf{x}$? Một cách ngây thơ, ta có thể xét tất cả các chuỗi đầu ra, tính xác suất dự đoán cho từng chuỗi, rồi chọn chuỗi có xác suất cao nhất. Phương pháp này bảo đảm tìm được nghiệm tối ưu toàn cục, nhưng tìm kiếm vét cạn (*exhaustive search*) trực tiếp là bất khả thi với LLM vì số chuỗi đầu ra có thể có tăng theo hàm mũ với độ dài của $\mathbf{y}$.

Trong thực tế, nhiều thuật toán tìm kiếm heuristic (*heuristic search*) khác nhau, chẳng hạn tìm kiếm tham lam và tìm kiếm dựa trên lấy mẫu, thường được dùng để xấp xỉ nghiệm. Mỗi phương pháp này đưa ra một sự đánh đổi giữa chất lượng tìm kiếm và hiệu suất tính toán. Vì vậy, bài toán tìm kiếm trở thành việc cân bằng giữa khám phá (*exploration*) và khai thác (*exploitation*), trong đó mục tiêu là tìm một chiến lược hiệu quả tạo ra đầu ra chất lượng cao mà không phải khám phá toàn bộ không gian.

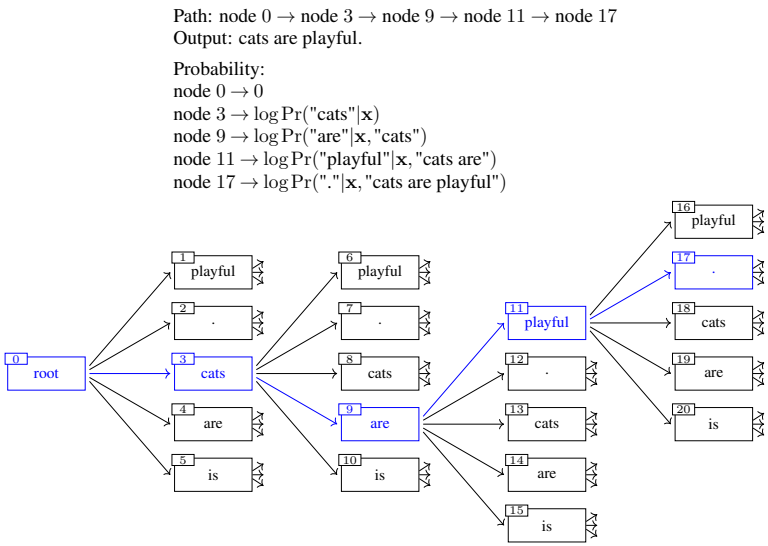
Trước khi bàn chi tiết hơn về các phương pháp này, hãy định nghĩa một cách không hình thức thế nào là một không gian tìm kiếm và nó được biểu diễn ra sao. Trong suy luận LLM, ta định nghĩa một giả thuyết (*hypothesis*) là một bộ gồm chuỗi đầu vào và chuỗi đầu ra. Vì $\mathbf{x}$ cố định trong lúc suy luận, ta có thể đơn giản xem mỗi giả thuyết là một chuỗi đầu ra. Khi đó, không gian tìm kiếm (*search space*), ký hiệu là $\mathcal{Y}$, là tập hợp mọi giả thuyết (tức là chuỗi đầu ra) mà mô hình có thể sinh ra. Bài toán tìm kiếm cho suy luận LLM có thể được viết lại là:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}\in\mathcal{Y}} \Pr(\mathbf{y}|\mathbf{x}). \tag{11.14}$$

Trong NLP, $\mathcal{Y}$ thường được biểu diễn bằng một cấu trúc dữ liệu cây (*tree*) để thuận tiện cho việc tìm kiếm. Hình 11.4 cho thấy một ví dụ về cây tìm kiếm tạo ra từ một bộ từ vựng nhỏ. Trong ví dụ này, một nút biểu diễn một chuỗi tiền tố có thể được nhiều chuỗi dùng chung. Việc tìm kiếm bắt đầu tại gốc của cây, có thể xem là điểm khởi đầu của mọi chuỗi có thể được sinh ra[1]. Mỗi nút con mở rộng tiền tố của nút cha bằng cách thêm

[1]Ở đây, vì các dự đoán của LLM dựa trên $\mathbf{x}$, ta có thể xem gốc là một biểu diễn của $\mathbf{x}$.

một token từ bộ từ vựng vào chuỗi, kèm theo xác suất dự đoán token đó khi biết tiền tố. Quá trình này tiếp diễn khi mỗi nút lại phân nhánh thành các nút con mới, mỗi nút con biểu diễn một khả năng mở rộng chuỗi bằng một token khác. Vì vậy cây tìm kiếm ngày càng sâu và rộng hơn, biểu diễn một số lượng chuỗi tiềm năng tăng theo hàm mũ khi thêm nhiều token. Cấu trúc này cho phép ta duyệt hiệu quả qua các chuỗi có thể có, đánh giá mỗi chuỗi theo log-xác suất tích lũy dọc đường đi từ gốc đến nút đó. Ví dụ, trong Hình 11.4, đường đi từ gốc đến nút 17 ứng với chuỗi đầu ra "*Cats are playful.*" (Mèo thì tinh nghịch.). Log-xác suất dự đoán $\log \Pr(\mathbf{y}|\mathbf{x})$ là tổng log-xác suất của tất cả các nút trên đường đi này.



Hình 11.4: Một cây tìm kiếm cho giải mã. Tại mỗi nút, ta mở rộng cây bằng cách xét mọi token có thể có, mỗi token dẫn đến một nút mới biểu diễn một khả năng tiếp nối của văn bản. Ở đây ta tô sáng một đường đi qua các nút 0, 3, 9, 11 và 17. Đường đi này biểu diễn chuỗi đầu ra "*cats are playful.*", mà log-xác suất có thể tính bằng cách cộng dồn log-xác suất của các nút này. (Phần chữ phía trên hình: *Path*: đường đi; *Output*: đầu ra; *Probability*: xác suất, liệt kê log-xác suất ứng với từng nút; *root*: gốc.)

Nói chung, cây tìm kiếm được tổ chức thành các mức, mỗi mức gồm mọi nút ở cùng độ sâu. Vì vậy, cây có thể được xem là được mở rộng theo

từng mức, trong đó mỗi mức ứng với một bước giải mã. Khi việc tìm kiếm tiến triển, các token mới được nối vào các chuỗi này, mở rộng chúng dần dần.

Gọi Y_i là tập các chuỗi mà LLM sinh ra ở bước i. Y_i có thể thu được bằng cách mở rộng mỗi chuỗi trong Y_{i-1} với mọi token tiếp theo có thể có trong bộ từ vựng V, theo dạng đệ quy sau:

$$Y_i \;=\; Y_{i-1} \times V, \tag{11.15}$$

trong đó $Y_{i-1} \times V$ ký hiệu tích Descartes (*Cartesian product*) của Y_{i-1} và V (tức là mỗi chuỗi trong Y_{i-1} được nối với mỗi token trong V). Lưu ý rằng nếu một chuỗi trong Y_{i-1} đã hoàn chỉnh (chẳng hạn kết thúc bằng token $\langle\text{EOS}\rangle$) thì nó sẽ không được mở rộng thêm. Gọi $\Psi(Y_i)$ là tập mọi chuỗi hoàn chỉnh trong Y_i. Khi đó, không gian tìm kiếm có thể được biểu diễn là:

$$\mathcal{Y} \;=\; \Psi(Y_1) \bigcup \Psi(Y_2) \bigcup \cdots \bigcup \Psi(Y_{n_{\max}}), \tag{11.16}$$

trong đó $n_{\max}$ là độ dài tối đa của một chuỗi.

Chú giải của người dịch: Không gian tìm kiếm lớn đến mức nào?

Tích Descartes giống bài toán đếm ở lớp 11 (quy tắc nhân): nếu Y_{i-1} có a chuỗi và V có b token thì $Y_{i-1} \times V$ có $a \cdot b$ chuỗi. Với bộ từ vựng 5 token như trong Hình 11.4: bước 1 có 5 chuỗi, bước 2 có 25, bước 3 có 125, bước i có 5^i. Với bộ từ vựng thật cỡ 10^5 token, chỉ sau 3 bước đã có 10^{15} chuỗi. Phương trình (11.16) nói rằng $\mathcal{Y}$ là hợp của mọi chuỗi đã kết thúc ở bước 1, bước 2, …, cho tới độ dài tối đa.

Hầu hết các thuật toán giải mã đều theo quá trình tìm kiếm từng mức này. Tuy nhiên, $\mathcal{Y}$ chứa một số lượng chuỗi lớn theo hàm mũ, và việc tìm kiếm trực tiếp trong một không gian rộng lớn như vậy là bất khả thi về mặt tính toán. Vì vậy, các thuật toán giải mã thực tế thường dựa vào các chiến lược tỉa (*prune*) không gian tìm kiếm và tránh khám phá các chuỗi chất lượng thấp. Ví dụ, tại mỗi bước giải mã, Y_i có thể thu được như sau:

$$Y_i \;=\; \text{Prune}(Y_{i-1} \times V), \tag{11.17}$$

trong đó $\text{Prune}(\cdot)$ là một hàm loại bỏ có chọn lọc những chuỗi ít có khả năng dẫn đến kết quả chất lượng cao. Nói chung, ta mong $|Y_i| \ll |Y_{i-1}| \cdot$

$|V|$. Nhờ đó ta có thể giảm mạnh số chuỗi cần xét ở mỗi bước, bảo đảm khối lượng tính toán không tăng theo hàm mũ với độ dài chuỗi.

Tiếp theo, chúng tôi sẽ giới thiệu các thuật toán giải mã này. Một số đã được bàn trong các mô hình chuỗi-sang-chuỗi (xem Chương 5), số khác thường được dùng nhiều hơn trong LLM.

1. Giải mã tham lam

Tìm kiếm tham lam (hay giải mã tham lam (*greedy decoding*)) là một trong những phương pháp giải mã được dùng rộng rãi nhất trong NLP, đặc biệt trong các nhiệm vụ sinh văn bản như dịch máy. Ý tưởng của tìm kiếm tham lam rất đơn giản: tại mỗi bước sinh, nó chọn token tiếp theo có xác suất dự đoán cao nhất. Với mỗi chuỗi $\mathbf{y} = y_1 \cdots y_i \in Y_{i-1} \times V$, ta có thể đánh giá nó bằng $\log \Pr(\mathbf{y}|\mathbf{x})$. Log-xác suất này có thể tính dễ dàng nhờ nhận xét rằng:

$$\begin{aligned}\log \Pr(\mathbf{y}|\mathbf{x}) &= \log \Pr(y_1 \cdots y_i|\mathbf{x}) \\ &= \underbrace{\log \Pr(\mathbf{y}_{<i}|\mathbf{x})}_{\text{tích lũy đến nút cha}} + \underbrace{\log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})}_{\text{mới tính cho nút hiện tại}},\end{aligned} \tag{11.18}$$

Ở đây số hạng thứ nhất là tổng log-xác suất của đường đi từ gốc đến nút cha, đã được tính ở các bước giải mã trước. Tại bước i, ta chỉ cần tính số hạng thứ hai, chính là log-xác suất dự đoán token chuẩn do LLM tạo ra.

Token "tốt nhất" tại bước i khi đó được chọn là:

$$\begin{aligned}y_i^{\text{top1}} &= \arg\max_{y_i \in V} \log \Pr(y_1 \cdots y_i|\mathbf{x}) \\ &= \arg\max_{y_i \in V} \big[\underbrace{\log \Pr(\mathbf{y}_{<i}|\mathbf{x})}_{\text{cố định đối với } y_i} + \log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})\big] \\ &= \arg\max_{y_i \in V} \log \Pr(y_i|\mathbf{x}, \mathbf{y}_{<i}).\end{aligned} \tag{11.19}$$

Như vậy, chuỗi "tốt nhất" được sinh đến bước i là:

$$\mathbf{y}^{\text{top1}} \;=\; y_1 \cdots y_{i-1} y_i^{\text{top1}}. \tag{11.20}$$

Cuối cùng, Y_i chỉ chứa chuỗi này:

$$Y_i \;=\; \{\mathbf{y}^{\text{top1}}\}. \tag{11.21}$$

Lựa chọn tham lam trong một bước giải mã được minh họa trong Hình 11.5 (a). Tìm kiếm tham lam mang lại hiệu suất tính toán cao và dễ cài đặt. Tuy nhiên, nhược điểm chính của nó là tính chất không tối ưu — các chuỗi chất lượng cao rất có thể bị tỉa mất ở những giai đoạn đầu của giải mã. Vì vậy, tìm kiếm tham lam hấp dẫn với các nhiệm vụ đòi hỏi tốc độ và sự đơn giản. Với các nhiệm vụ cần kết quả tìm kiếm tốt hơn, nên dùng các chiến lược thay thế như tìm kiếm chùm, vốn khám phá đồng thời nhiều đường đi tiềm năng.

Chú giải của người dịch: Vì sao "tham lam" có thể bỏ lỡ chuỗi tốt nhất?

Giả sử ở bước 1, token A có xác suất 0.6 và B có 0.4. Tham lam chọn A. Nhưng nếu sau A, token tốt nhất chỉ có xác suất 0.3 (tổng: $0.6 \times 0.3 = 0.18$), còn sau B có một token với xác suất 0.9 (tổng: $0.4 \times 0.9 = 0.36$), thì chuỗi bắt đầu bằng B mới là tốt nhất. Tham lam đã "tỉa" mất B từ bước đầu và không bao giờ quay lại được — giống như leo núi luôn chọn lối dốc nhất trước mắt mà có thể không lên được đỉnh cao nhất.

2. Giải mã chùm

Tìm kiếm chùm (*beam search*) (hay giải mã chùm) là một sự mở rộng tự nhiên của tìm kiếm tham lam. Thay vì chỉ giữ lại một chuỗi có xác suất cao nhất ở mỗi bước, tìm kiếm chùm duy trì một số lượng cố định các chuỗi ứng viên tốt nhất, gọi là "độ rộng chùm (*beam width*)" K. Xem Hình 11.5 (b) để thấy minh họa.

Khác với tìm kiếm tham lam vốn chỉ mở rộng một nút cha, tìm kiếm chùm xét các khả năng mở rộng từ tất cả K chuỗi được lưu trong Y_{i-1}. Tại bước i, ta đánh giá mọi khả năng tiếp nối bằng cách tính log-xác suất của toàn chuỗi cho tập $Y_{i-1} \times V$. K chuỗi có xác suất tích lũy cao nhất sau đó được chọn để tạo thành Y_i:

$$\begin{aligned} Y_i &= \{\mathbf{y}^{\text{top1}}, \cdots, \mathbf{y}^{\text{top}K}\} \\ &= \underset{\mathbf{y} \in Y_{i-1} \times V}{\text{argTopK}} \log \Pr(\mathbf{y}|\mathbf{x}), \end{aligned} \tag{11.22}$$

trong đó argTopK là một hàm xếp hạng xác suất của mọi chuỗi đã đánh giá và trả về K ứng viên đứng đầu. Điều chỉnh độ rộng chùm K cho phép ta cân bằng giữa hiệu suất tìm kiếm và độ chính xác. Dù một chùm lớn

hơn khám phá không gian tìm kiếm rộng hơn, mức cải thiện hiệu năng thường giảm dần. Trong suy luận LLM thực tế, một độ rộng chùm tương đối nhỏ, chẳng hạn từ $K = 2$ đến $K = 5$, thường đủ để sinh văn bản chất lượng cao. Lưu ý rằng dù tìm kiếm chùm từng đóng vai trò quan trọng trong các nhiệm vụ sinh chuỗi như dịch máy, các phương pháp giải mã dựa trên lấy mẫu hiện nay được dùng phổ biến hơn trong sinh văn bản mở bằng LLM vì chúng tạo ra đầu ra đa dạng và linh hoạt hơn.

3. Giải mã dựa trên lấy mẫu

Cả tìm kiếm tham lam và tìm kiếm chùm đều sinh ra đầu ra tất định (*deterministic*), tức là với một LLM cho trước, đầu ra của mô hình sẽ luôn giống nhau mỗi lần nó xử lý cùng một đầu vào. Tính tất định của tìm kiếm tham lam và tìm kiếm chùm bảo đảm tính dự đoán được và độ tin cậy trong các ứng dụng mà kết quả nhất quán là quan trọng, chẳng hạn sinh văn bản hành chính, nơi đầu ra thay đổi có thể gây nhầm lẫn hoặc lỗi. Mặt khác, một nhược điểm của các phương pháp này là thiếu tính đa dạng và linh hoạt. Ví dụ, trong các nhiệm vụ sáng tạo như sinh truyện hay các tác tử hội thoại, những đầu ra chung chung hoặc lặp lại do hệ thống tất định sinh ra thường kém hấp dẫn.

Để đưa sự biến thiên vào đầu ra của LLM, ta có thể dùng các phương pháp giải mã dựa trên lấy mẫu (*sampling*). Có hai phương pháp thường dùng.

- **Lấy mẫu top-k** (*top-k sampling*). Phương pháp này chọn token tiếp theo từ k ứng viên có khả năng cao nhất ở mỗi bước của quá trình sinh [Fan và cộng sự, 2018]. Gọi $\overline{V}_i$ là tập chọn (*selection pool*) cho lấy mẫu top-k. Ta có thể định nghĩa nó là:

$$\overline{V}_i \;=\; \{y_i^{\text{top1}}, \cdots, y_i^{\text{top}k}\}, \tag{11.23}$$

  trong đó $\{y_i^{\text{top1}}, \cdots, y_i^{\text{top}k}\}$ là k token đứng đầu được chọn dựa trên xác suất dự đoán của chúng (xem Phương trình (11.22)). Khi đã xác định được tập chọn, ta chuẩn hóa lại (*renormalize*) phân phối xác suất trên $\overline{V}_i$. Một trong những cách đơn giản nhất để làm việc này là chuẩn hóa lại các xác suất của chúng:

$$\overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i})}{\sum_{y_j \in \overline{V}_i} \Pr(y_j|\mathbf{x}, \mathbf{y}_{<i})}. \tag{11.24}$$

Ngoài ra, ta có thể tính phân phối bằng hàm Softmax:

$$\overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\exp(u_{y_i})}{\sum_{y_j \in \overline{V}_i} \exp(u_{y_j})}, \tag{11.25}$$

trong đó u_{y_i} là logit (*logit*) của token y_i. Sau đó, ta lấy mẫu một token $\bar{y}_i$ từ phân phối này:

$$\bar{y}_i \;\sim\; \overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}). \tag{11.26}$$

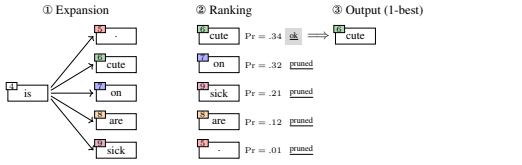
Chuỗi tương ứng là $\bar{\mathbf{y}} = y_1 \cdots y_{i-1}\bar{y}_i$, và Y_i được cho bởi:

$$Y_i \;=\; \{\bar{\mathbf{y}}\}. \tag{11.27}$$

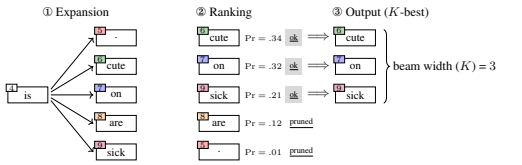
- **Lấy mẫu top-p** (*top-p sampling*). Phương pháp lấy mẫu này, còn gọi là **lấy mẫu hạt nhân** (*nucleus sampling*), theo một quy trình tương tự lấy mẫu top-k. Thay vì rút từ một tập ứng viên có kích thước cố định, nó chọn token tiếp theo từ tập token nhỏ nhất có xác suất tích lũy cùng nhau lớn hơn một ngưỡng p định trước [Holtzman và cộng sự, 2020b]. Bằng cách này, ta ngăn việc dự đoán chọn phải các token xác suất thấp ở "đuôi dài (*long tail*)" vốn có thể dẫn đến đầu ra thiếu mạch lạc hoặc vô nghĩa. Để thu được tập ứng viên trong phương pháp lấy mẫu top-p, ta có thể sắp xếp mọi token theo xác suất dự đoán. Sau đó, bắt đầu từ token có xác suất cao nhất, ta tiếp tục thêm token vào tập ứng viên cho đến khi xác suất tích lũy của các token trong tập đạt hoặc vượt p (ta ký hiệu kích thước của tập ứng viên lúc này là k_p). Tập ứng viên khi đó có thể được biểu diễn là:
$$\overline{V}_i \;=\; \{y_i^{\text{top1}}, \cdots, y_i^{\text{top}k_p}\}. \tag{11.28}$$
Các bước tiếp theo, như chuẩn hóa lại phân phối và lấy mẫu, giống như trong phương pháp lấy mẫu top-k (xem Phương trình (11.24–11.27)).

Chú giải của người dịch: Tính tay ví dụ trong Hình 11.5

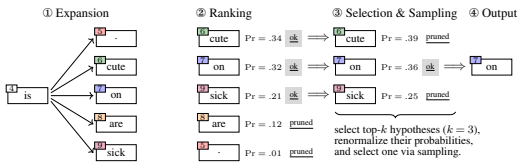
Sau tiền tố "*is*", năm token có xác suất: *cute* 0.34, *on* 0.32, *sick* 0.21, *are* 0.12, "." 0.01. *Top-k với* $k = 3$: giữ *cute, on, sick*, tổng $0.34 + 0.32 + 0.21 = 0.87$; chuẩn hóa lại theo Phương trình (11.24): $0.34/0.87 \approx 0.39$,



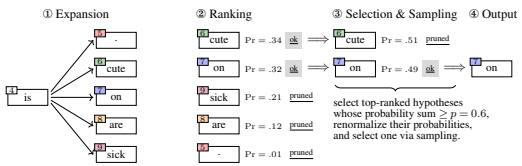
(a) Greedy search



(b) Beam search



(c) Top-k Sampling



(d) Top-p Sampling

Hình 11.5: Minh họa các phương pháp giải mã tham lam, giải mã chùm, giải mã top-k và giải mã top-p (trong một bước giải mã). (a) Tìm kiếm tham lam; (b) Tìm kiếm chùm; (c) Lấy mẫu top-k; (d) Lấy mẫu top-p. Các bước: *Expansion* (mở rộng), *Ranking* (xếp hạng), *Selection & Sampling* (chọn lọc và lấy mẫu), *Output* (đầu ra; *1-best*: một chuỗi tốt nhất, K*-best*: K chuỗi tốt nhất); *ok*: giữ lại, *pruned*: bị tỉa; *beam width* $(K) = 3$: độ rộng chùm $K = 3$. Ghi chú ở (c): chọn k giả thuyết đứng đầu ($k = 3$), chuẩn hóa lại xác suất của chúng, rồi chọn một giả thuyết bằng lấy mẫu. Ghi chú ở (d): chọn các giả thuyết xếp hạng cao nhất có tổng xác suất $\geq p = 0.6$, chuẩn hóa lại xác suất của chúng, rồi chọn một giả thuyết bằng lấy mẫu.

$0.32/0.87 \approx 0.37, 0.21/0.87 \approx 0.24$ (hình ghi các giá trị xấp xỉ $0.39, 0.36, 0.25$). Rồi "gieo xúc xắc" theo các xác suất này, ví dụ ra *on*. *Top-p với* $p = 0.6$: cộng dần 0.34, rồi $0.34 + 0.32 = 0.66 \geq 0.6$ nên dừng, tập ứng viên là *cute, on* ($k_p = 2$); chuẩn hóa lại: $0.34/0.66 \approx 0.51$, $0.32/0.66 \approx 0.49$. Điểm khác biệt: k cố định, còn k_p tự co giãn theo hình dạng phân phối. Ký hiệu $\bar{y}_i \sim \overline{\Pr}(\cdot)$ đọc là "$\bar{y}_i$ được rút ngẫu nhiên theo phân phối $\overline{\Pr}$".

Xem Hình 11.5 (c–d) để thấy minh họa các phương pháp lấy mẫu top-k và top-p. Bằng cách giới hạn lựa chọn vào một tập nhỏ hơn các token có xác suất cao, các phương pháp này đạt được sự cân bằng giữa tính ngẫu nhiên và tính mạch lạc. Chúng cho phép tạo ra đầu ra đa dạng hơn trong khi vẫn duy trì mức độ phù hợp và trôi chảy hợp lý. Tuy nhiên, giá trị của k hay p phải được chọn cẩn thận: nếu k hay p quá nhỏ, đầu ra vẫn có thể quá tất định (giống giải mã tham lam hơn), còn nếu k hay p quá lớn, LLM có thể sinh ra đầu ra suy biến (*degenerate*).

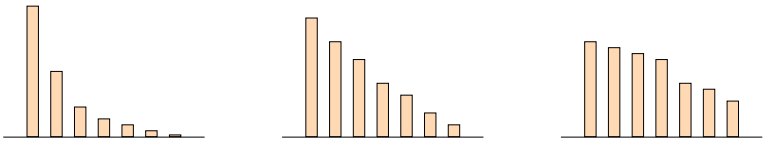
Để kiểm soát thêm tính ngẫu nhiên của quá trình chọn token, phân phối đã chuẩn hóa lại $\overline{\Pr}(\cdot)$ thường được tính bằng hàm Softmax với tham số nhiệt độ (*temperature*), cho bởi:

$$\overline{\Pr}(y_i|\mathbf{x}, \mathbf{y}_{<i}) \;=\; \frac{\exp(u_{y_i}/\beta)}{\sum_{y_j \in \overline{V}_i} \exp(u_{y_j}/\beta)}, \tag{11.29}$$

Ở đây β là tham số nhiệt độ điều khiển độ "nhọn" của phân phối xác suất được suy ra từ các logit. Trong Hình 11.6, chúng tôi đưa ra các ví dụ đơn giản về các phân phối được hàm trên sinh ra với các nhiệt độ khác nhau. Khi nhiệt độ được đặt ở giá trị cao hơn, phân phối xác suất thu được trở nên đều hơn, vì chênh lệch giữa các logit bị thu hẹp. Điều này có nghĩa là mỗi token trong tập ứng viên có cơ hội được chọn ngang nhau hơn, dẫn đến đầu ra đa dạng hơn. Ngược lại, khi nhiệt độ được đặt ở giá trị thấp hơn, phân phối trở nên nhọn hơn, khiến các token xác suất cao càng dễ được chọn, thường dẫn đến đầu ra tất định hơn. Ví dụ, nếu đặt p bằng 1 và β bằng một số rất nhỏ (tiến về 0), phương pháp lấy mẫu top-p sẽ trở nên tương đương với phương pháp tìm kiếm tham lam.

Chú giải của người dịch: Nhiệt độ β hoạt động thế nào?

Lấy hai token có logit $u_1 = 2$, $u_2 = 1$. Với $\beta = 1$: $\Pr_1 = e^2/(e^2 + e^1) \approx 0.73$. Với $\beta = 0.5$ (chia cho 0.5 tức nhân đôi logit thành 4 và 2):



(a) $\beta = 0.1$ (b) $\beta = 0.8$ (c) $\beta = 2.0$

Hình 11.6: Ước lượng dạng biểu đồ tần suất của các phân phối được sinh ra bởi hàm Softmax với các giá trị khác nhau của tham số nhiệt độ β.

$\Pr_1 = e^4/(e^4 + e^2) \approx 0.88$ — nhọn hơn. Với $\beta = 2$ (logit thành 1 và 0.5): $\Pr_1 \approx 0.62$ — đều hơn. Khi $\beta \to 0$, token có logit lớn nhất chiếm gần trọn xác suất (giống tham lam); khi $\beta \to \infty$, mọi token gần như ngang nhau (chọn ngẫu nhiên hoàn toàn). Tên "nhiệt độ" mượn từ vật lý: nóng thì các phân tử chuyển động hỗn loạn, lạnh thì trật tự.

4. Giải mã với các số hạng phạt

Một cải tiến phổ biến cho các phương pháp giải mã trong sinh văn bản là sửa đổi mục tiêu tìm kiếm. Ví dụ, có thể thay giải mã cực đại hậu nghiệm (*maximum a posteriori, MAP*) bằng giải mã rủi ro Bayes tối thiểu (*minimum Bayes risk, MBR*) [Kumar và Byrne, 2004b], trong đó trọng tâm chuyển từ việc chọn một đầu ra có xác suất cao nhất sang chọn một đầu ra cực tiểu hóa rủi ro kỳ vọng trên một phân phối các đầu ra có thể có. Chi tiết hơn về giải mã MBR có thể tìm thấy ở Chương 5. Ở đây chúng ta khám phá các phương pháp đưa các số hạng phạt (*penalty term*) vào giải mã. Các phương pháp này cung cấp một cách đơn giản mà hiệu quả để làm cho việc giải mã dễ kiểm soát hơn.

Nhắc lại từ Phương trình (11.14) rằng mục tiêu của giải mã là cực đại hóa độ hợp lý (*likelihood*) của chuỗi đầu ra. Với các số hạng phạt, mục tiêu được mở rộng để bao gồm các yếu tố bổ sung nhằm phạt những hành vi nhất định trong văn bản được sinh ra. Một dạng tổng quát của mục tiêu mới được cho bởi:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y} \in \mathcal{Y}} \left[\log \Pr(\mathbf{y}|\mathbf{x}) - \lambda \cdot \text{Penalty}(\mathbf{x}, \mathbf{y}) \right], \tag{11.30}$$

trong đó $\text{Penalty}(\mathbf{x}, \mathbf{y})$ là một hàm định lượng mức độ mà chuỗi được sinh $\mathbf{y}$ vi phạm một số ràng buộc nhất định hoặc thể hiện những hành vi không

mong muốn khi biết đầu vào $\mathbf{x}$. Việc thiết kế $\text{Penalty}(\cdot)$ rất linh hoạt, cho phép ta đưa vào nhiều loại ràng buộc hoặc tri thức tiên nghiệm khác nhau. Dưới đây, chúng tôi trình bày một số loại hàm phạt thường gặp.

- **Phạt lặp lại** (*repetition penalty*). Một hàm phạt lặp lại (*repetition penalty*) ngăn mô hình sinh văn bản lặp lại hoặc dư thừa. Hàm phạt có thể đo tần suất các token hoặc cụm từ bị lặp lại trong chuỗi được sinh và áp một mức phạt tỉ lệ với số lần xuất hiện của chúng.

- **Phạt độ dài** (*length penalty*). Một hàm phạt độ dài (*length penalty*) bảo đảm chuỗi được sinh tuân theo một độ dài mong muốn. Ví dụ, trong các nhiệm vụ tóm tắt văn bản, hàm phạt có thể phạt những đầu ra quá ngắn hoặc quá dài.

- **Phạt đa dạng** (*diversity penalty*). Một hàm phạt đa dạng (*diversity penalty*) thúc đẩy sự biến thiên trong văn bản được sinh. Ví dụ, trong tìm kiếm chùm, ta có thể đo độ tương đồng giữa các giả thuyết được sinh ra và khuyến khích mô hình khám phá các giả thuyết khác nhau.

- **Phạt dựa trên ràng buộc** (*constraint-based penalty*). Một hàm phạt dựa trên ràng buộc (*constraint-based penalty*) áp đặt các ràng buộc cụ thể liên quan đến nội dung hoặc văn phong của văn bản được sinh. Ví dụ, trong dịch máy, hàm phạt có thể phạt những đầu ra lệch khỏi một giọng văn hoặc hệ thuật ngữ mong muốn.

Nói chung, ta có thể xem $\text{Penalty}(\mathbf{x}, \mathbf{y})$ là một hàm xác định chi phí gán cho chuỗi được sinh $\mathbf{y}$ khi biết chuỗi đầu vào $\mathbf{x}$. Ngoài ra, hàm này cũng có thể được định nghĩa để đánh giá các trạng thái ẩn của LLM khi sinh $\mathbf{y}$. Ví dụ, Su và cộng sự [2022] xây dựng một số hạng phạt tính khoảng cách lớn nhất giữa biểu diễn của token được dự đoán và biểu diễn của các token đã sinh trước đó. Nhờ vậy, mục tiêu tìm kiếm sẽ phạt các đầu ra suy biến, chẳng hạn văn bản có nhiều lặp lại.

Phương pháp mô tả trong Phương trình (11.30) mang tính tổng quát và có thể dễ dàng điều chỉnh cho các thuật toán tìm kiếm khác nhau. Ví dụ, trong tìm kiếm tham lam, ta có thể giữ chuỗi duy nhất cực đại hóa $\log \Pr(\mathbf{y}|\mathbf{x}) - \lambda \cdot \text{Penalty}(\mathbf{x}, \mathbf{y})$ ở mỗi bước giải mã; trong tìm kiếm dựa trên lấy mẫu, ta có thể xếp hạng và chọn các chuỗi đứng đầu dựa trên $\log \Pr(\mathbf{y}|\mathbf{x}) - \lambda \cdot \text{Penalty}(\mathbf{x}, \mathbf{y})$ để tạo thành tập ứng viên.

Chú giải của người dịch: Số hạng phạt giống "trừ điểm" khi chấm thi

Phương trình (11.30) giống cách chấm một bài văn: điểm nội dung ($\log \Pr$) trừ đi điểm phạt cho lỗi (lặp ý, quá dài, lạc đề), nhân với hệ số $\lambda \geq 0$ cho biết ta coi lỗi nặng đến đâu. Ví dụ, chuỗi A có $\log \Pr = -5$ nhưng lặp từ 3 lần (phạt $= 3$), chuỗi B có $\log \Pr = -6$ và không lặp (phạt $= 0$). Với $\lambda = 0.5$: A được $-5 - 1.5 = -6.5$, B được -6, nên B thắng dù xác suất thấp hơn. Với $\lambda = 0$, ta quay về giải mã thông thường.

5. Giải mã suy đoán

Giải mã suy đoán (*speculative decoding*) bắt nguồn từ khái niệm **thực thi suy đoán** (*speculative execution*), trong đó một hệ thống đưa ra những phỏng đoán có cơ sở về các hành động trong tương lai và thực hiện chúng trước. Nếu phỏng đoán đúng, kết quả có sẵn ngay lập tức, giúp tăng tốc xử lý. Trong trường hợp suy luận LLM, giả sử ta có hai mô hình. Một là mô hình nhỏ hơn, nhanh hơn (gọi là mô hình nháp (*draft model*)), và mô hình kia là mô hình đầy đủ, mạnh hơn (gọi là mô hình kiểm chứng (*verification model*)). Hai mô hình này đại diện cho hai đầu của phổ suy luận LLM: mô hình nháp hiệu quả nhưng không chính xác lắm; mô hình kiểm chứng thường là mô hình ta muốn chạy, nhưng nó rất chậm. Cho một tiền tố, trước hết ta dùng mô hình nháp để dự đoán một cách suy đoán một chuỗi các token tương lai có khả năng xảy ra. Đây là một quá trình giải mã tự hồi quy chuẩn, nhưng trong thực tế vẫn nhanh nhờ hiệu suất cao của mô hình nháp. Sau đó, mô hình kiểm chứng đánh giá song song các token đã suy đoán. Nó kiểm tra xem các token dự đoán có thể được chấp nhận theo mô hình kiểm chứng hay không. Lưu ý rằng vì ta có thể xử lý các token này cùng một lúc, việc kiểm chứng có thể được thực hiện trong một bước duy nhất cho mọi token đồng thời, thay vì theo từng token một. Nếu các token suy đoán là đúng, chúng được chấp nhận và quá trình tiếp tục với nhóm token tiếp theo. Nếu chúng sai, các suy đoán sai bị loại bỏ, và mô hình kiểm chứng được dùng để sinh token mới.

Chú giải của người dịch: Ý tưởng của giải mã suy đoán qua một ví dụ đời thường

Hãy hình dung một học sinh (mô hình nháp) viết nháp nhanh 5 từ tiếp theo của bài văn, rồi thầy giáo (mô hình kiểm chứng) đọc cả 5 từ *một lượt* và gạch từ chỗ sai đầu tiên trở đi. Nếu học sinh đoán đúng 3 từ đầu, thầy chỉ cần một lượt đọc để "duyệt" 3 từ và tự viết thêm từ thứ 4, thay vì phải tự viết lần lượt từng từ trong 4 lượt. Phép tăng tốc có được vì với LLM lớn, chạy một lượt trên 5 token song song tốn thời gian gần bằng chạy một lượt cho 1 token (giai đoạn giải mã bị giới hạn bởi bộ nhớ, như đã nói ở Mục 11.1.2).

Cụ thể hơn, hãy xem phương pháp giải mã suy đoán được trình bày trong công trình của Leviathan và cộng sự [2023]. Trong phương pháp này, mô hình nháp là một mô hình ngôn ngữ nhỏ, ký hiệu là $\Pr_q(y_i|\mathbf{x}, \mathbf{y}_{<i})$, còn mô hình kiểm chứng là LLM đầy đủ, ký hiệu là $\Pr_p(y_i|\mathbf{x}, \mathbf{y}_{<i})$. Mục tiêu là dùng mô hình nháp, khi biết một tiền tố, để dự đoán tự hồi quy tối đa τ token. Mô hình kiểm chứng sau đó được dùng để sinh token cuối cùng tại điểm mà các dự đoán suy đoán bị bác bỏ. Hình 11.7 minh họa một bước trong quá trình giải mã này. Để đơn giản, chúng tôi mô tả phiên bản tham lam của giải mã suy đoán.

Thuật toán giải mã suy đoán có thể được tóm tắt như sau.

- Cho tiền tố $[\mathbf{x}, \mathbf{y}_{\leq i}]$, ta dùng mô hình nháp để dự đoán τ token liên tiếp tiếp theo, ký hiệu là $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+\tau}\}$. Đây là một quá trình sinh theo từng token, cho bởi:

$$\hat{y}_{i+t} = \arg\max_{y_{i+t}} \Pr_q(y_{i+t}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+t-1}). \tag{11.31}$$

- Ta đánh giá $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+\tau}\}$ bằng mô hình kiểm chứng, tức là ta tính $\{\Pr_p(\hat{y}_{i+1}|\mathbf{x}, \mathbf{y}_{\leq i}), \cdots, \Pr_p(\hat{y}_{i+\tau}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+\tau-1})\}$. Lưu ý rằng ta có thể tính các xác suất này song song, nên bước kiểm chứng này hiệu quả.

- Ta xác định số lượng tối đa các token suy đoán được chấp nhận. Để ký hiệu gọn gàng, ta viết $\Pr_q(\hat{y}_{i+t}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+t-1})$ và $\Pr_p(\hat{y}_{i+t}|\mathbf{x}, \mathbf{y}_{\leq i}, \hat{y}_{i+1} \cdots \hat{y}_{i+t-1})$ đơn giản lần lượt là $q(\hat{y}_{i+t})$ và $p(\hat{y}_{i+t})$. Sau đó ta định nghĩa quy tắc chấp nhận như sau: nếu $q(\hat{y}_{i+t}) \leq p(\hat{y}_{i+t})$ thì ta chấp nhận suy đoán này. Ngược lại, nếu $q(\hat{y}_{i+t}) > p(\hat{y}_{i+t})$, ta bác bỏ suy đoán này với xác suất $1 - \frac{p(\hat{y}_{i+t})}{q(\hat{y}_{i+t})}$.

Bắt đầu từ $\hat{y}_{i+1}$, số lượng tối đa các token suy đoán liên tiếp được chấp nhận được định nghĩa là:

$$n_a \;=\; \min\left\{ t-1 \;\middle|\; 1 \le t \le \tau,\; r_t > \frac{p(\hat{y}_{i+t})}{q(\hat{y}_{i+t})} \right\}, \tag{11.32}$$

trong đó r_t là một biến được rút từ phân phối đều (*uniform distribution*) $U(0,1)$. Nếu mọi token suy đoán đều được chấp nhận, ta đặt $n_a = \tau$.

- Cho n_a, ta giữ lại các token suy đoán $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+n_a}\}$. Sau đó ta dùng mô hình kiểm chứng để đưa ra một dự đoán mới tại vị trí $i+n_a+1$:

$$\bar{y}_{i+n_a+1} \;=\; \underset{y_{i+n_a+1}}{\arg\max}\, \underset{p}{\Pr}(y_{i+n_a+1} | \mathbf{x}, \mathbf{y}_{\le i}, \hat{y}_{i+1} \cdots \hat{y}_{i+n_a}). \tag{11.33}$$

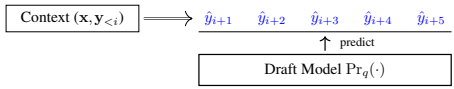
- Trên đây, chúng tôi đã mô tả một bước của giải mã suy đoán. Chuỗi kết quả (gồm cả ngữ cảnh và các token được dự đoán) được minh họa như sau

$$\underbrace{[\mathbf{x},\; \mathbf{y}_{<i}]}_{\text{Ngữ cảnh}} \quad \underbrace{\hat{y}_{i+1} \cdots \hat{y}_{i+n_a}}_{\substack{n_a \text{ token dự đoán} \\ \text{bằng mô hình nháp}}} \quad \underbrace{\bar{y}_{i+n_a+1}}_{\substack{\text{một token dự đoán} \\ \text{bằng mô hình kiểm chứng}}}$$

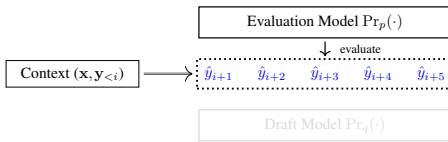
Khi đã hoàn tất bước này, ta thêm các token được dự đoán $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+n_a}, \bar{y}_{i+n_a+1}\}$ vào ngữ cảnh, và lặp lại quá trình trên.

Chú giải của người dịch: Đọc Phương trình (11.32)

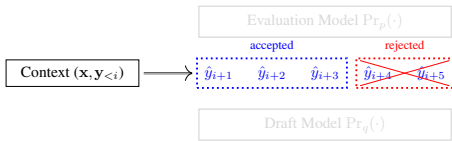
Với mỗi token nháp, ta so sánh hai ý kiến: mô hình nháp tin token đó với xác suất q, mô hình lớn tin với xác suất p. Nếu $q \le p$ (mô hình lớn còn tin hơn), ta chấp nhận ngay. Nếu $q > p$, ta "tung đồng xu": rút r_t ngẫu nhiên trong $[0,1]$, chấp nhận nếu $r_t \le p/q$. Ví dụ $q = 0.8$, $p = 0.4$ thì $p/q = 0.5$: chấp nhận với xác suất 50%. (Khi $q \le p$ thì $p/q \ge 1 \ge r_t$, nên điều kiện bác bỏ $r_t > p/q$ không bao giờ xảy ra — hai trường hợp được gộp gọn trong một công thức.) n_a là số token liên tiếp được chấp nhận trước token đầu tiên bị bác bỏ: nếu token thứ $t = 4$ là token đầu tiên bị bác bỏ thì $n_a = 4 - 1 = 3$, như trong Hình 11.7 (c).



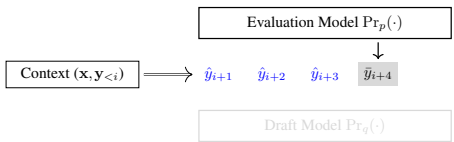
(a) Predict the next τ tokens given the context using the draft model ($\tau = 5$)



(b) Evaluate the predicted tokens using the evaluation model



(c) Determine the number of accepted tokens



(d) Predict a new token following the accepted tokens using the evaluation model

Hình 11.7: Minh họa một bước của giải mã suy đoán. Mục tiêu là dự đoán được càng nhiều token tiếp theo càng tốt bằng mô hình nháp. Có bốn bước con. Cho ngữ cảnh, trước hết ta dùng mô hình nháp để dự đoán τ token tiếp theo. Sau đó, ta đánh giá song song các dự đoán này bằng mô hình kiểm chứng. Tiếp theo, ta xác định số lượng tối đa các token dự đoán có thể được chấp nhận. Cuối cùng, ta dùng mô hình đánh giá để dự đoán một token mới theo sau các token đã được chấp nhận. Các nhãn con: (a) dự đoán τ token tiếp theo khi biết ngữ cảnh bằng mô hình nháp ($\tau = 5$); (b) đánh giá các token đã dự đoán bằng mô hình đánh giá; (c) xác định số token được chấp nhận; (d) dự đoán một token mới theo sau các token đã được chấp nhận bằng mô hình đánh giá. (*Context*: ngữ cảnh; *Draft Model*: mô hình nháp; *Evaluation Model*: mô hình đánh giá, tức mô hình kiểm chứng; *predict/evaluate*: dự đoán/đánh giá; *accepted/rejected*: được chấp nhận/bị bác bỏ.)

Trong thực tế, ta thường muốn dùng một mô hình nháp nhỏ hơn để việc dự đoán $\{\hat{y}_{i+1}, \cdots, \hat{y}_{i+n_a}\}$ rẻ hơn về mặt tính toán. Nhưng một mô hình nháp quá nhỏ sẽ kém chính xác và có thể dẫn đến n_a nhỏ hơn. Vì vậy ta cần chọn mô hình nháp cẩn thận để cân bằng giữa hiệu suất tính toán và độ chính xác.

6. Tiêu chí dừng

Tiêu chí dừng (*stopping criteria*) là một thành phần then chốt của suy luận LLM. Chúng thường gồm các quy tắc hoặc điều kiện quy định khi nào mô hình nên ngừng sinh văn bản trong lúc giải mã. Hầu hết các LLM được huấn luyện để sinh một token kết thúc chuỗi (ví dụ $\langle$EOS$\rangle$ hoặc $\langle$/s$\rangle$) để báo hiệu sự kết thúc của văn bản được sinh. Do đó, chiến lược đơn giản nhất là dừng quá trình sinh khi token này được tạo ra. Với tìm kiếm chùm, vốn khám phá nhiều giả thuyết đồng thời, quá trình có thể tiếp tục cho đến khi đã sinh được một số lượng chuỗi hoàn chỉnh cho trước.

Trong các ứng dụng thực tế, nói chung không mong muốn sinh ra các chuỗi rất dài, vì vậy ta cần giảm chi phí giải mã và sự dài dòng không cần thiết. Một tiêu chí dừng thường dùng là độ dài tối đa của đầu ra. Mô hình ngừng sinh văn bản khi đã tạo ra một số lượng token định trước. Ngoài ra, ta có thể dừng giải mã dựa trên chi phí tính toán thực tế, chẳng hạn tài nguyên tính toán hoặc ràng buộc thời gian. Ví dụ, trong các ứng dụng thời gian thực như chatbot, có thể cần dừng giải mã sau một giới hạn thời gian nhất định để bảo đảm khả năng phản hồi nhanh.

Một cách tiếp cận khác là thiết kế tiêu chí dừng dựa trên hành vi của LLM. Ví dụ, có thể dừng giải mã nếu xác suất dự đoán token tiếp theo giảm xuống dưới một ngưỡng nhất định. Ngoài việc dừng dựa trên xác suất, có thể cài đặt một mô-đun phát hiện lặp lại để buộc mô hình dừng nếu nó bắt đầu lặp lại các token hoặc cụm từ vượt quá một giới hạn định trước. Điều này giúp ngăn các đầu ra dư thừa hoặc thiếu mạch lạc.

11.1.4 Các thước đo đánh giá suy luận LLM

Đánh giá hiệu năng của các hệ thống suy luận LLM bao gồm nhiều thước đo khác nhau nhằm đánh giá mức độ các mô hình này đáp ứng các tiêu chuẩn mong muốn, như chất lượng, độ bền vững (*robustness*), tính khả dụng và hiệu suất. Như với hầu hết các hệ thống NLP, ta có thể đánh giá

LLM bằng các thước đo đánh giá NLP chuẩn, chẳng hạn độ hỗn loạn (*perplexity*) và điểm F1 (*F1 score*). Ta cũng có thể kiểm tra độ bền vững của chúng bằng cách thử xem chúng xử lý các đầu vào mơ hồ hoặc khó tốt đến đâu, bao gồm các ví dụ đối kháng, đầu vào bị nhiễu và dữ liệu ngoài phân phối (*out-of-distribution*). Ngoài ra, tính khả dụng có thể được đánh giá bằng cách đo mức độ đầu ra được sinh phù hợp với kỳ vọng của người dùng về độ trôi chảy, tính mạch lạc, tính liên quan và tính đa dạng. Người đánh giá có thể chấm độ tự nhiên của văn bản hoặc đánh giá xem các phản hồi có phù hợp với ngữ cảnh và nhất quán về mặt logic hay không. Các thước đo về đạo đức và công bằng cũng có thể được đưa vào để bảo đảm LLM tránh duy trì các thiên kiến hoặc sinh nội dung có hại.

Tất cả các thước đo đánh giá nêu trên chủ yếu tập trung vào việc đánh giá chất lượng đầu ra. Do chi phí triển khai và phục vụ LLM cao, các thước đo hiệu suất cũng rất quan trọng với người làm thực tế. Dưới đây là một số thước đo hiệu suất thường dùng [Nvidia, 2025]:

- **Độ trễ yêu cầu** (*request latency*). Thước đo này đo tổng thời gian tính từ khi một yêu cầu được gửi đến LLM cho đến khi nhận được phản hồi hoàn chỉnh. Nó bao gồm thời gian truyền dữ liệu, thời gian mô hình xử lý và thời gian trả đầu ra về cho người dùng.

- **Thông lượng** (*throughput*). Thước đo này đo số yêu cầu hoặc số token được sinh mà hệ thống có thể xử lý trong một đơn vị thời gian. Thông lượng thường được báo cáo dưới dạng số yêu cầu mỗi giây (RPS) hoặc số token mỗi giây (TPS).

- **Thời gian đến token đầu tiên** (*time to first token*, TTFT). Thước đo này đo khoảng thời gian giữa lúc gửi một yêu cầu và lúc sinh ra token đầu tiên của phản hồi. Nếu việc truyền dữ liệu không tốn quá nhiều thời gian, thì TTFT chủ yếu gồm thời gian cần cho giai đoạn điền trước và việc sinh token đầu tiên.

- **Độ trễ giữa các token** (*inter-token latency*, ITL). Thước đo này chỉ thời gian cần để sinh mỗi token tiếp theo sau token đầu tiên. Nó phản ánh hiệu suất của quá trình giải mã.

- **Mức sử dụng tài nguyên** (*resource utilization*). Thước đo này bao gồm việc đo mức sử dụng tài nguyên tính toán (chẳng hạn mức sử dụng CPU và GPU) và lượng bộ nhớ mô hình tiêu thụ trong lúc suy

luận.

Chú giải của người dịch: Liên hệ giữa TTFT, ITL và độ trễ yêu cầu

Khi trò chuyện với một chatbot, TTFT là lúc bạn chờ chữ đầu tiên hiện ra, còn ITL quyết định chữ "chạy" ra nhanh hay chậm. Bỏ qua thời gian truyền mạng, một phản hồi dài n token có độ trễ xấp xỉ TTFT $+ (n-1) \times$ ITL. Ví dụ, TTFT $= 0.5$ giây, ITL $= 0.02$ giây, phản hồi 201 token: độ trễ $\approx 0.5 + 200 \times 0.02 = 4.5$ giây. TTFT gắn với giai đoạn điền trước, ITL gắn với giai đoạn giải mã (Mục 11.1.2). Thông lượng thì khác: nó đo tổng "sản lượng" của cả hệ thống khi phục vụ nhiều người dùng cùng lúc.

Ngoài các thước đo này, hiệu quả năng lượng và hiệu quả chi phí là những cân nhắc thực tế khi triển khai LLM ở quy mô lớn. Hiệu quả năng lượng đo lượng điện năng mà mô hình tiêu thụ trong lúc suy luận. Hiệu quả chi phí, mặt khác, đánh giá tổng chi phí liên quan đến việc triển khai và duy trì mô hình.

Nói chung, việc chọn đúng thước đo đánh giá phụ thuộc vào nhiệm vụ và ứng dụng cụ thể. Dù các thước đo tập trung vào chất lượng là thiết yếu để đánh giá LLM, các thước đo hiệu suất cũng quan trọng không kém đối với việc triển khai hiệu quả trong các ứng dụng thực tế. Một khung đánh giá toàn diện nên bao gồm cả hai nhóm thước đo để đo chính xác hiệu năng và tính thực tiễn của một LLM.

11.2 Các kỹ thuật suy luận hiệu quả

Trong các ứng dụng thực tế, ta thường mong muốn một hệ thống càng hiệu quả càng tốt. Với suy luận (*inference*) của LLM, điều này thường gồm hai kiểu cải tiến: giảm yêu cầu bộ nhớ và tăng tốc suy luận. Chẳng hạn, ta có thể sửa đổi kiến trúc Transformer để tránh bùng nổ bộ nhớ khi xử lý các chuỗi đầu vào rất dài. Một ví dụ khác là ta có thể nén các chuỗi đầu vào để giảm chi phí tính toán mà vẫn giữ được thông tin ngữ nghĩa của chúng. Ngoài ra, các kỹ thuật như lượng tử hóa (*quantization*) và tỉa (*pruning*) có thể được dùng để tối ưu thêm mức dùng bộ nhớ và tốc độ suy luận.

Suy luận hiệu quả là một chủ đề rộng, giao thoa với nhiều hướng nghiên cứu LLM, chẳng hạn thiết kế kiến trúc và nén mô hình. Phần lớn

các chủ đề này đã được đề cập trong các chương trước. Ví dụ, ở Chương 6 ta đã bàn về các kiến trúc Transformer hiệu quả; ở Chương 8 ta đã bàn về các LLM ngữ cảnh dài; và ở Chương 9 ta đã bàn về các phương pháp nén prompt nhằm giảm độ dài prompt. Trong mục này, chúng ta tập trung vào các kỹ thuật thường được dùng khi triển khai và phục vụ LLM.

11.2.1 Lưu đệm tiền tố

Trong các ứng dụng thực tế, việc lưu đệm (*cache*) các yêu cầu thường gặp cùng các phản hồi tương ứng là rất phổ biến. Khi một yêu cầu mới *trúng* bộ nhớ đệm, hệ thống có thể lấy phản hồi trực tiếp từ bộ nhớ đệm thay vì tính lại kết quả. Một cách cài đặt đơn giản là dùng một kho dữ liệu khóa–giá trị (ví dụ một bảng băm) ánh xạ các chuỗi đầu vào tới các chuỗi đầu ra do LLM sinh ra. Trong trường hợp đơn giản nhất, ta có thể thu thập các truy vấn thường gặp, dùng LLM sinh phản hồi cho chúng, rồi lưu các cặp truy vấn–phản hồi này vào kho dữ liệu. Điều này tạo ra một cơ chế lưu đệm cơ bản ở mức chuỗi, cho phép hệ thống bỏ qua tính toán của LLM khi chuỗi đầu vào khớp chính xác với một truy vấn đã được lưu đệm.

Một mở rộng hiển nhiên của cơ chế lưu đệm là lưu đệm các tiền tố cùng các trạng thái bộ nhớ đệm KV (*KV cache*) tương ứng của chúng. Cho một chuỗi đầu vào $\mathbf{x}$ trong tập dữ liệu $\mathcal{D}$, ta có thể xử lý nó như trong pha điền trước (*prefilling*) tiêu chuẩn. Việc này tạo ra một dãy các tiền tố và các trạng thái bộ nhớ đệm KV tương ứng:

$$\begin{aligned} x_0 \text{ (tức là } \mathbf{x}_{<1}) &\Rightarrow \text{cache}_{<1} \\ x_0x_1 \text{ (tức là } \mathbf{x}_{<2}) &\Rightarrow \text{cache}_{<2} \\ &\cdots \\ x_0x_1\cdots x_{m-1} \text{ (tức là } \mathbf{x}_{<m}) &\Rightarrow \text{cache}_{<m} \end{aligned}$$

trong đó $\text{cache}_{<i}$ ký hiệu bộ nhớ đệm KV của tiền tố $\mathbf{x}_{<i}$ (xem thêm Phương trình (11.10)). Tất cả các ánh xạ này có thể được lưu trong bộ nhớ đệm tiền tố để tái sử dụng hiệu quả.

Khi xử lý một chuỗi mới có chung tiền tố với một chuỗi đã gặp trước đó trong $\mathcal{D}$, ta có thể nạp các trạng thái KV đã lưu đệm tương ứng thay vì tính lại chúng. Cụ thể, nếu một đầu vào mới $\mathbf{x}'$ có chung tiền tố $\mathbf{x}_{<k}$ với một chuỗi sẵn có (tức là $\mathbf{x}'_{<k} = \mathbf{x}_{<k}$ với một $k \leq m$ nào đó), ta có thể

khởi tạo bộ nhớ đệm KV bằng cache$_{<k}$ và chỉ cần tính các trạng thái KV cho các token còn lại $\mathbf{x}'_{\geq k}$.

Như thường lệ, ta có thể duy trì một kho dữ liệu khóa–giá trị ánh xạ các tiền tố hay gặp tới các bộ nhớ đệm KV đã được tính sẵn của chúng. Việc tra cứu có thể thực hiện bằng giá trị băm của các token tiền tố, cho phép truy cập các trạng thái đã lưu đệm trong thời gian hằng số. Cần chú ý quản lý mức dùng bộ nhớ, vì lưu tất cả các tiền tố có thể có là điều không khả thi với các tập dữ liệu lớn. Các hệ thống thực tế thường dùng phương pháp lưu đệm ít được dùng gần đây nhất (*least recently used, LRU*) hoặc các chiến lược khác để cân bằng giữa lượng tính toán tiết kiệm được và giới hạn bộ nhớ.

Chú giải của người dịch: Lưu đệm tiền tố trong đời thường

Hãy hình dung nhiều người dùng cùng gửi cho một chatbot những prompt bắt đầu bằng cùng một đoạn chỉ dẫn hệ thống dài 1000 token ("Bạn là trợ lý hữu ích, hãy trả lời ngắn gọn…"), chỉ khác nhau ở câu hỏi cuối dài khoảng 20 token. Nếu đã lưu sẵn bộ nhớ đệm KV của 1000 token chung đó, mỗi yêu cầu mới chỉ cần tính thêm cho 20 token, tức là bớt được khoảng $1000/1020 \approx 98\%$ công việc ở pha điền trước. *Bảng băm* là cấu trúc dữ liệu cho phép tìm "khóa" (ở đây là dãy token tiền tố) gần như tức thì, giống như tra từ điển theo số trang thay vì lật từng trang. *LRU* là quy tắc "khi đầy thì vứt bỏ mục lâu nhất không được dùng tới", giống như dọn tủ: món đồ lâu ngày không đụng tới sẽ bị cất đi trước.

11.2.2 Xử lý theo lô

Xử lý theo lô (*batching*) trong suy luận LLM là việc xử lý đồng thời nhiều chuỗi đầu vào như một nhóm (gọi là một lô (*batch*)) thay vì xử lý lần lượt từng chuỗi. Vì các GPU hiện đại rất mạnh về xử lý song song, xử lý theo lô cho phép chúng tính toán nhiều chuỗi trong một lượt truyền xuôi (*forward pass*) duy nhất, giữ cho phần cứng luôn được dùng hết công suất. Do đó, khi phục vụ LLM ở quy mô lớn, xử lý theo lô rất quan trọng để nâng cao hiệu quả tính toán và tối đa hóa mức sử dụng phần cứng[2].

Để minh họa ý tưởng xử lý theo lô, Hình 11.8 (a–b) trình bày các ví

[2] Xem `https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html#understand-perf` để có một đánh giá đơn giản.

dụ đơn giản với kích thước lô lần lượt là 1 và 4. Khi dùng kích thước lô bằng 1 (tức là không xử lý theo lô), GPU xử lý từng chuỗi đầu vào một. Như vậy, việc xử lý là tuần tự: chuỗi tiếp theo phải chờ phép tính hiện tại kết thúc. Ngược lại, khi dùng kích thước lô bằng 4, GPU có thể xử lý đồng thời bốn chuỗi trong một lượt truyền xuôi duy nhất. Vì các chuỗi đầu vào có độ dài khác nhau, ta cần chuẩn hóa độ dài của chúng bằng các kỹ thuật đệm (*padding*). Ở đây ta dùng đệm bên trái (*left padding*): thêm các token giả vào đầu các chuỗi ngắn, để mọi chuỗi trong lô có cùng độ dài ở pha điền trước. Ở pha giải mã (*decoding*), các token được sinh đồng thời cho tất cả các chuỗi này, và quá trình sinh tiếp tục cho đến khi mọi chuỗi trong lô đều giải mã xong.

Các ví dụ trên ngụ ý một sự đánh đổi giữa thông lượng (*throughput*) và độ trễ (*latency*), đây là một cân nhắc rất quan trọng khi thiết kế và cài đặt các hệ thống suy luận LLM. Nếu chọn kích thước lô nhỏ hơn, độ trễ sẽ thấp hơn, vì cần xử lý ít token hơn trong một lần chạy suy luận. Hãy hình dung ta chỉ có một chuỗi. Kết quả có ngay sau khi quá trình sinh kết thúc, không có thêm chi phí tính toán nào. Tuy nhiên, ưu điểm độ trễ thấp này phải trả giá bằng việc dùng không hết tài nguyên tính toán song song, vì khả năng song song của GPU phần lớn bị bỏ không trong quá trình xử lý tuần tự. Mặt khác, nếu dùng lô lớn hơn, ta có thể tận dụng tốt hơn khả năng song song, vì GPU được lấp đầy bởi các phép tính ma trận quy mô lớn. Nhờ đó, ta có thể xử lý nhiều token hơn trong cùng một khoảng thời gian và thông lượng được cải thiện. Tuy nhiên, vì kết quả chỉ có được khi token cuối cùng trong lô được dự đoán, độ trễ sẽ cao hơn.

Trong thực tế, ta thường thích dùng lô hơi lớn một chút, nhưng cố gắng lấp đầy lô bằng các chuỗi có độ dài tương tự nhau để giảm số token đệm và cải thiện mức sử dụng thiết bị. Chẳng hạn, ta có thể gom các yêu cầu người dùng đến trong một khoảng thời gian ngắn vào các *thùng* (*bucket*), mỗi thùng chứa các chuỗi có độ dài tương tự nhau. Sau đó, ta lấp đầy lô bằng các chuỗi trong cùng một thùng, nhờ đó giảm thiểu tài nguyên tính toán bị lãng phí, như minh họa ở Hình 11.8 (c).

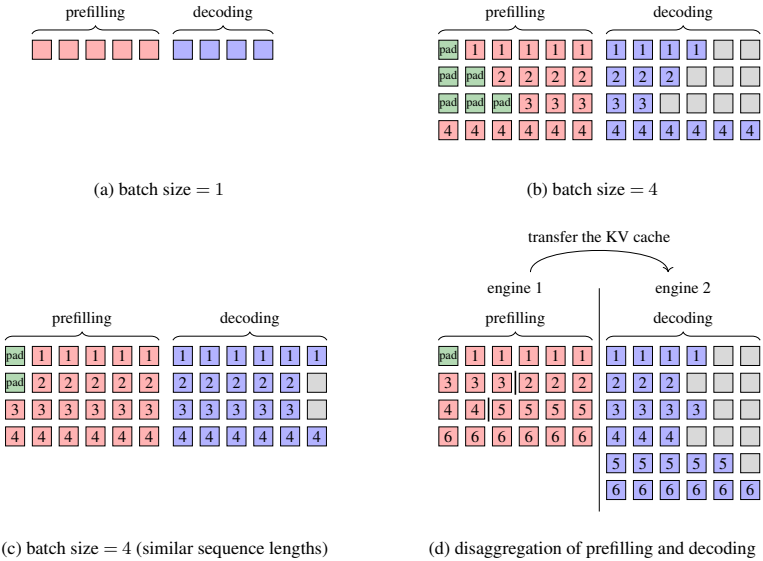
Một cách tiếp cận khác để cài đặt xử lý theo lô trong LLM là tách rời (*disaggregate*) quá trình điền trước và quá trình giải mã [Wu và cộng sự, 2023a; Patel và cộng sự, 2024; Zhong và cộng sự, 2024]. Ví dụ, ta có thể thực hiện điền trước trên một GPU và thực hiện giải mã trên một GPU khác. Một ưu điểm của việc tách rời là ta có thể sắp xếp lại các chuỗi đầu

vào trong lô để lấp đầy nó tốt hơn, vì không có sự can thiệp lẫn nhau giữa điền trước và giải mã. Chẳng hạn, ta có thể ghép nhiều chuỗi ngắn lại với nhau, qua đó bảo đảm độ dài của các chuỗi trong lô đồng đều nhất có thể, như minh họa ở Hình 11.8 (d). Bằng cách này, ta có thể tối đa hóa thông lượng của pha điền trước. Tuy nhiên, đổi lại, ta cần truyền bộ nhớ đệm KV tới các thiết bị thực hiện giải mã, điều này cũng phát sinh thêm chi phí truyền thông. Thông thường, phương pháp này cần một mạng băng thông cao, độ trễ thấp để đạt hiệu năng tối ưu.

Chú giải của người dịch: Thông lượng và độ trễ: ví dụ xe buýt

Hãy so sánh LLM với một chiếc xe buýt. *Độ trễ* là thời gian một hành khách phải chờ từ lúc lên xe đến lúc tới nơi; *thông lượng* là số hành khách được chở trong một giờ. Nếu xe chạy ngay khi có 1 khách (lô kích thước 1), khách đó tới nơi rất nhanh, nhưng xe gần như trống, lãng phí. Nếu xe đợi đủ 40 khách mới chạy (lô lớn), mỗi chuyến chở được nhiều người (thông lượng cao), nhưng ai cũng phải chờ lâu hơn và ai cũng chỉ "xong" khi xe tới bến cuối (độ trễ cao). Token đệm giống như những ghế trống trên xe: vẫn tốn chỗ và nhiên liệu nhưng không chở ai, nên gom các chuỗi dài tương tự nhau là cách bớt "ghế trống".

Trong mục này, chúng ta sẽ bàn về một số cải tiến cho các chiến lược xử lý theo lô cơ bản ở trên. Phần lớn chúng dựa trên một kiến trúc không tách rời, tức là giải mã và điền trước có thể được xem như các giai đoạn khác nhau của một mô hình được thực thi trên cùng một thiết bị.



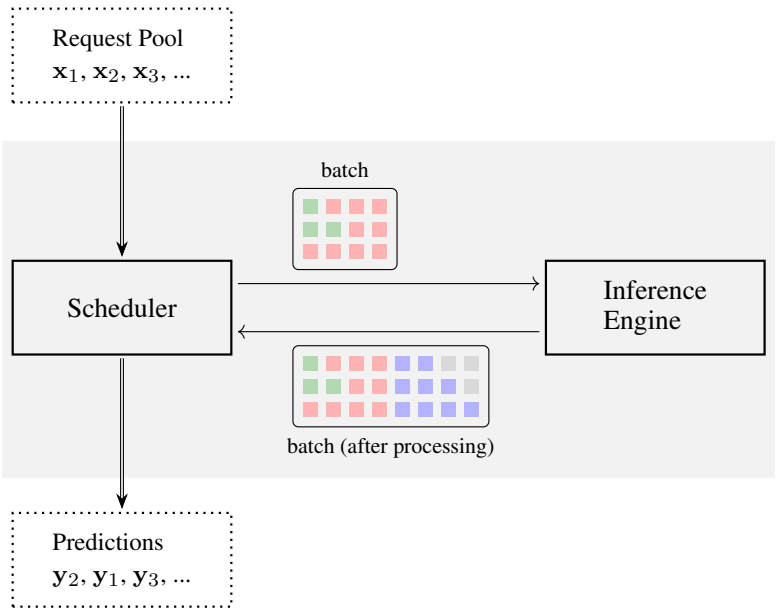
(a) batch size $= 1$

(b) batch size $= 4$

(c) batch size $= 4$ (similar sequence lengths)

(d) disaggregation of prefilling and decoding

Hình 11.8: Minh họa các phương pháp xử lý theo lô cơ bản. Chúng tôi dùng một bố cục 2D để minh họa lô, trong đó mỗi ô vuông biểu diễn một token. Các ô đỏ chỉ các token ở pha điền trước (*prefilling*), các ô xanh lam biểu diễn các token ở pha giải mã (*decoding*), các ô xanh lục ký hiệu các token đệm (*pad*), và các ô xám tương ứng với các token vô nghĩa. Hình con (a) và (b) so sánh các trường hợp kích thước lô (*batch size*) lần lượt bằng 1 và 4. Hình con (c) cho thấy chiến lược gom các chuỗi có độ dài tương tự nhau vào cùng một lô. Hình con (d) minh họa việc tách rời điền trước và giải mã. Trong cách tiếp cận này, ta có thể tận dụng tốt hơn khả năng song song của GPU bằng cách nối nhiều chuỗi ngắn thành một chuỗi dài duy nhất để xử lý chung. Điều này cho phép tối đa hóa số token được xử lý trong một lô đồng thời giảm thiểu số token đệm. Tuy nhiên, đổi lại, ta cần sao chép bộ nhớ đệm KV sang bộ máy giải mã (*engine 2*) và tổ chức lại nó sau pha điền trước, điều này đưa thêm chi phí truyền dữ liệu (*transfer the KV cache*: truyền bộ nhớ đệm KV).



Hình 11.9: Minh họa kiến trúc suy luận LLM gồm một bộ lập lịch (*Scheduler*) và một bộ máy suy luận (*Inference Engine*). Mỗi lần, bộ lập lịch chọn một số yêu cầu của người dùng từ kho yêu cầu (*Request Pool*) để tạo thành một lô và gửi nó tới bộ máy suy luận. Bộ lập lịch có thể tương tác với bộ máy suy luận và điều chỉnh lô tại một số thời điểm nhất định trong quá trình suy luận, chẳng hạn tại lúc bắt đầu xử lý lô và tại lúc bắt đầu mỗi bước dự đoán token. (*batch (after processing)*: lô sau khi xử lý; *Predictions*: các dự đoán.)

1. Lập lịch

Một hệ thống suy luận LLM thực tế thường gồm hai thành phần:

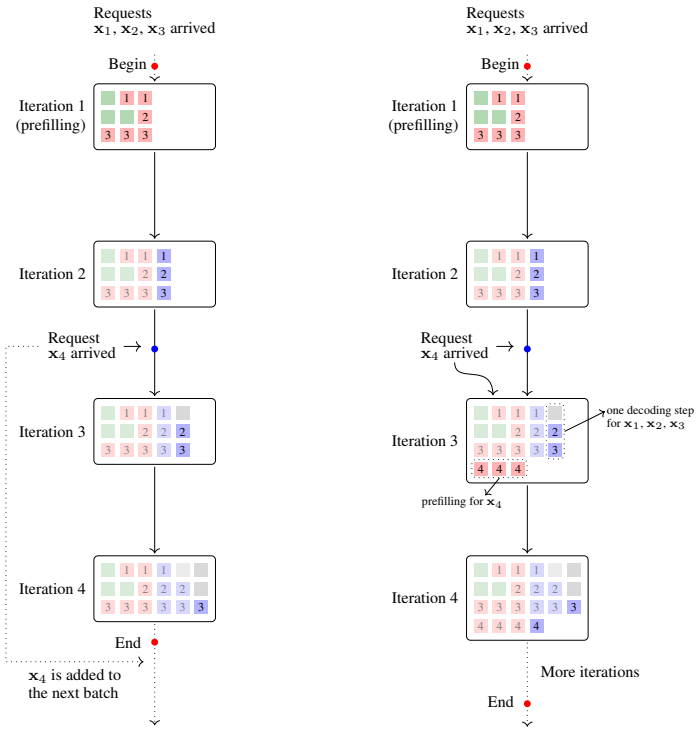
- **Bộ lập lịch** (*Scheduler*). Vai trò chính của nó là xếp hàng và phân phát các tác vụ (tức là các chuỗi đầu vào) tới bộ máy suy luận một cách hiệu quả, dựa trên tải hiện tại của hệ thống và mức ưu tiên của các yêu cầu. Việc này thường gồm nhiều chiến lược xử lý theo lô khác nhau, gom một số yêu cầu nhất định lại với nhau để tối đa hóa hiệu quả xử lý.

- **Bộ máy suy luận** (*Inference Engine*). Nó chịu trách nhiệm thực thi LLM trên thực tế, xử lý các yêu cầu trong hàng đợi khi chúng đến. Như đã bàn ở trên, bộ máy này gồm cả quá trình điền trước và quá trình giải mã.

Kiến trúc này được minh họa ở Hình 11.9. Đưa việc lập lịch vào xử lý theo lô mang lại một cách linh hoạt để tối ưu cả thông lượng lẫn độ trễ của hệ thống, qua đó đạt được sự cân bằng tốt hơn giữa hai đại lượng này. Ví dụ, các phương pháp xử lý theo lô ở Hình 11.8 (a) và (b) có thể được xem là những dạng lập lịch đơn giản, gọi là **lập lịch mức yêu cầu** (*request-level scheduling*). Trong chiến lược này, một khi lô đã được lấp đầy và gửi tới bộ máy, việc xử lý toàn bộ lô không thể bị ngắt quãng. Bộ lập lịch chờ lô này xử lý xong rồi mới xử lý lô tiếp theo [Timonin và cộng sự, 2022].

Một chiến lược lập lịch tinh vi hơn, gọi là **lập lịch mức vòng lặp** (*iteration-level scheduling*), tương tác với bộ máy suy luận ở mỗi bước dự đoán token thay vì ở mức chuỗi. Cách tiếp cận này cho phép điều chỉnh lô một cách động trong quá trình suy luận, như minh họa ở Hình 11.10.

Việc điều khiển chi tiết như vậy cho phép hệ thống ưu tiên các yêu cầu hoặc các chuỗi nhạy cảm với độ trễ. Chẳng hạn, nếu một yêu cầu khẩn cấp đến ở một bước giải mã nào đó, bộ lập lịch có thể thêm yêu cầu này vào lô để nó được xử lý sớm nhất có thể. Trong các tiểu mục sau, chúng ta sẽ bàn về các phương pháp xử lý theo lô dựa trên lập lịch mức vòng lặp.



(a) Request-level Scheduling

(b) Iteration-level Scheduling

Hình 11.10: Minh họa lập lịch mức yêu cầu và lập lịch mức vòng lặp. Trong lập lịch mức yêu cầu, một khi lô đã được tạo và gửi tới bộ máy suy luận, ta không thể điều chỉnh lô. Nói cách khác, việc lập lịch chỉ diễn ra sau khi xử lý xong một lô. Trong lập lịch mức vòng lặp, ta có thể lập lịch ngay trong lúc đang xử lý lô. Chẳng hạn, nếu một yêu cầu mới đến vào một thời điểm nào đó trong quá trình suy luận, ta có thể thêm nó vào lô và tiếp tục xử lý. (Nhãn trong hình: *Requests* $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ *arrived*: các yêu cầu $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ đến; *Iteration*: vòng lặp; *Begin/End*: bắt đầu/kết thúc; $\mathbf{x}_4$ *is added to the next batch*: $\mathbf{x}_4$ được đưa vào lô kế tiếp; *one decoding step for* $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$: một bước giải mã cho $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$; *prefilling for* $\mathbf{x}_4$: điền trước cho $\mathbf{x}_4$; *More iterations*: các vòng lặp tiếp theo.)

2. Xử lý theo lô liên tục

Xử lý theo lô liên tục (*continuous batching*) là một phương pháp lập lịch mức vòng lặp được dùng trong hệ thống Orca [Yu và cộng sự, 2022]. Trong phương pháp này, một vòng lặp là hoặc toàn bộ thủ tục điền trước, hoặc một bước giải mã đơn lẻ. Ví dụ, cho một chuỗi đầu vào $\mathbf{x} = x_0 \cdots x_m$ và một chuỗi đầu ra $\mathbf{y} = y_1 \cdots y_n$, có tổng cộng $n + 1$ vòng lặp: một vòng cho điền trước, và n vòng để sinh các token đầu ra (mỗi token một vòng). Trong khi lập lịch, lô có thể được điều chỉnh giữa các vòng lặp. Chẳng hạn, ở một vòng lặp nào đó, ta có thể thêm một chuỗi đầu vào mới vào lô, hoặc loại một chuỗi đã hoàn thành khỏi lô, ngay cả khi lô vẫn đang được xử lý.

Quy trình chung của xử lý theo lô liên tục gồm các bước sau:

- Ban đầu, một lô được tạo với một hoặc nhiều chuỗi đầu vào, dựa trên cả năng lực xử lý của bộ máy suy luận lẫn các yêu cầu hiện có của người dùng. Sau đó lô được đưa vào bộ máy suy luận.
- Bộ máy suy luận xử lý lô theo từng vòng lặp. Sau mỗi vòng lặp, bộ lập lịch có thể điều chỉnh lô theo một trong các cách sau:
  - Nếu một chuỗi trong lô hoàn thành việc sinh (tức là sinh ra ký hiệu kết thúc chuỗi), chuỗi đó bị loại khỏi lô.
  - Nếu một yêu cầu mới của người dùng đến và bộ máy suy luận còn dư năng lực xử lý, yêu cầu đó được thêm vào lô.
  - Nếu không có chuỗi nào được thêm vào hay loại khỏi lô, lô giữ nguyên.
- Quá trình xử lý chỉ kết thúc khi tất cả các chuỗi đã hoàn thành và không có yêu cầu mới nào của người dùng đến.

Xem Hình 11.11 để có một ví dụ về xử lý theo lô liên tục. Trong ví dụ này, ta bắt đầu với hai yêu cầu của người dùng, $\mathbf{x}_1$ và $\mathbf{x}_2$. Hai chuỗi này được gói vào một lô và gửi tới bộ máy suy luận để xử lý. Sau khi bộ máy hoàn thành hai vòng lặp, một yêu cầu mới của người dùng, $\mathbf{x}_3$, đến. Lúc này, bộ lập lịch điều chỉnh lô bằng cách thêm $\mathbf{x}_3$ vào. Bộ máy suy luận sau đó tiếp tục xử lý lô đã cập nhật. Lưu ý rằng lúc này bộ máy suy luận xử lý các chuỗi khác nhau theo những cách khác nhau: $\mathbf{x}_1$ và $\mathbf{x}_2$ tiếp tục quá trình giải mã (tức là dự đoán các token tiếp theo), trong khi $\mathbf{x}_3$ trải

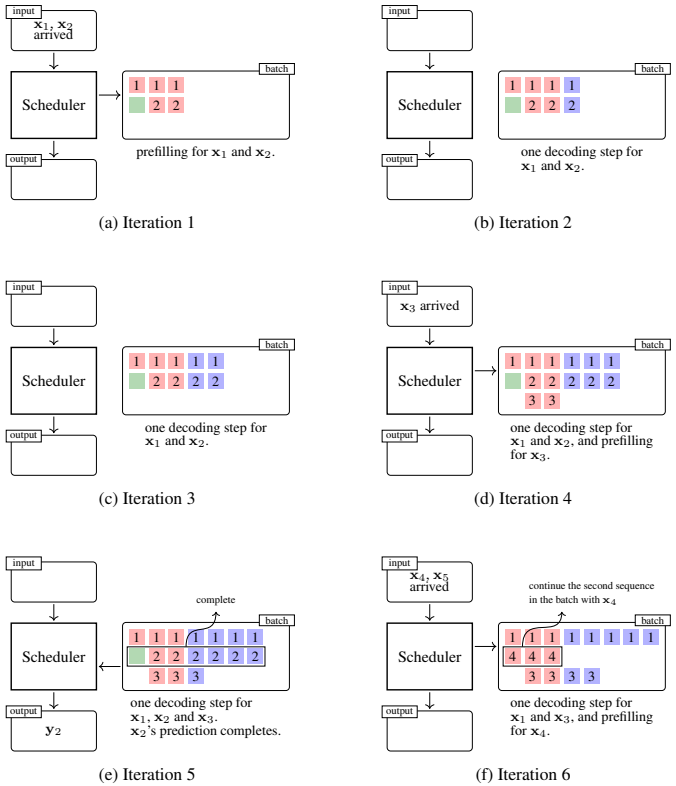
qua quá trình điền trước. Sau một thời gian, việc sinh cho $\mathbf{x}_2$ hoàn thành. Đúng lúc đó, thêm hai yêu cầu của người dùng, $\mathbf{x}_4$ và $\mathbf{x}_5$, đến. Bộ lập lịch loại chuỗi đã hoàn thành $\mathbf{x}_2$ khỏi lô và, xét đến tải hiện tại của bộ máy suy luận, thêm $\mathbf{x}_4$ vào lô. Tuy nhiên, $\mathbf{x}_5$ phải chờ đến khi một chuỗi khác trong lô kết thúc mới có thể được thêm vào.

Ý tưởng đằng sau xử lý theo lô liên tục là giữ cho bộ máy suy luận luôn được dùng hết công suất bằng cách xử lý càng nhiều chuỗi càng tốt, qua đó tối đa hóa mức sử dụng tài nguyên tính toán. Một khác biệt then chốt giữa xử lý theo lô liên tục và xử lý theo lô tiêu chuẩn (xem Hình 11.8) nằm ở chỗ: trong xử lý theo lô liên tục, điền trước và giải mã có thể diễn ra đồng thời trên các chuỗi khác nhau, còn trong xử lý theo lô tiêu chuẩn, hai pha này được thực hiện tuần tự cho toàn bộ lô. Như đã bàn ở Mục 11.1.2, điền trước được xem là một quá trình bị giới hạn bởi tính toán (*compute-bound*), còn giải mã được xem là một quá trình bị giới hạn bởi bộ nhớ (*memory-bound*). Trực giác đằng sau việc chồng lấp điền trước và giải mã là giảm thời gian nhàn rỗi cho cả tính toán lẫn truyền dữ liệu. Hãy xét hai lô nhỏ: một cho điền trước và một cho giải mã. Trong khi lô nhỏ điền trước giữ cho GPU bận rộn, lô nhỏ giải mã có thể đồng thời thực hiện việc truyền dữ liệu bộ nhớ.

Chú giải của người dịch: Vì sao chồng lấp hai pha lại có lợi?

Pha điền trước xử lý cả prompt một lúc, nên GPU chủ yếu bận *tính toán* (nhân các ma trận lớn). Pha giải mã mỗi bước chỉ sinh một token, phép tính ít nhưng phải đọc toàn bộ tham số mô hình và bộ nhớ đệm KV từ bộ nhớ, nên GPU chủ yếu bận *chờ dữ liệu*. Giống như trong một nhà bếp: một đầu bếp đang thái rau (việc "tốn sức tay") thì người phụ bếp có thể tranh thủ đi lấy nguyên liệu từ kho (việc "tốn công di chuyển"). Làm xen kẽ hai loại việc thì cả hai người đều ít phải đứng chờ.

Một khác biệt nữa giữa xử lý theo lô liên tục và xử lý theo lô tiêu chuẩn là xử lý theo lô liên tục *ưu tiên điền trước*, còn xử lý theo lô tiêu chuẩn *ưu tiên giải mã* [Agrawal và cộng sự, 2024]. Trong xử lý theo lô liên tục, hễ bộ máy suy luận có tài nguyên tính toán dư, bộ lập lịch sẽ thêm các yêu cầu mới vào lô. Nói cách khác, các yêu cầu mới thêm vào này sẽ được điền trước sớm nhất có thể. Cách làm này cải thiện thông lượng hệ thống, nhưng phải trả giá bằng độ trễ tăng lên, vì các yêu cầu mới thêm vào kéo dài thời gian xử lý của các yêu cầu trước đó. Ngược lại, trong xử lý theo



(a) Iteration 1

(b) Iteration 2

(c) Iteration 3

(d) Iteration 4

(e) Iteration 5

(f) Iteration 6

Hình 11.11: Minh họa việc điều chỉnh lô trong xử lý theo lô liên tục. Thay vì cố định một lô các chuỗi đầu vào và xử lý chúng đến khi xong (như trong xử lý theo lô mức yêu cầu), xử lý theo lô liên tục cập nhật lô một cách động trong quá trình suy luận. Hệ thống liên tục tiếp nhận và thêm các yêu cầu mới (ví dụ $\mathbf{x}_3$ và $\mathbf{x}_4$) vào lô hiện tại chừng nào vẫn còn năng lực tính toán. (Nhãn trong hình: *Iteration*: vòng lặp; *prefilling for …*: điền trước cho…; *one decoding step for …*: một bước giải mã cho…; *complete*: hoàn thành; $\mathbf{x}_2$*'s prediction completes*: dự đoán của $\mathbf{x}_2$ hoàn thành; *continue the second sequence in the batch with* $\mathbf{x}_4$: tiếp tục vị trí chuỗi thứ hai trong lô bằng $\mathbf{x}_4$.)

lô tiêu chuẩn, một khi lô đã được tạo, ta phải chờ chuỗi cuối cùng trong lô hoàn thành rồi mới xử lý các yêu cầu mới. Điều này bảo đảm độ trễ tương đối thấp, nhưng dẫn tới mức sử dụng thiết bị và thông lượng hệ thống thấp hơn.

Cần lưu ý rằng cái giá của xử lý theo lô liên tục là ta phải liên tục tổ chức lại các lô, việc này đòi hỏi sắp xếp lại dữ liệu trong bộ nhớ. Mỗi lần một yêu cầu mới được thêm vào, bộ lập lịch cần đánh giá lại và tối ưu cấu trúc lô hiện tại. Việc điều chỉnh động này có thể gây thêm chi phí bộ nhớ và tính toán, đặc biệt khi các lô bị điều chỉnh thường xuyên. Vì vậy, dù phương pháp này có thể cải thiện thông lượng, nó cũng có thể làm tăng phân mảnh bộ nhớ (*memory fragmentation*) và trong một số trường hợp đưa thêm độ trễ.

3. PagedAttention

PagedAttention (hay lưu đệm KV phân trang (*paged KV caching*)) là một kỹ thuật được dùng trong hệ thống vLLM [Kwon và cộng sự, 2023]. Lấy cảm hứng từ cơ chế phân trang (*paging*) của hệ điều hành, nó tối ưu mức dùng bộ nhớ trong suy luận LLM — đặc biệt là cho bộ nhớ đệm KV — bằng cách giải quyết vấn đề cấp phát bộ nhớ bị phân mảnh trong các tình huống xử lý theo lô động với các chuỗi có độ dài thay đổi. Ý tưởng đằng sau PagedAttention là chia bộ nhớ đệm KV thành các trang bộ nhớ nhỏ hơn. Bằng cách này, ta không cần lưu bộ nhớ đệm KV của cả chuỗi trong một vùng bộ nhớ liền mạch. Thay vào đó, bộ nhớ đệm KV được chia thành các khối có kích thước cố định (tương tự các trang bộ nhớ trong hệ điều hành), và các khối này có thể được cấp phát không liền mạch trong bộ nhớ vật lý. Một ưu điểm của PagedAttention là nó cho phép quản lý bộ nhớ linh hoạt, hỗ trợ chuỗi tăng trưởng động mà không cần cấp phát lại hay sao chép các vùng bộ nhớ liền mạch lớn vốn tốn kém. Lưu ý rằng PagedAttention không được thiết kế riêng cho xử lý theo lô. Nhưng nó thực sự giúp cải thiện hiệu quả bộ nhớ trong các tình huống suy luận theo lô, nơi việc quản lý bộ nhớ đòi hỏi cao hơn và phức tạp hơn.

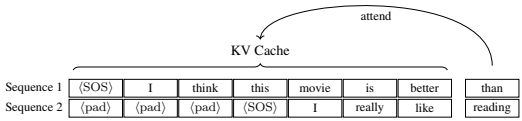
Xét một ví dụ đơn giản về cấp phát bộ nhớ ở Hình 11.12, trong đó tự chú ý được thực hiện cho một lô gồm hai chuỗi. Với mỗi chuỗi, ta cần cho token hiện tại chú ý tới các cặp khóa–giá trị trong bộ nhớ đệm KV của chuỗi này, theo yêu cầu của tự chú ý. Trong cài đặt tiêu chuẩn của tự chú ý, bộ nhớ đệm KV được lưu trong một khối bộ nhớ liền mạch, cho phép

ta truy cập hiệu quả vùng bộ nhớ liền mạch này. Tuy nhiên, trong một hệ thống lưu đệm KV phân trang, bộ nhớ đệm KV được chia thành các khối bộ nhớ nhỏ hơn, có kích thước cố định và không nhất thiết liền mạch. Các khối bộ nhớ đệm KV nhỏ hơn này có thể được cấp phát hiệu quả hơn vào các vùng bộ nhớ bị phân mảnh, nhờ đó cải thiện mức sử dụng bộ nhớ. Một lợi ích khác của việc phân bố các đoạn của bộ nhớ đệm KV trên các khối bộ nhớ khác nhau là nó cho phép song song hóa quá trình lưu đệm. Chẳng hạn, nếu chuỗi đầu vào dài và băng thông bộ nhớ đủ lớn, việc ghi và đọc song song các vectơ khóa và giá trị của các đoạn khác nhau của chuỗi trên nhiều khối bộ nhớ sẽ có lợi.

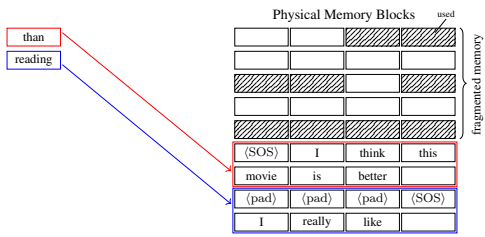
Nói chung, lưu dữ liệu liền mạch vào các vùng không liền mạch có thể gây ra vấn đề, chẳng hạn có thể phát sinh thêm chi phí truy cập bộ nhớ, làm giảm hiệu quả vào/ra (I/O). Tuy nhiên, khi xử lý dữ liệu quy mô lớn (ví dụ thực hiện các phép nhân ma trận lớn), ta thường không xử lý toàn bộ dữ liệu một lúc mà chia nó thành các khối nhỏ hơn để tính toán theo khối. Từ góc nhìn này, việc phân chia phép tính chú ý cũng là hợp lý. Nếu chiến lược phân trang được thiết kế tốt, chi phí truy cập bộ nhớ phát sinh thêm có thể rất nhỏ, trong khi mức cải thiện về sử dụng bộ nhớ có thể đáng kể.

Chú giải của người dịch: Phân trang: ví dụ bãi đỗ xe

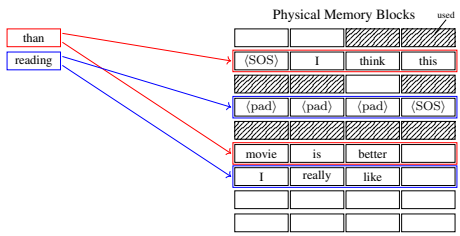
Hãy tưởng tượng bộ nhớ GPU là một bãi đỗ xe có các ô liên tiếp. Cách tiêu chuẩn đòi hỏi bộ nhớ đệm KV của một chuỗi phải nằm trong *một dãy ô liền nhau*: nếu một đoàn xe 8 chiếc cần đỗ cạnh nhau mà bãi chỉ còn những chỗ trống lẻ tẻ (2 ô chỗ này, 3 ô chỗ kia), thì dù tổng số ô trống đủ, đoàn xe vẫn không đỗ được — đó là *phân mảnh bộ nhớ*. PagedAttention cho phép "đoàn xe" tách ra thành từng nhóm nhỏ cố định (ví dụ 4 xe một nhóm) đỗ ở bất kỳ đâu còn trống, chỉ cần ghi lại một "sổ tay" (bảng trang) cho biết nhóm thứ mấy đang ở chỗ nào. Hệ điều hành máy tính cũng quản lý bộ nhớ RAM theo đúng cách này.



(a) Two sequences in a batch



(b) Memory allocation for KV caching in standard self-attention



(c) Memory allocation for KV caching in PagedAttention

Hình 11.12: Minh họa cấp phát bộ nhớ trong PagedAttention. Có hai chuỗi trong lô, như minh họa ở hình con (a). Vì bộ nhớ bị phân mảnh, trong tự chú ý tiêu chuẩn bộ nhớ đệm KV được lưu trong một khối bộ nhớ lớn chưa dùng (xem hình con (b)), còn phần bộ nhớ bị phân mảnh thì không được dùng. Ngược lại, trong PagedAttention (xem hình con (c)), bộ nhớ đệm KV được chia thành các khối nhỏ hơn và nhờ đó vừa với phần bộ nhớ bị phân mảnh. (Nhãn trong hình: *Sequence 1/2*: chuỗi 1/2, với nội dung "I think this movie is better than" — "Tôi nghĩ bộ phim này hay hơn" và "I really like reading" — "Tôi thực sự thích đọc"; *attend*: chú ý; *Physical Memory Blocks*: các khối bộ nhớ vật lý; *used*: đã dùng; *fragmented memory*: bộ nhớ bị phân mảnh.)

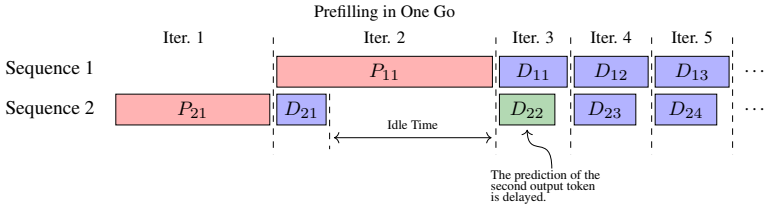
4. Điền trước theo khúc

Ta đã thấy rằng trong lập lịch mức vòng lặp, điền trước và giải mã cho các chuỗi khác nhau có thể diễn ra đồng thời. Điều này có thể được xem là một chiến lược ưu tiên điền trước, giúp tối đa hóa thông lượng. Tuy nhiên, một vòng lặp như vậy có thể kéo dài nếu chuỗi đầu vào rất dài và quá trình điền trước chiếm phần lớn lượng tính toán. Khi đó, việc giải mã cho các chuỗi khác phải chờ đến khi điền trước hoàn thành, dẫn tới độ trễ sinh token đầu ra tăng lên. Vì vậy, dù các chiến lược ưu tiên điền trước hiệu quả trong việc tối đa hóa mức sử dụng phần cứng, chúng có thể khiến độ trễ sinh token biến động đáng kể, đặc biệt khi hệ thống phải xử lý lẫn lộn các chuỗi đầu vào dài và ngắn.

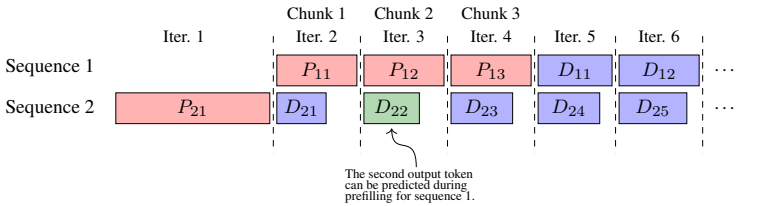
Một cách đơn giản để giảm độ trễ giải mã là làm cho lượng tính toán của các chuỗi khác nhau trong lô tương đương nhau. Một phương pháp như vậy là chia các chuỗi thành các khúc và thực hiện điền trước theo từng khúc. Cách tiếp cận này, thường được gọi là điền trước theo khúc (*chunked prefilling*), mỗi lần xử lý một phần nhỏ hơn của mỗi chuỗi, cho phép hệ thống cân bằng tốt hơn tải tính toán giữa các chuỗi [Agrawal và cộng sự, 2023]. Bằng cách chọn kích thước khúc phù hợp, ta có thể bảo đảm rằng khi điền trước và giải mã chồng lấp nhau trên hai chuỗi, việc xử lý chúng trong cùng một vòng lặp có xu hướng tốn lượng thời gian tương tự nhau. Nhờ đó, thời gian nhàn rỗi của giải mã giảm và thông lượng tổng thể được cải thiện.

Hình 11.13 minh họa điền trước theo khúc qua vài vòng lặp. Trong ví dụ này, lô chứa hai chuỗi. Toàn bộ quá trình điền trước của chuỗi thứ nhất được chia thành ba bước điền trước, tạo ra các khúc ký hiệu là P_{11}, P_{12} và P_{13}. Mỗi khúc tương ứng với một vòng lặp và do đó có thể chồng lấp với một bước giải mã. Bằng cách này, trong khi điền trước chuỗi thứ nhất, ta có thể thực hiện ba bước giải mã thay vì chỉ một bước giải mã như trong lập lịch mức vòng lặp tiêu chuẩn. Nhờ đó, thời gian nhàn rỗi của quá trình giải mã giảm và các token đầu ra có thể được sinh sớm hơn.

Điền trước theo khúc cải thiện hiệu quả giải mã bằng cách chồng lấp điền trước và giải mã, nhưng phải trả giá bằng chi phí bộ nhớ bổ sung và độ phức tạp lập lịch. Trong điền trước tiêu chuẩn, ta xử lý toàn bộ chuỗi đầu vào một lần, xây dựng bộ nhớ đệm KV trong một lượt. Ngược lại, trong điền trước theo khúc, mỗi khúc cần một lượt truyền xuôi riêng để tính đầu ra chú ý của nó và cập nhật bộ nhớ đệm KV. Do đó, ta cần duy



(a) Simple Iteration-level Scheduling



(b) Chunked Prefilling

Hình 11.13: So sánh lập lịch mức vòng lặp đơn giản và điền trước theo khúc. P_{xy} ký hiệu bước điền trước thứ y của chuỗi x, và D_{xy} ký hiệu bước giải mã thứ y của chuỗi x. Trong lập lịch mức vòng lặp đơn giản (hay lập lịch ưu tiên điền trước), vì điền trước được coi là một vòng lặp duy nhất, D_{22} phải chờ quá trình điền trước của chuỗi 1 hoàn thành. Trong điền trước theo khúc, quá trình điền trước có thể được chia thành nhiều bước. Do đó, D_{22} có thể thực thi trong khi điền trước chuỗi 1 (tức là trong P_{12}). (Nhãn trong hình: *Prefilling in One Go*: điền trước một lượt; *Iter.*: vòng lặp; *Idle Time*: thời gian nhàn rỗi; *Chunk*: khúc; *The prediction of the second output token is delayed*: việc dự đoán token đầu ra thứ hai bị trễ; *The second output token can be predicted during prefilling for sequence 1*: token đầu ra thứ hai có thể được dự đoán trong khi điền trước chuỗi 1.)

trì bộ nhớ đệm KV của các khúc trước trong khi xử lý các khúc sau. Điều này cũng làm giảm khả năng song song so với việc hoàn tất điền trước cho cả chuỗi trong một lượt. Trong thực tế, thường có thể cân bằng thông lượng và độ trễ bằng cách chọn kích thước khúc phù hợp.

Chú giải của người dịch: Ví dụ số về điền trước theo khúc

Giả sử điền trước một prompt 3000 token mất 300 ms, còn một bước giải mã của chuỗi khác chỉ mất khoảng 10 ms. Nếu điền trước một lượt, chuỗi kia phải chờ khoảng 300 ms mới sinh được token tiếp theo — người dùng thấy chữ "khựng lại". Nếu chia prompt thành 3 khúc 1000 token (mỗi khúc khoảng 100 ms), sau mỗi khúc chuỗi kia lại được sinh thêm một token, nên khoảng chờ giữa hai token giảm còn khoảng 100 ms. Cái giá là phải giữ bộ nhớ đệm KV của các khúc trước và lập lịch phức tạp hơn. (Các con số chỉ mang tính minh họa.)

Đáng chú ý là các phương pháp bàn trong tiểu mục này có thể được xếp chung vào nhóm các phương pháp lập lịch dựa trên mức ưu tiên (*priority-based scheduling*). Trong các phương pháp này, ta có thể ưu tiên một số yêu cầu nhất định, hoặc một số bước điền trước hay giải mã nhất định, để tài nguyên hệ thống được phân bổ phù hợp hơn với các mục tiêu hiệu năng cụ thể. Chẳng hạn, như đã trình bày ở trên, ta có thể ưu tiên giải mã hơn điền trước để giảm thiểu độ trễ sinh token, hoặc ưu tiên điền trước hơn giải mã để tối đa hóa thông lượng tổng thể trong các tình huống xử lý theo lô. Những người làm thực tế có thể thiết kế các chính sách ưu tiên tùy biến cho các nhu cầu cụ thể và các ràng buộc vận hành trong ứng dụng thực tế, chẳng hạn thời hạn của yêu cầu và mức độ quan trọng do người dùng xác định.

11.2.3 Song song hóa

Song song hóa (*parallelization*) là một cách tiếp cận được dùng rộng rãi để mở rộng quy mô suy luận LLM, đặc biệt cho các triển khai quy mô lớn. Ở Chương 7, chúng ta đã bàn về một số chiến lược phổ biến để song song hóa quá trình tiền huấn luyện (*pre-training*) LLM, chẳng hạn song song mô hình (*model parallelism*), song song tensor (*tensor parallelism*) và song song đường ống (*pipeline parallelism*). Chúng ta cũng đã bàn về các kiến trúc hiệu quả, dễ triển khai trên các hệ thống tính toán phân tán.

Ví dụ, trong các mô hình MoE, ta gán các chuyên gia khác nhau cho các thiết bị khác nhau[3]. Chỉ những chuyên gia được kích hoạt cho một đầu vào cho trước mới được thực thi, điều này cải thiện đáng kể hiệu quả tính toán trong khi vẫn giữ được chất lượng mô hình. Nhiều phương pháp trong số này có thể được áp dụng trực tiếp cho suy luận LLM với rất ít sửa đổi.

Tuy nhiên, áp dụng các kỹ thuật song song hóa này cho suy luận đặt ra những thách thức mới so với tiền huấn luyện. Các vấn đề này trở nên đặc biệt rõ rệt trong các tình huống suy luận thời gian thực hoặc độ trễ thấp, nơi sự mất cân bằng tải giữa các thiết bị và chi phí truyền thông có thể ảnh hưởng đáng kể đến hiệu năng. Chẳng hạn, khác với tiền huấn luyện, nơi các lô có thể được chuẩn bị trước, suy luận phải xử lý các chuỗi có độ dài thay đổi theo thời gian thực. Điều này khiến việc duy trì mức sử dụng thiết bị tối ưu khó khăn hơn và làm phức tạp việc lập lịch trên các tài nguyên tính toán không đồng nhất. Một vấn đề liên quan là cân bằng tải (*load balancing*). Khi một số lượng lớn yêu cầu đến trong một khoảng thời gian ngắn, hệ thống phải phân phối khối lượng công việc một cách hiệu quả lên các thiết bị sẵn có. Ví dụ, các yêu cầu trong thực tế thường có nhu cầu tính toán biến động rất mạnh do khác biệt về loại nhiệm vụ và độ dài prompt. Sự biến động này khiến các cách cân bằng tải tĩnh đơn giản trở nên kém hiệu quả, vì vậy ta cần dùng những chiến lược chi tiết hơn, có thể thích ứng với điều kiện lúc chạy. Bài toán còn phức tạp hơn nữa khi ta triển khai hệ thống trên phần cứng không đồng nhất và có các ràng buộc độ trễ nghiêm ngặt.

Trong việc phát triển LLM, song song hóa gắn chặt với việc phục vụ LLM (*LLM serving*). Xây dựng một hệ thống phục vụ LLM chất lượng cao không phải là việc đơn giản — nó thường đòi hỏi kết hợp nhiều kỹ thuật, chẳng hạn thiết kế kiến trúc, phân phối khối lượng công việc, và các tối ưu phần cứng/phần mềm dành riêng cho LLM. Do đó, phục vụ LLM là một chủ đề rộng, thường đòi hỏi nhiều chuyên môn kỹ thuật. Ở đây, chúng ta sẽ không đi vào chi tiết của việc phục vụ LLM. Về các khái niệm và kỹ thuật liên quan, bạn đọc có thể tham khảo các hệ thống mã nguồn mở liên quan (như vLLM[4], TensorRT-LLM[5] và TGI[6]) và các bài

[3] Trong LLM, các chuyên gia thường là các mô-đun FFN. Vì vậy mỗi chuyên gia là một phần của thành phần FFN trong kiến trúc Transformer.

[4] `https://github.com/vllm-project/vllm`

[5] `https://github.com/NVIDIA/TensorRT-LLM`

[6] `https://github.com/huggingface/text-generation-inference`

báo [Pope và cộng sự, 2023; Li và cộng sự, 2024a].

11.2.4 Nhận xét

Trong chương này và các chương trước, chúng ta đã xem xét nhiều phương pháp nâng cao hiệu quả của LLM. Dù các cách tiếp cận này giải quyết những vấn đề khác nhau, phần lớn chúng về bản chất là khám phá các sự đánh đổi giữa nhiều yếu tố hiệu năng. Một sự đánh đổi quan trọng là giữa tốc độ suy luận và độ chính xác. Chẳng hạn, các kỹ thuật như lượng tử hóa, tỉa và chưng cất tri thức (*knowledge distillation*) có thể giảm đáng kể chi phí tính toán và độ trễ nhưng có thể làm giảm nhẹ hiệu năng mô hình. Ngược lại, dùng độ chính xác số cao hơn hoặc mô hình lớn hơn giúp tăng độ chính xác nhưng phải trả giá bằng suy luận chậm hơn và nhu cầu tài nguyên cao hơn.

Một cân nhắc quan trọng khác trong suy luận LLM là sự đánh đổi bộ nhớ–tính toán (*memory-compute trade-off*). Như trong thiết kế hệ thống máy tính, ta cần cân nhắc sự cân bằng giữa lượng bộ nhớ dùng và lượng tính toán cần thiết để sinh đầu ra. Đặc biệt, việc lưu các kết quả trung gian như bộ nhớ đệm KV trong khi suy luận có thể giảm đáng kể tính toán dư thừa, nhưng phải trả giá bằng mức dùng bộ nhớ tăng lên. Trong lưu đệm KV, việc lưu các trạng thái chú ý trước đó giúp tránh tính lại tự chú ý trên các token trước, nhờ đó giảm thời gian tính toán cho mỗi token. Tuy nhiên, khi số token tăng, dung lượng bộ nhớ của bộ nhớ đệm KV cũng tăng theo, đặc biệt khi xử lý các chuỗi rất dài hoặc nhiều chuỗi song song. Để đối phó, nhiều kỹ thuật đã được phát triển nhằm giảm tiêu thụ bộ nhớ bằng cách tính lại một phần các trạng thái trung gian. Chẳng hạn, chú ý theo khúc hoặc chú ý theo cửa sổ giới hạn phạm vi chú ý vào một tập con các token gần đây, và do đó giảm kích thước bộ nhớ đệm KV. Tuy nhiên, việc này thường đòi hỏi thêm tính toán nếu thông tin trong quá khứ phải được xử lý lại.

Lưu ý rằng việc cân nhắc đánh đổi bộ nhớ–tính toán là một nguyên lý rất tổng quát. Nó có thể được mở rộng ra ngoài cơ chế chú ý và Transformer, sang các thành phần khác trong thiết kế hệ thống. Một ví dụ là việc chọn độ chính xác dữ liệu. Dùng các định dạng có độ chính xác thấp hơn như FP16 hay INT8 có thể giảm cả mức dùng bộ nhớ lẫn yêu cầu băng thông bộ nhớ, giảm tải hiệu quả cho phân hệ bộ nhớ. Tuy nhiên,

độ chính xác thấp hơn có thể dẫn tới mất ổn định số hoặc giảm nhẹ độ chính xác, đòi hỏi phải hiệu chỉnh hoặc huấn luyện lại cẩn thận. Vì vậy, sự đánh đổi này cũng có thể được xem như một tam giác bộ nhớ–tính toán–độ chính xác, trong đó cải thiện ở một chiều có thể phải trả giá bằng một chiều khác.

Chú giải của người dịch: FP16 và INT8 là gì?

Máy tính lưu mỗi số bằng một số lượng bit nhất định. FP32 dùng 32 bit (4 byte) cho một số thực, FP16 dùng 16 bit (2 byte), INT8 dùng 8 bit (1 byte) cho một số nguyên trong khoảng -128 đến 127. Một mô hình 7 tỷ tham số cần khoảng $7 \times 10^9 \times 2 = 14$ GB nếu lưu ở FP16, nhưng chỉ khoảng 7 GB nếu lưu ở INT8. Đổi lại, số được lưu "thô" hơn: giống như làm tròn điểm thi từ hai chữ số thập phân xuống còn số nguyên — gọn hơn nhưng mất bớt độ chi tiết.

Ngoài tốc độ, độ chính xác và bộ nhớ, một số chiều khác cũng ảnh hưởng tới hiệu quả suy luận LLM. Một số chiều đã được bàn trong chương này, số khác thì chưa. Ở đây chúng tôi phác thảo chúng như sau.

- **Thông lượng và độ trễ**: Trong các tình huống phục vụ LLM quy mô lớn cho nhiều người dùng, ta thường muốn tối đa hóa thông lượng hệ thống. Chẳng hạn, như đã bàn trong mục này, ta có thể gom nhiều yêu cầu thành lô để tăng số token được xử lý cùng lúc. Tuy nhiên, xử lý theo lô làm tăng thời gian chờ và có thể dẫn tới độ trễ cao hơn cho mỗi yêu cầu, đặc biệt với các yêu cầu ngắn hoặc mang tính tương tác. Ngược lại, tối ưu cho độ trễ thấp thường đòi hỏi phục vụ từng yêu cầu riêng lẻ hoặc theo các lô nhỏ hơn, khiến tài nguyên phần cứng không được dùng hết và làm giảm thông lượng. Đạt được sự cân bằng tốt tùy thuộc vào các yêu cầu về chất lượng dịch vụ (*quality of service*) và kiểu tương tác của người dùng.

- **Tổng quát hóa và chuyên biệt hóa**: Các LLM đa năng được huấn luyện để thực hiện nhiều loại nhiệm vụ với một bộ tham số duy nhất. Dù linh hoạt, chúng có thể kém hiệu quả hoặc kém chính xác hơn cho các nhiệm vụ cụ thể. Các mô hình chuyên biệt có thể cho hiệu năng tốt hơn và chi phí suy luận thấp hơn cho các ứng dụng mục tiêu. Tuy nhiên, duy trì nhiều mô hình chuyên biệt làm tăng độ phức tạp của hệ thống và yêu cầu lưu trữ. Sự đánh đổi giữa việc duy

trì một mô hình tổng quát duy nhất và nhiều mô hình chuyên biệt là một lựa chọn thiết kế quan trọng ở cấp hệ thống.

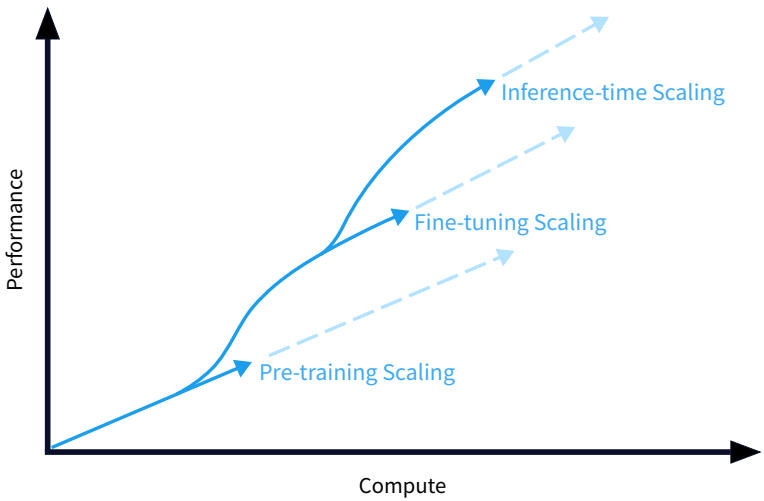
- **Hiệu quả năng lượng và hiệu năng**: Suy luận hiệu năng cao thường đòi hỏi chạy các mô hình lớn với thông lượng cao trên các bộ tăng tốc mạnh, vốn tiêu thụ rất nhiều năng lượng. Điều này có thể là vấn đề với các triển khai ở thiết bị biên hoặc các môi trường nhạy cảm về năng lượng. Các kỹ thuật như nén mô hình có thể cải thiện hiệu quả năng lượng, nhưng thường kèm theo một mức suy giảm chất lượng đầu ra hoặc tăng độ trễ. Do đó, các ràng buộc năng lượng đưa thêm một chiều quan trọng nữa vào việc tối ưu suy luận LLM.

11.3 Co giãn lúc suy luận

Định luật co giãn (*scaling law*) có thể được xem là một trong những nguyên lý nền tảng định hướng sự phát triển của LLM. Ở các chương trước, chúng ta đã nhiều lần bàn rằng việc tăng quy mô dữ liệu huấn luyện, kích thước mô hình và lượng tính toán có thể cải thiện hiệu quả hiệu năng của LLM. Thực ra, định luật co giãn cũng áp dụng cho các giai đoạn hạ nguồn như tinh chỉnh (*fine-tuning*) và suy luận (xem Hình 11.14). Ở đây ta xét **co giãn lúc suy luận** (*inference-time scaling*), một kỹ thuật đã được các LLM gần đây dùng rộng rãi để giải các bài toán khó, chẳng hạn các bài toán toán học phức tạp [Snell và cộng sự, 2025]. Khác với co giãn trong tiền huấn luyện và tinh chỉnh, vốn tập trung cải thiện LLM qua việc cập nhật tham số, co giãn lúc suy luận cải thiện hiệu năng mô hình trong lúc suy luận mà không cần huấn luyện thêm. Các phương pháp này co giãn suy luận theo nhiều cách khác nhau, chẳng hạn kết hợp nhiều đầu ra của mô hình, tăng độ dài ngữ cảnh, áp dụng các thuật toán giải mã "mạnh tay" hơn, và dùng công cụ bên ngoài để mở rộng năng lực của mô hình.

Dù co giãn lúc suy luận là một chủ đề rộng, trong mục này ta xét các phương pháp đưa thêm lượng tính toán vào suy luận (gọi là co giãn tính toán lúc suy luận (*inference-time compute scaling*)). Dưới đây là danh sách các phương pháp co giãn tính toán lúc suy luận (hay lúc kiểm tra — *test-time*), sắp xếp theo nhóm:

- **Co giãn ngữ cảnh** (*Context Scaling*). Bao gồm việc mở rộng đầu



Hình 11.14: Co giãn cho các giai đoạn tiền huấn luyện, tinh chỉnh và suy luận [Briski, 2025]. (Nhãn trong hình: *Performance*: hiệu năng; *Compute*: lượng tính toán; *Pre-training Scaling*: co giãn tiền huấn luyện; *Fine-tuning Scaling*: co giãn tinh chỉnh; *Inference-time Scaling*: co giãn lúc suy luận.)

vào hoặc ngữ cảnh để cải thiện việc sinh (hoặc có thể mở rộng cả đầu ra).

- **Co giãn tìm kiếm** (*Search Scaling*). Bao gồm việc tăng công sức tính toán trong khi giải mã.
- **Kết hợp đầu ra** (*Output Ensembling*). Bao gồm việc kết hợp nhiều đầu ra của mô hình.
- **Sinh và kiểm chứng các đường suy nghĩ** (*Generating and Verifying Thinking Paths*). Bao gồm việc hướng dẫn LLM sinh và kiểm chứng các đường suy nghĩ để giải các bài toán lập luận (*reasoning*) phức tạp.

Chúng ta sẽ mô tả các phương pháp này trong các tiểu mục sau.

Chú giải của người dịch: Co giãn lúc suy luận: "nghĩ lâu hơn" thay vì "học nhiều hơn"

Ba đường cong ở Hình 11.14 cho thấy ba cách tăng hiệu năng bằng cách bỏ thêm tính toán: lúc tiền huấn luyện (mô hình to hơn, nhiều dữ liệu hơn), lúc tinh chỉnh, và lúc suy luận. Cách cuối cùng giống như một học sinh đi thi: kiến thức (tham số) đã cố định, nhưng nếu được phép nháp nhiều hơn, thử vài cách giải rồi so sánh, và kiểm tra lại bài trước khi nộp, thì vẫn có thể đạt điểm cao hơn. Cái giá là mất nhiều thời gian (tính toán) hơn cho mỗi câu hỏi.

11.3.1 Co giãn ngữ cảnh

Co giãn ngữ cảnh (*context scaling*) cải thiện hiệu năng LLM bằng cách mở rộng đầu vào cho mô hình. Một cách tiếp cận trực tiếp là đưa thêm ngữ cảnh liên quan vào lúc suy luận để mô hình có thể dựa các dự đoán của nó trên nhiều thông tin có trước hơn. Một ví dụ là prompt few-shot (*few-shot prompting*). Nó bổ sung vào ngữ cảnh nhiều ví dụ đầu vào–đầu ra, nhờ đó mô hình có thể học ngầm hành vi của nhiệm vụ từ các ví dụ này mà không cần cập nhật tham số. Bên cạnh prompt few-shot, ta có thể dùng prompt chuỗi suy nghĩ (*chain-of-thought prompting*) để khuyến khích mô hình tạo ra các bước lập luận trung gian trước khi đưa ra câu trả lời cuối cùng. Lưu ý rằng prompt chuỗi suy nghĩ là một trong những phương pháp quan trọng nhất để cải thiện hiệu năng lập luận. Bằng cách cung cấp tường minh các bước trung gian trong quá trình giải bài toán, ta có thể nhắc mô

hình chia nhỏ các nhiệm vụ phức tạp thành các nhiệm vụ con đơn giản hơn, điều được cho là rất có lợi để sinh ra các đầu ra chính xác và dễ diễn giải.

Ngoài việc mở rộng prompt bằng các ví dụ hay các bước lập luận, một cách tiếp cận khác của co giãn ngữ cảnh là đưa tri thức bên ngoài vào một cách động. Điều này thường được thực hiện bằng sinh văn bản tăng cường truy xuất (*retrieval-augmented generation, RAG*). Các hệ thống RAG trước hết truy xuất các đoạn tài liệu liên quan từ một bộ sưu tập tài liệu lớn hoặc một cơ sở dữ liệu dựa trên đầu vào hiện tại. Những mẩu thông tin truy xuất được này sau đó được thêm vào ngữ cảnh cung cấp cho LLM. Về bản chất, việc này mở rộng ngữ cảnh để bao gồm tri thức bên ngoài mang tính thời sự hoặc chuyên biệt. Bằng cách đó, mô hình neo các phản hồi của nó vào tri thức cụ thể tìm thấy ở nguồn bên ngoài. Nhờ vậy, LLM có thể sinh ra các phản hồi không chỉ liên quan tới đầu vào mà còn chính xác về mặt dữ kiện và cập nhật.

Tuy nhiên, khi ngữ cảnh tăng lên, các phương pháp này thường gặp phải giới hạn của một cửa sổ ngữ cảnh (*context window*) hữu hạn. Dù các kiến trúc mô hình và kỹ thuật (như các mô hình chú ý hiệu quả) liên tục phát triển để hỗ trợ ngữ cảnh dài hơn, việc xử lý các đầu vào cực dài vẫn đặt ra thách thức. Chi phí tính toán tăng lên là một yếu tố. Quan trọng hơn, khi cửa sổ ngữ cảnh trở nên rất lớn, mô hình có thể gặp khó khăn trong việc chú ý hiệu quả tới những thông tin liên quan nhất (ví dụ hiện tượng "lạc ở giữa" — *lost in the middle*). Do đó, co giãn ngữ cảnh hiệu quả không chỉ là thêm nhiều thông tin hơn, mà còn là lựa chọn, cấu trúc và trình bày một cách có chiến lược những thông tin thích đáng nhất trong phạm vi năng lực xử lý của mô hình.

Ở đây chúng tôi bỏ qua phần bàn chi tiết về các phương pháp này, vì chúng đã được đề cập ở các chương trước. Xem Chương 8 và 9 để biết thêm chi tiết, bao gồm các phương pháp prompt, RAG và mô hình hóa chuỗi dài.

11.3.2 Co giãn tìm kiếm

Trong LLM, giải mã là một quá trình tìm kiếm nhằm tìm ra một cách hiệu quả chuỗi đầu ra tốt nhất cho chuỗi đầu vào đã cho. Co giãn tìm kiếm (*search scaling*) (hay co giãn giải mã) thường gồm hai khía cạnh: co giãn

độ dài đầu ra và co giãn không gian tìm kiếm.

Co giãn độ dài đầu ra là việc tăng số token được sinh ra trong khi suy luận. Điều này đặc biệt quan trọng với các nhiệm vụ cần sinh văn bản dài, chẳng hạn viết truyện. Gần đây hơn, việc sinh đầu ra kèm các đường suy nghĩ dài đã cho hiệu năng mạnh trong giải toán và sinh mã. Chẳng hạn, người ta thấy rằng khuyến khích mô hình sinh các đường suy nghĩ dài trước khi đưa ra câu trả lời cuối cùng rất có lợi cho việc thực hiện lập luận phức tạp. Ý tưởng này đã được dùng rộng rãi trong việc phát triển các LLM lập luận gần đây, chẳng hạn o1 của OpenAI [2024] và R1 của DeepSeek [2025]. Chúng ta sẽ bàn chi tiết hơn về co giãn độ dài đầu ra ở Mục 11.3.4.

Mặt khác, co giãn không gian tìm kiếm là việc mở rộng tập các chuỗi đầu ra ứng viên được xem xét trong quá trình tìm kiếm, để có thể tìm được các đầu ra chất lượng cao hơn. Như đã bàn ở Mục 11.1.3, một ví dụ đơn giản là trong tìm kiếm chùm (*beam search*), việc tăng độ rộng chùm (*beam width*) cho phép khám phá nhiều chuỗi ứng viên hơn ở mỗi bước giải mã. Điều này làm tăng khả năng phát hiện các đầu ra tốt hơn, đặc biệt trong các nhiệm vụ mà lời giải tối ưu không thể thấy ngay từ các quyết định cục bộ.

Ngoài việc điều chỉnh thuật toán giải mã, cũng có thể khai thác các cấu trúc gọn để mã hóa một số lượng lớn đầu ra. Chẳng hạn, ta có thể xây dựng và duyệt một cây hoặc một đồ thị các bước lập luận [Yao và cộng sự, 2024]. Trong mô thức này, mỗi nút biểu diễn một lời giải bộ phận hoặc một bước trung gian, còn các cạnh biểu diễn sự chuyển tiếp giữa các trạng thái lập luận. Kiểu tìm kiếm có cấu trúc này cho phép mô hình xem xét đồng thời nhiều đường đi. Một hướng liên quan khác là giải mã lấy cảm hứng từ tìm kiếm cây Monte Carlo (*Monte Carlo tree search*), trong đó mô hình khám phá ngẫu nhiên và chấm điểm các đường đi khác nhau dựa trên các heuristic (*heuristic*) đã học hoặc các mô hình phần thưởng (*reward model*) bên ngoài.

Co giãn tìm kiếm là một ý tưởng rất tổng quát, và nó thường ngầm có mặt trong việc thiết kế các thủ tục tìm kiếm khai thác cấu trúc tìm kiếm, các heuristic và độ bất định của mô hình. Nhiều phương pháp ở trên đã được bàn trước đây, dù ban đầu chúng không được phát triển với mục tiêu chính là co giãn. Tuy nhiên, co giãn tìm kiếm vốn dĩ đi kèm chi phí tính toán. Chẳng hạn, tăng độ rộng chùm trực tiếp dẫn tới mức dùng bộ nhớ

cao hơn và thời gian suy luận dài hơn. Trong thực tế, thường có một điểm *lợi ích giảm dần*, mà vượt qua đó việc mở rộng thêm không gian tìm kiếm chỉ cải thiện chút ít chất lượng đầu ra với chi phí tính toán đáng kể. Vì vậy, một chiến lược hiệu quả thường bao gồm việc tìm sự cân bằng tối ưu giữa co giãn và tính khả thi về tính toán.

Chú giải của người dịch: Heuristic và lợi ích giảm dần

Heuristic là một "mẹo kinh nghiệm" giúp đoán nhanh hướng nào có triển vọng, không bảo đảm đúng tuyệt đối — giống như khi chơi cờ, ta ưu tiên xét những nước đi chiếm trung tâm trước. *Lợi ích giảm dần*: ví dụ, tăng độ rộng chùm từ 1 lên 4 có thể làm chất lượng tăng rõ rệt, từ 4 lên 16 tăng thêm một chút, còn từ 16 lên 64 hầu như không thay đổi, trong khi chi phí tính toán vẫn tăng gấp 4 lần mỗi lần.

11.3.3 Kết hợp đầu ra

Nếu có sẵn nhiều đầu ra của mô hình, việc kết hợp chúng thường có lợi để giảm ảnh hưởng của các lỗi riêng lẻ và tạo ra một đầu ra cuối cùng đáng tin cậy hơn. Mỗi mô hình có thể nắm bắt những khía cạnh khác nhau của phân phối dữ liệu nền, hoặc có những điểm mạnh và điểm yếu riêng. Nhờ kết hợp (*ensembling*), ta có thể lấy trung bình để triệt tiêu nhiễu hoặc các lỗi ngẫu nhiên có trong từng dự đoán riêng lẻ, dẫn tới một kết quả ổn định và đáng tin cậy hơn. Trong việc kết hợp LLM, một trong những cách đơn giản nhất là lấy trung bình các phân phối xác suất (*probability distribution*) của token tiếp theo từ mỗi mô hình, rồi chọn token tiếp theo theo phân phối trung bình này. Hoặc, nếu ta coi bài toán là một nhiệm vụ ra quyết định rời rạc, có thể dùng bỏ phiếu đa số (*majority voting*). Các phương pháp tinh vi hơn có thể bao gồm xếp hạng lại các đầu ra ứng viên do các mô hình khác nhau sinh ra dựa trên một hàm chấm điểm riêng, hoặc thậm chí dùng một bộ học meta (*meta-learner*) để kết hợp các dự đoán một cách thông minh.

Kết hợp đầu ra đòi hỏi thêm tính toán lúc suy luận, chẳng hạn chạy nhiều mô hình hoặc lấy mẫu nhiều đầu ra. Điều này không chỉ làm tăng độ trễ suy luận mà còn dẫn tới độ phức tạp hệ thống bổ sung. Nhưng chất lượng đầu ra không tiếp tục cải thiện mãi khi thêm nhiều mô hình hơn. Trong một số trường hợp, lợi ích của kết hợp đầu ra có thể giảm dần khi

số mô hình thành phần trong tổ hợp vượt quá một ngưỡng nhất định. Thay vào đó, lợi ích của việc kết hợp thường lớn hơn khi các mô hình riêng lẻ đa dạng (tức là chúng mắc những lỗi khác nhau), ngay cả khi số mô hình thành phần tương đối ít. Do đó, nếu có sẵn nhiều mô hình, sự đa dạng giữa các thành viên của tổ hợp cũng là điều đáng mong muốn.

Chú giải của người dịch: Vì sao kết hợp giúp ích? Một ví dụ số

Giả sử ba mô hình cho phân phối xác suất của token tiếp theo trên hai lựa chọn "A" và "B" lần lượt là $(0.6,\ 0.4)$, $(0.3,\ 0.7)$ và $(0.7,\ 0.3)$. Trung bình là $\big((0.6+0.3+0.7)/3,\ (0.4+0.7+0.3)/3\big) \approx (0.53,\ 0.47)$, nên chọn "A". Với bỏ phiếu đa số, hai mô hình chọn "A", một chọn "B", kết quả cũng là "A". Nếu mỗi mô hình đúng với xác suất 0.7 và mắc lỗi *độc lập* với nhau, thì xác suất ít nhất 2 trên 3 mô hình đúng là $0.7^3 + 3 \cdot 0.7^2 \cdot 0.3 = 0.343 + 0.441 = 0.784 > 0.7$. Đây chính là lý do sự *đa dạng* (lỗi khác nhau) quan trọng: nếu ba mô hình luôn sai cùng một chỗ, bỏ phiếu chẳng giúp được gì.

Kết hợp đầu ra cho thấy co giãn lúc suy luận không chỉ giới hạn ở việc tăng kích thước mô hình hay độ dài đầu ra. Bằng cách phân bổ thêm tính toán để sinh và tổng hợp nhiều đầu ra ứng viên, việc kết hợp có thể cải thiện độ vững và chất lượng đầu ra. Cũng như các dạng co giãn lúc suy luận khác, những lợi ích này phải được cân đối với chi phí tính toán bổ sung.

11.3.4 Sinh và kiểm chứng đường suy nghĩ

Cho đến giờ, chúng ta đã xem co giãn lúc suy luận là một lớp phương pháp tổng quát để co giãn nhiều khía cạnh khác nhau của suy luận, chẳng hạn độ dài chuỗi, kích thước mô hình và/hoặc chiến lược tìm kiếm. Thực ra, một ứng dụng thành công là dùng co giãn lúc suy luận để nâng cao năng lực lập luận của LLM. Như ta đã thấy, prompt chuỗi suy nghĩ cải thiện hiệu năng lập luận của LLM bằng cách khuyến khích sinh các bước lập luận trung gian trước khi đi đến câu trả lời cuối cùng. Tuy nhiên, các bài toán lập luận thường phức tạp tới mức ta không thể có được lời giải chất lượng cao chỉ bằng cách đưa ra các prompt chuỗi suy nghĩ đơn giản. Chẳng hạn, khi giải một bài toán, ta thường cần lập luận qua một dãy các bước. Ở mỗi bước, ta cần tìm ra một kết quả trung gian nào đó, kiểm chứng nó, rồi quyết định làm gì tiếp theo. Đường lập luận không phải là

một khuôn mẫu cố định mà là một quá trình suy nghĩ được sinh ra một cách động, thường bao gồm thử và sai, quay lui và tự sửa lỗi. Điều này đòi hỏi các chiến lược prompt hoặc các thuật toán tìm kiếm tinh vi hơn để dẫn đường trong kiểu lập luận phức tạp như vậy. Trong tiểu mục này, chúng ta tập trung vào các phương pháp co giãn lúc suy luận vượt ra ngoài chuỗi suy nghĩ đơn giản để giải quyết hiệu quả hơn các bài toán lập luận phức tạp.

Ở mức tổng quát, các phương pháp co giãn khả năng lập luận của LLM có thể được chia thành hai lớp:

- **Các phương pháp không cần huấn luyện** (*Training-free Methods*). Các phương pháp này nhằm cải thiện năng lực lập luận mà không cập nhật các tham số của mô hình đã tiền huấn luyện. Thay vào đó, chúng tập trung vào các kỹ thuật áp dụng trong lúc suy luận, chẳng hạn các chiến lược prompt tinh vi (ví dụ chuỗi suy nghĩ) và việc điều khiển quá trình lập luận bằng thuật toán (ví dụ tìm kiếm).
- **Các phương pháp dựa trên huấn luyện** (*Training-based Methods*). Các phương pháp này bao gồm việc huấn luyện thêm hoặc tinh chỉnh các tham số mô hình để cải thiện tường minh khả năng lập luận, chẳng hạn tinh chỉnh có giám sát (*supervised fine-tuning*) trên các tập dữ liệu chứa ví dụ lập luận (ví dụ các bài toán kèm lời giải từng bước).

Sau đây, trước hết chúng ta bàn về các phương pháp không cần huấn luyện, rồi đến các phương pháp dựa trên huấn luyện.

1. Tìm kiếm mức lời giải với bộ kiểm chứng

Cho một chuỗi đầu vào (ví dụ một bài toán), có nhiều chuỗi đầu ra khả dĩ (ví dụ các lời giải của bài toán). Nếu có một mô hình để đánh giá hay kiểm chứng từng lời giải, ta có thể chọn ra lời giải tốt nhất. Đây là nguyên lý nền tảng đằng sau các phương pháp như lấy mẫu tốt nhất trong N (*best-of-N sampling*), trong đó nhiều đầu ra được sinh ra và kết quả tối ưu được chọn dựa trên một cơ chế lựa chọn nào đó. Một quá trình lựa chọn như vậy có thể được xem là một bài toán tìm kiếm, gồm hai thành phần:

- **Thuật toán tìm kiếm** (*Search Algorithm*). Thành phần này xác định chiến lược dùng để khám phá không gian các chuỗi đầu ra (lời giải)

khả dĩ và sinh ra một tập ứng viên. Nó có thể đi từ lấy mẫu độc lập đơn giản tới các kỹ thuật tìm kiếm tinh vi hơn như đã bàn ở Mục 11.1.3.

- **Bộ kiểm chứng** (*Verifier*). Đây là một mô hình hoặc một hàm chịu trách nhiệm đánh giá chất lượng, tính đúng đắn hoặc mức hữu ích của từng lời giải ứng viên do thuật toán tìm kiếm sinh ra. Nó đưa ra một điểm số, một xác suất hoặc một phán quyết cho phép hệ thống chọn ra ứng viên tốt nhất. Bộ kiểm chứng có thể là một LLM khác, hoặc thậm chí là một tập luật định sẵn hay các heuristic.

Cho một bài toán đầu vào $\mathbf{x}$, ta có thể biểu diễn một lời giải đầu ra $\mathbf{y}$ dưới dạng một dãy các bước lập luận:

$$\mathbf{y} = (a_1, a_2, \cdots, a_{n_r}), \tag{11.34}$$

trong đó a_i là bước lập luận thứ i, và a_{n_r} là bước cuối cùng, chứa câu trả lời của bài toán. Xem Hình 11.15 để có một ví dụ về đường lập luận nhiều bước.

Thuật toán tìm kiếm có thể sinh ra một tập các lời giải ứng viên:

$$\mathcal{D}_c = \{\mathbf{y}_1, \cdots, \mathbf{y}_K\}. \tag{11.35}$$

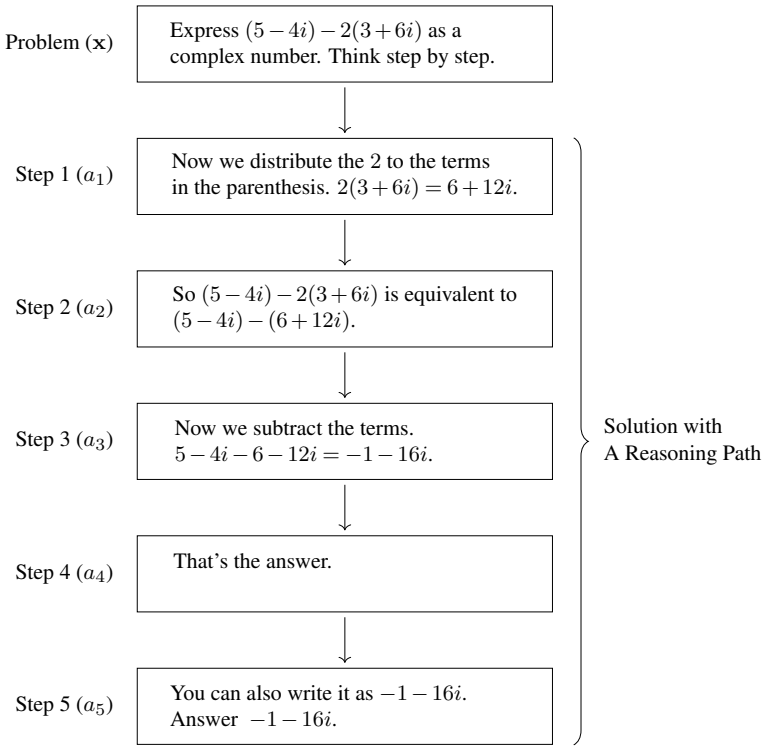
Sau đó, ta có thể dùng một bộ kiểm chứng, đánh giá từng lời giải bằng hàm $V(\mathbf{y})$, để chấm điểm các ứng viên trong $\mathcal{D}_c$. Đầu ra cuối cùng là ứng viên tốt nhất được bộ kiểm chứng chọn:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y} \in \mathcal{D}_c} V(\mathbf{y}). \tag{11.36}$$

Chú giải của người dịch: Đọc Phương trình (11.34–11.36)

Hãy hình dung ta cho LLM giải cùng một bài toán $K = 5$ lần, được 5 lời giải $\mathbf{y}_1, \ldots, \mathbf{y}_5$ (đó là tập $\mathcal{D}_c$). Bộ kiểm chứng giống như một giáo viên chấm bài, cho mỗi lời giải một điểm $V(\mathbf{y})$, chẳng hạn $3, 8, 5, 8.5, 2$. Ký hiệu arg max nghĩa là "lấy *lời giải* có điểm lớn nhất" (chứ không phải lấy giá trị điểm lớn nhất), nên ở đây $\hat{\mathbf{y}} = \mathbf{y}_4$. Ký hiệu n_r chỉ số bước lập luận của một lời giải, ví dụ ở Hình 11.15 thì $n_r = 5$.

Dù có thể kiểm chứng toàn bộ đường lập luận, một phương án đơn giản hơn là chỉ kiểm chứng bước lập luận cuối cùng. Bằng cách này, hàm



Hình 11.15: Minh họa lập luận nhiều bước. Ví dụ này lấy từ tập dữ liệu PRM800K [Lightman và cộng sự, 2024]. Cho một bài toán, LLM được prompt để sinh một đường suy nghĩ (hay đường lập luận) gồm nhiều bước lập luận. Mỗi bước giải quyết một bài toán con dựa trên kết quả của các bước trước. Câu trả lời cho bài toán ban đầu nằm trong bước cuối cùng. (Nội dung trong hình: *Bài toán* (**x**): "Biểu diễn $(5-4i)-2(3+6i)$ dưới dạng một số phức. Hãy suy nghĩ từng bước." *Bước 1* (a_1): "Bây giờ ta phân phối số 2 vào các số hạng trong ngoặc. $2(3+6i)=6+12i$." *Bước 2* (a_2): "Vậy $(5-4i)-2(3+6i)$ tương đương với $(5-4i)-(6+12i)$." *Bước 3* (a_3): "Bây giờ ta trừ các số hạng. $5-4i-6-12i=-1-16i$." *Bước 4* (a_4): "Đó là câu trả lời." *Bước 5* (a_5): "Bạn cũng có thể viết nó là $-1-16i$. Đáp số $-1-16i$." *Solution with A Reasoning Path*: lời giải kèm một đường lập luận.)

kiểm chứng $V(\mathbf{y})$ được đơn giản hóa để chỉ phụ thuộc vào câu trả lời cuối cùng chứa trong a_{n_r}. Điều này có thể đạt được theo nhiều cách, tùy thuộc vào bản chất của bài toán và định dạng câu trả lời mong đợi.

- Với một số bài toán toán học và lập trình, ta có thể dùng các công cụ có sẵn làm bộ kiểm chứng. Ví dụ bao gồm các bộ kiểm tra chứng minh cho các định lý toán học, các trình thông dịch hoặc trình biên dịch để thực thi mã, và các hệ thống kiểm thử đơn vị để kiểm chứng tính đúng đắn của chương trình trên các ca kiểm thử định sẵn.

- Nếu có dữ liệu gán nhãn để đánh giá câu trả lời, chẳng hạn dữ liệu về sở thích của con người (*human preference*), ta có thể huấn luyện một mô hình phần thưởng trên dữ liệu đó. Mô hình phần thưởng học được sau đó được dùng làm bộ kiểm chứng, gán một điểm số vô hướng cho mỗi câu trả lời ứng viên.

- Nếu không có hệ thống sẵn có hay mô hình phần thưởng phù hợp, ta có thể dùng một LLM khác đóng vai trò bộ kiểm chứng. LLM này được prompt để đánh giá chất lượng của câu trả lời ứng viên. Nó có thể là một mô hình mạnh hơn, hoặc chính LLM đó được dùng với một prompt "người đánh giá" cụ thể.

- Ngoài ra, có thể thiết kế các bộ kiểm chứng đơn giản hơn dựa trên heuristic. Một cách thường dùng là bỏ phiếu đa số, trong đó câu trả lời xuất hiện nhiều nhất trong một tập ứng viên được chọn.

Dùng các bộ kiểm chứng này, ta có thể đánh giá một tập lời giải ứng viên được sinh ra trong quá trình tìm kiếm. Một chiến lược đơn giản, thường được gọi là **co giãn song song** (*parallel scaling*) [Brown và cộng sự, 2024; Snell và cộng sự, 2024], bao gồm việc sinh K lời giải ứng viên bằng cách chạy LLM gốc K lần một cách độc lập. Trong quá trình này, ta có thể điều chỉnh nhiệt độ (*temperature*) khi lấy mẫu để kiểm soát độ đa dạng của các đầu ra. Bộ kiểm chứng sau đó đánh giá từng lời giải trong K lời giải hoàn chỉnh này, và lời giải có điểm cao nhất được chọn làm đầu ra cuối cùng. Về mặt khái niệm, điều này rất giống với lấy mẫu tốt nhất trong N, phương pháp mà ở các chương trước chúng ta chủ yếu mô tả như một cách chọn ra đầu ra tốt nhất từ một tập đầu ra được lấy mẫu bằng một mô hình phần thưởng.

Một cách tiếp cận khác là **co giãn tuần tự** (*sequential scaling*), xây

dựng một dãy lời giải theo kiểu tăng dần [Gou và cộng sự, 2024; Zhang và cộng sự, 2024]. Nó bắt đầu với một lời giải ban đầu do LLM sinh ra nhờ prompt. Sau đó, ta dùng một bộ kiểm chứng (thường chính là LLM đó) để đánh giá lời giải. Đây có thể xem là một giai đoạn *phê bình*. Đầu ra của giai đoạn này là một dạng phản hồi nào đó, chẳng hạn các nhận xét bằng văn bản chỉ ra lỗi hoặc gợi ý cải tiến, các điểm số phản ánh chất lượng lời giải, hoặc thậm chí một kế hoạch đã sửa đổi hay một bước trung gian để định hướng lần sinh tiếp theo. Phản hồi này, cùng với bài toán ban đầu và lời giải hiện tại, sau đó được dùng để prompt LLM sinh ra một lời giải có thể tốt hơn. Đây có thể xem là một giai đoạn *tinh chỉnh lại*. Chu trình phê bình–tinh chỉnh lại này có thể được lặp lại, tạo thành một vòng lặp:

$$\mathbf{y}_{k+1} \;=\; \text{Refine}(\mathbf{x}, \mathbf{y}_k, \text{Feedback}(\mathbf{y}_k)), \tag{11.37}$$

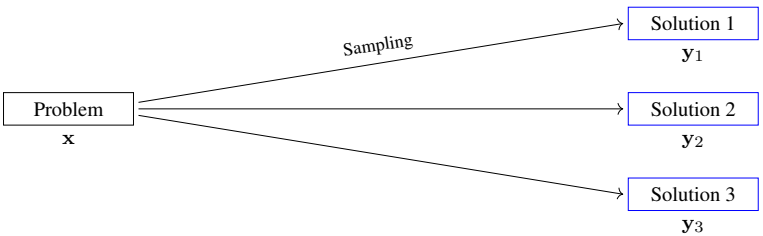
trong đó $\text{Feedback}(\mathbf{y}_k)$ biểu diễn phản hồi từ bộ kiểm chứng. Hàm $\text{Refine}(\cdot)$ sinh ra lời giải cải tiến $\mathbf{y}_{k+1}$ bằng cách prompt LLM với bài toán ban đầu $\mathbf{x}$, lời giải trước đó $\mathbf{y}_k$ và phản hồi này. Quá trình có thể được lặp lại K vòng, hoặc cho đến khi chất lượng lời giải, theo đánh giá của bộ kiểm chứng, hội tụ về một mức thỏa đáng. Khung lặp này, trong đó một lời giải được cải thiện dần qua các chu trình sinh, đánh giá (phê bình) và sửa đổi, chính là cái tạo nên tự tinh chỉnh (*self-refinement*) [Shinn và cộng sự, 2023; Madaan và cộng sự, 2024]. Trong các tình huống như vậy, vai trò của bộ kiểm chứng không chỉ là chọn lời giải hoàn chỉnh tốt nhất từ một tập cố định, mà còn chủ động định hướng chính quá trình sinh.

Chú giải của người dịch: Song song hay tuần tự?

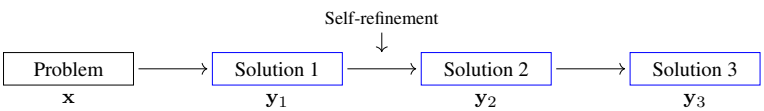
Co giãn song song giống như nộp cùng một đề cho 5 học sinh làm độc lập rồi chọn bài tốt nhất. Co giãn tuần tự giống như một học sinh làm bài, giáo viên phê "bước 2 sai dấu", học sinh sửa lại, giáo viên phê tiếp, và cứ thế. Trong Phương trình (11.37), $\mathbf{y}_k$ là bản làm thứ k, $\text{Feedback}(\mathbf{y}_k)$ là lời phê, còn Refine là việc "sửa bài theo lời phê" để được bản $\mathbf{y}_{k+1}$.

Xem Hình 11.16 để có minh họa về co giãn song song và co giãn tuần tự. Lưu ý rằng còn có những cách khác để thực hiện tìm kiếm và thu được các tập lời giải ứng viên khác nhau. Một phương pháp thay thế là tổ chức việc tìm kiếm thành một cấu trúc cây. Cách tiếp cận này, thường được gọi là tìm kiếm cây (*tree search*), mang lại một cách có cấu trúc hơn để khám phá không gian các đường lập luận khả dĩ. Trong tìm kiếm mức lời giải,

mỗi nút của cây biểu diễn một lời giải hoàn chỉnh. Trong quá trình tìm kiếm, ta cần mở rộng một nút thành một tập các nút con, biểu diễn các lời giải mới có thể được xem xét để kiểm chứng. Quá trình mở rộng thường gồm việc lấy một lời giải sẵn có (nút cha) và dùng LLM để sinh ra các biến thể hoặc các lời giải thay thế.



(a) Parallel Scaling



(b) Sequential Scaling

Hình 11.16: Minh họa co giãn song song (*Parallel Scaling*) và co giãn tuần tự (*Sequential Scaling*). Trong co giãn song song, ta thu được nhiều lời giải (*Solution*) bằng cách chạy LLM nhiều lần một cách độc lập (*Sampling*: lấy mẫu). Trong co giãn tuần tự, LLM sinh ra một lời giải ban đầu. Sau đó, ta dùng LLM để tinh chỉnh lại nó một cách lặp đi lặp lại (*Self-refinement*: tự tinh chỉnh), mỗi lần tinh chỉnh tạo ra một lời giải mới, có thể tốt hơn. (*Problem*: bài toán.)

2. Tìm kiếm mức bước với bộ kiểm chứng

Trong khi các phương pháp bàn ở trên chủ yếu tập trung vào việc sinh các lời giải hoàn chỉnh trước khi lựa chọn cuối cùng, quá trình tìm kiếm cũng có thể được tích hợp sâu hơn vào chính việc sinh từng bước của đường lập luận. Điều này dẫn tới các cách tiếp cận thực hiện tìm kiếm mức bước (*step-level search*) với bộ kiểm chứng, trong đó việc định hướng hoặc cắt tỉa diễn ra ở các bước lập luận trung gian $\{a_1, \cdots, a_i\}$ thay vì chỉ sau khi đã hình thành một lời giải đầy đủ $\mathbf{y}$.

Việc điều khiển chi tiết như vậy đặc biệt có lợi cho các bài toán lập luận phức tạp, nơi chỉ một bước trung gian sai cũng có thể làm toàn bộ chuỗi lập luận phía sau trở nên vô giá trị. Bằng cách đánh giá hoặc định hướng việc sinh ở mỗi bước trung gian, LLM có thể khám phá không gian lập luận hiệu quả hơn, có thể sớm cắt tỉa các đường không triển vọng hoặc dành nhiều tài nguyên hơn để khám phá các đường hợp lý hơn.

Tìm kiếm mức bước với bộ kiểm chứng cũng có thể được mô hình hóa như một bài toán tìm kiếm cây. Trong mô thức này, mỗi nút (hay trạng thái) tương ứng với một đường lập luận bộ phận, $\mathbf{a}_{\leq i} = (a_1, \cdots, a_i)$, biểu diễn dãy i bước lập luận đã thực hiện cho đến lúc này (tức là một đường đi từ nút gốc tới nút hiện tại). Mục tiêu của quá trình tìm kiếm là khám phá không gian trạng thái nền, bắt đầu từ một đường rỗng ban đầu, để tìm ra một đường hoàn chỉnh tạo thành một lời giải đúng. Lưu ý rằng ở đây ta dùng $\mathbf{a}_{\leq i}$ để biểu diễn một đường lập luận bộ phận thay vì $\mathbf{y}_{\leq i}$. Dù điều này làm ký hiệu hơi thiếu nhất quán với ký hiệu dùng cho các lời giải hoàn chỉnh ($\mathbf{y}$) hay các đường đầy đủ trong tìm kiếm mức lời giải, nó giúp nhấn mạnh trọng tâm vào từng hành động hay từng bước riêng lẻ.

Các thành phần cốt lõi của tìm kiếm mức bước với bộ kiểm chứng là:

- **Biểu diễn nút** (*Node Representation*). Một nút là một đường lập luận bộ phận $\mathbf{a}_{\leq i} = (a_1, \cdots, a_i)$. Nút gốc là một đường rỗng, và các nút lá (nút kết thúc) là các đường lập luận hoàn chỉnh.

- **Mở rộng nút** (*Node Expansion*). Cho một đường bộ phận hiện tại $\mathbf{a}_{\leq i}$, LLM được dùng để sinh một hoặc nhiều bước lập luận tiếp theo ứng viên $\{a_{i+1}^{(1)}, \cdots, a_{i+1}^{(M)}\}$. Mỗi bước ứng viên, khi được nối vào $\mathbf{a}_{\leq i}$, tạo thành một đường bộ phận tiềm năng mới $\mathbf{a}_{\leq i+1} = (a_1, \cdots, a_i, a_{i+1}^{(j)})$.

- **Kiểm chứng** (*Verification*). Bộ kiểm chứng $V(\cdot)$ đánh giá chất lượng của một bước mới sinh trong ngữ cảnh của đường bộ phận hiện tại $\mathbf{a}_{\leq i} = (a_1, \cdots, a_i)$ và bài toán ban đầu $\mathbf{x}$. Cũng như kiểm chứng mức lời giải, các bộ kiểm chứng mức bước có thể đưa ra một điểm số, một nhãn phân loại, hoặc phản hồi bằng văn bản.

- **Tìm kiếm** (*Search*). Thành phần này chi phối cách khám phá không gian tìm kiếm. Dựa trên các đánh giá từ bộ kiểm chứng, chiến lược tìm kiếm quyết định những đường bộ phận nào được mở rộng tiếp,

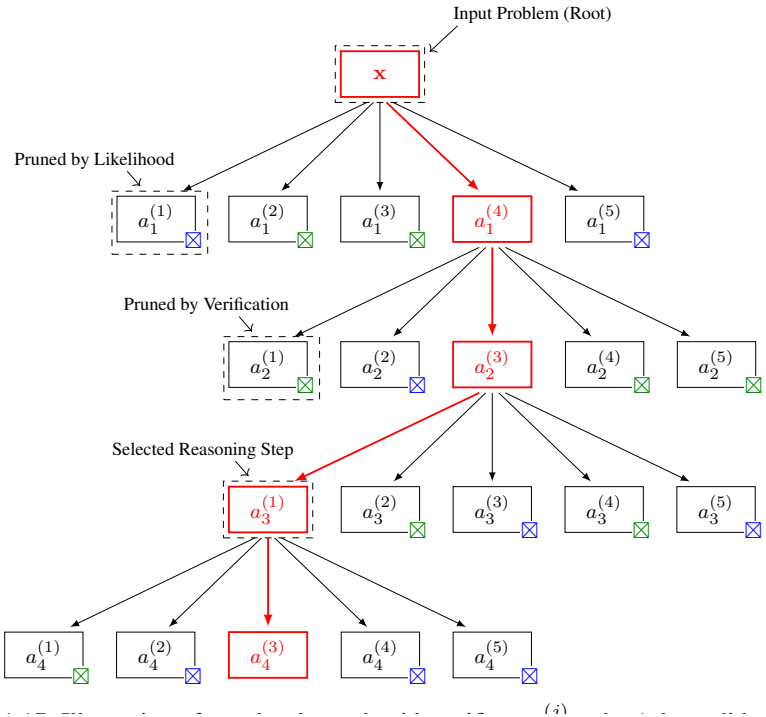
những đường nào bị cắt tỉa, và thứ tự khám phá.

Việc kiểm chứng từng bước này cho phép điều chỉnh động quá trình lập luận. Nếu một bước a_{i+1} bị $V(\cdot)$ đánh giá là sai hoặc không triển vọng, thuật toán tìm kiếm có thể quay lui và khám phá các bước thay thế từ $\mathbf{a}_{\leq i}$, hoặc thậm chí từ một nút sớm hơn $\mathbf{a}_{\leq i'}$ (với $i' < i$). Ngược lại, nếu một bước được đánh giá cao, tài nguyên có thể được tập trung vào việc mở rộng đường đó. Xem Hình 11.17 để có minh họa về tìm kiếm mức bước với bộ kiểm chứng.

Rõ ràng, khung tìm kiếm này rất giống khung được dùng trong các phương pháp giải mã cho LLM, như đã bàn ở Mục 11.1.3. Chẳng hạn, tìm kiếm chùm duy trì một tập gồm K chuỗi bộ phận triển vọng nhất ở mỗi bước sinh. Đây là một dạng tìm kiếm mức bước, trong đó "bộ kiểm chứng" ngầm định chính là mô hình xác suất của LLM, còn "tìm kiếm" là cơ chế cắt tỉa để duy trì kích thước chùm.

Tuy nhiên, tìm kiếm mức bước với các bộ kiểm chứng tường minh, như mô tả ở đây, khác với các phương pháp giải mã tiêu chuẩn. Một khác biệt then chốt là bộ kiểm chứng có thể là một thành phần tinh vi hơn nhiều so với chỉ các xác suất đầu ra thô của LLM sinh. Thiết kế các bộ kiểm chứng mức bước tương tự như thiết kế kiểm chứng mức lời giải. Một bộ kiểm chứng mức bước có thể là một mô hình ngôn ngữ đánh giá chất lượng của một bước lập luận riêng lẻ trong ngữ cảnh của đường đi phía trước. LLM này thậm chí có thể được tinh chỉnh để tăng cường khả năng kiểm chứng. Ngoài ra, với các lĩnh vực có luật lệ được định nghĩa rõ ràng, nó có thể là một bộ máy ký hiệu hoặc một tập các phép kiểm tra bằng chương trình. Hơn nữa, các bộ kiểm chứng có thể được thiết kế để dự đoán mức hữu ích trong tương lai hoặc khả năng thành công khi biết đường bộ phận hiện tại, lấy cảm hứng từ các hàm giá trị (*value function*) trong học tăng cường (*reinforcement learning*). Chuyên môn của con người cũng có thể được đưa vào để đưa ra phán đoán ở các bước then chốt, đặc biệt trong các tình huống có rủi ro cao.

Một ví dụ về bộ kiểm chứng mức bước như vậy, đặc biệt khi dùng phản hồi của con người để đánh giá tiến trình trung gian, là **mô hình phần thưởng quá trình** (*process reward model*) (**PRM**). Một PRM thường là một mô hình ngôn ngữ riêng biệt, được huấn luyện để đưa ra một phần thưởng vô hướng cho mỗi bước lập luận $a_{i'}$ trong một đường bộ phận $\mathbf{a}_{\leq i}$. Nó cung cấp một tín hiệu giám sát trực tiếp và chi tiết hơn so với các



Hình 11.17: Minh họa tìm kiếm mức bước với bộ kiểm chứng. $a_i^{(j)}$ = ứng viên thứ j cho bước lập luận thứ i; dấu ☒ (xanh lam) = ứng viên bị cắt tỉa theo xác suất đầu ra của LLM; dấu ☒ (xanh lục) = ứng viên bị bộ kiểm chứng cắt tỉa. Với bài toán đầu vào làm nút gốc (*Input Problem (Root)*), ta mở rộng cây bằng cách sinh nhiều bước lập luận ở mỗi lần mở rộng. Mỗi ứng viên có thể bị cắt tỉa hoặc theo khả năng (*likelihood*) (như trong giải mã tiêu chuẩn; *Pruned by Likelihood*) hoặc theo kiểm chứng mức bước (*Pruned by Verification*). Các ứng viên không bị cắt tỉa sau đó được mở rộng để sinh thêm các bước lập luận (*Selected Reasoning Step*: bước lập luận được chọn). Quá trình được lặp lại cho đến khi sinh ra một chuỗi lập luận hoàn chỉnh dẫn tới câu trả lời cuối cùng, hoặc cho đến khi đạt tới một giới hạn tìm kiếm định sẵn.

mô hình phần thưởng kết quả (*outcome reward model*) (**ORM**), vốn chỉ đánh giá lời giải cuối cùng. Tuy nhiên, việc phát triển PRM dựa vào các chú thích mức bước của con người, chẳng hạn mức ưa thích đối với các bước tiếp theo khác nhau. Thu thập tín hiệu giám sát cho từng bước trung gian tốn nhiều công sức hơn đáng kể và đòi hỏi người gán nhãn nỗ lực nhận thức nhiều hơn so với chỉ gán nhãn kết quả cuối cùng.

Chú giải của người dịch: PRM và ORM: chấm từng bước hay chỉ chấm đáp số?

Hãy nghĩ tới cách chấm bài thi tự luận môn Toán. ORM giống như chỉ nhìn đáp số cuối cùng: đúng thì cho điểm, sai thì không, bất kể cách làm. PRM giống như chấm theo *biểu điểm từng bước*: bước 1 đúng được 0.25, bước 2 đúng được 0.5,… Nhờ vậy PRM chỉ ra được *chỗ nào* trong lời giải bắt đầu sai, rất hữu ích để cắt tỉa sớm các nhánh sai trong cây tìm kiếm ở Hình 11.17. Nhưng để huấn luyện PRM, cần người gán nhãn đọc và đánh giá từng bước, tốn công hơn nhiều.

Một cách tiếp cận thay thế để xây dựng dữ liệu huấn luyện cho kiểm chứng mức bước là dùng LLM để tự động sinh ra các chú thích như vậy. Chẳng hạn, ta có thể lấy một LLM mạnh, gọi là mô hình thầy (*teacher model*), và prompt nó để trước hết sinh một đường lập luận hoàn chỉnh cho một bài toán cho trước. Sau đó, tại mỗi bước trung gian trong đường này, ta có thể prompt chính LLM thầy đó (hoặc một LLM có năng lực khác) để sinh ra một vài bước tiếp theo ứng viên thay thế, bên cạnh bước mà nó đã chọn ban đầu. LLM thầy sau đó lại có thể được prompt để đánh giá các phương án thay thế này. Các kết quả đánh giá này (ví dụ đúng hay sai) sau đó có thể dùng làm chú thích dữ liệu. Một cách khác là tận dụng khả năng tổng quát hóa của PRM. Ta có thể huấn luyện một PRM trên các nhiệm vụ mà việc kiểm chứng mức bước dễ hơn, rồi tổng quát hóa PRM này sang các nhiệm vụ khác với rất ít hoặc không cần huấn luyện thêm.

Lưu ý rằng kiểm chứng mức bước cũng có những vấn đề riêng. Kiểm chứng thường xuyên, đặc biệt nếu dùng LLM làm bộ kiểm chứng, có thể làm tăng đáng kể chi phí tính toán và độ trễ. Bản thân việc thiết kế các bộ kiểm chứng mức bước hiệu quả cũng không hề đơn giản. Một bộ kiểm chứng không chính xác có thể loại bỏ quá sớm các đường lập luận tốt hoặc không nhận ra các đường có lỗi, từ đó dẫn dắt sai việc tìm kiếm. Điều này khiến việc phát triển các hệ thống như vậy khó khăn hơn.

3. Khuyến khích suy nghĩ dài

Cho đến giờ trong tiểu mục này, phần lớn các phương pháp đều ngầm dựa trên một ý tưởng đơn giản: sinh các đường lập luận dài hơn có thể có ích. Ngoài CoT và tìm kiếm có kiểm chứng, ta có thể xem xét các phương pháp thay thế để đạt được điều này. Chẳng hạn, ta có thể prompt LLM bằng cách yêu cầu tường minh việc cân nhắc kỹ lưỡng hơn. Ngoài prompt trực tiếp, ta cũng có thể sửa đổi chính quá trình giải mã, chẳng hạn điều chỉnh giới hạn số token hoặc áp dụng hình phạt cho các đầu ra ngắn. Một cách tiếp cận khác là dùng các sơ đồ sinh nhiều giai đoạn, trong đó mô hình xây dựng dần dần dựa trên lập luận của chính nó.

4. Co giãn dựa trên huấn luyện

Bên cạnh việc xem xét các phương pháp co giãn lúc suy luận không cần huấn luyện, chúng ta cũng muốn xem xét các phương pháp có thể cải thiện năng lực lập luận nội tại của LLM bằng cách sửa đổi tham số của chúng thông qua huấn luyện thêm. Dù các phương pháp co giãn dựa trên huấn luyện như vậy thường đòi hỏi thêm chi phí huấn luyện và tài nguyên tính toán, chúng đưa các kỹ năng lập luận mạnh hơn trực tiếp vào tham số mô hình, điều này đến lượt nó có thể dẫn tới hiệu năng lập luận được cải thiện. Ta thậm chí có thể kết hợp chúng với các phương pháp không cần huấn luyện để cải thiện hơn nữa việc co giãn lúc suy luận.

Dù phần thảo luận ở đây chỉ giới hạn trong các bài toán lập luận, các phương pháp co giãn dựa trên huấn luyện là phổ biến. Phần lớn chúng đã được bàn ở Chương 10. Ở đây, chúng tôi sẽ mô tả ngắn gọn cách các phương pháp này có thể được áp dụng để cải thiện co giãn lúc suy luận cho các bài toán lập luận.

- **Tinh chỉnh trên dữ liệu lập luận** (*Fine-tuning on Reasoning Data*). Một trong những cách trực tiếp nhất để tăng cường khả năng lập luận là tinh chỉnh các LLM đã tiền huấn luyện trên các tập dữ liệu được tuyển chọn riêng cho các nhiệm vụ lập luận. Các tập dữ liệu này có thể đi từ các cặp đầu vào–đầu ra đơn giản đến các định dạng có cấu trúc hơn bao gồm cả quá trình lập luận từng bước. Các ví dụ điển hình gồm các tập dữ liệu bài toán có lời văn, các bài tập suy diễn logic, hoặc sinh mã kèm giải thích. Nhờ huấn luyện trên những dữ liệu như vậy, mô hình học được từ các khuôn

mẫu lập luận phổ biến, và do đó có thể sinh ra các đường lập luận chi tiết và mạch lạc lúc kiểm tra.

- **Học tăng cường cho lập luận** (*Reinforcement Learning for Reasoning*). Nếu coi bộ kiểm chứng là một mô hình phần thưởng, ta có thể thấy các phương pháp bàn ở tiểu mục trước là một ứng dụng trực tiếp của mô hình phần thưởng vào các bài toán lập luận, dù chúng không cần huấn luyện. Tất nhiên, ta có thể áp dụng mô hình phần thưởng này vào việc tinh chỉnh LLM. Điều này đi theo một mô thức tiêu chuẩn của học tăng cường. Cho một mô hình phần thưởng, LLM, đóng vai trò một chính sách (*policy*), được tinh chỉnh bằng các thuật toán học tăng cường. LLM sinh các bước lập luận hoặc các lời giải đầy đủ, nhận phản hồi (phần thưởng) từ mô hình phần thưởng, và cập nhật tham số để tạo ra các đầu ra tối đa hóa những phần thưởng này. Quá trình này căn chỉnh (*align*) đầu ra của LLM với hành vi lập luận chất lượng cao, qua đó khuyến khích LLM sinh ra các đường lập luận đáng tin cậy hơn. Một vấn đề then chốt khác là việc huấn luyện mô hình phần thưởng. Nói chung, mô hình phần thưởng này có thể là một mô hình phần thưởng kết quả, đánh giá tính đúng đắn hoặc chất lượng của câu trả lời cuối cùng, hoặc một mô hình phần thưởng quá trình, đánh giá chất lượng của từng bước lập luận trung gian, như đã bàn trong bối cảnh các bộ kiểm chứng mức bước. Trong một số trường hợp, ta thậm chí có thể xây dựng một mô hình phần thưởng dựa trên các luật đơn giản, chẳng hạn cộng thêm điểm thưởng cho các đầu ra dài hơn.

- **Chưng cất tri thức cho lập luận** (*Knowledge Distillation for Reasoning*). Trong cách tiếp cận này, một LLM *trò* nhỏ hơn, hiệu quả hơn được huấn luyện để bắt chước các đầu ra lập luận hoặc các biểu diễn bên trong của một LLM *thầy* lớn hơn, có năng lực hơn. Mô hình thầy có thể sinh ra các bước lập luận chi tiết cho nhiều loại bài toán. Mô hình trò sau đó học cách tái tạo những minh họa lập luận chất lượng cao này. Chiến lược này giúp năng lực lập luận mạnh hơn trở nên dễ tiếp cận hơn bằng cách triển khai chúng trong các mô hình nhỏ hơn, ít tốn kém tính toán hơn lúc suy luận.

- **Tinh chỉnh lặp** (*Iterative Refinement*). Co giãn dựa trên huấn luyện cũng có thể bao gồm việc tinh chỉnh lặp. Chẳng hạn, một LLM có

thể sinh lời giải cho một tập bài toán. Các lời giải này cùng đường lập luận của chúng sau đó được kiểm chứng, hoặc bởi con người hoặc bởi các bộ kiểm chứng tự động. Các đường lập luận đúng sau đó được thêm vào dữ liệu huấn luyện, và LLM được tinh chỉnh thêm trên tập dữ liệu đã được bổ sung này. Điều này tạo ra một chu trình trong đó LLM dần cải thiện năng lực lập luận của mình qua việc lặp lại sinh, phê bình và học.

Chú giải của người dịch: Chính sách và phần thưởng trong học tăng cường

Trong học tăng cường, *chính sách* là "cách hành động" của tác tử — ở đây chính là LLM, vì nó quyết định sinh token/bước lập luận nào tiếp theo. *Phần thưởng* là con số cho biết kết quả tốt đến đâu, ví dụ $+1$ nếu đáp số đúng và 0 nếu sai (một mô hình phần thưởng kết quả dựa trên luật). Mô hình thử nhiều lời giải, lời giải nào được thưởng cao thì tham số được điều chỉnh để lần sau dễ sinh ra kiểu lời giải đó hơn — giống như học sinh luyện đề: cách làm nào được điểm thì sẽ được dùng lại nhiều hơn.

Ưu điểm chính của các phương pháp co giãn dựa trên huấn luyện này là chúng trang bị cho LLM các kỹ năng lập luận nội tại mạnh hơn. Điều này trực tiếp góp phần cải thiện co giãn lúc suy luận theo nhiều cách: nó có thể dẫn tới suy luận hiệu quả hơn, vì LLM có thể cần tìm kiếm ít hơn hoặc ít mẫu sinh hơn để đi đến một lời giải đúng. Hơn nữa, chất lượng nền của các bước hay lời giải được sinh ra cao hơn. Do đó, một LLM được huấn luyện tốt có thể tổng quát hóa năng lực lập luận đã học sang các bài toán mới hiệu quả hơn so với một LLM chỉ dựa vào học trong ngữ cảnh (*in-context learning*) hoặc các sơ đồ suy luận không cần huấn luyện.

Mặt khác, so với các phương pháp không cần huấn luyện, các cách tiếp cận dựa trên huấn luyện cũng đặt ra những thách thức. Việc tạo ra các tập dữ liệu huấn luyện quy mô lớn, chất lượng cao cho lập luận có thể tốn kém và tốn nhiều công sức. Bản thân quá trình tinh chỉnh, đặc biệt với các LLM lớn nhất hoặc khi dùng học tăng cường, có thể đòi hỏi nhiều tính toán và công sức kỹ thuật đáng kể. Cũng có rủi ro mô hình bị quá khớp (*overfitting*) với các loại bài toán hay phong cách lập luận cụ thể có trong dữ liệu huấn luyện, có thể hạn chế hiệu năng của nó trên các nhiệm vụ ngoài phân phối (*out-of-distribution*).

11.4 Tóm tắt

Trong chương này, chúng ta đã bàn về suy luận cho LLM. Chúng ta đã trình bày khung điền trước–giải mã và các thuật toán giải mã liên quan cho suy luận LLM. Sau đó, chúng ta đã mô tả một số kỹ thuật suy luận hiệu quả. Chúng ta cũng đã bàn về co giãn lúc suy luận, một trong những cách tiếp cận quan trọng nhất để cải thiện khả năng lập luận của LLM.

Suy luận trên dữ liệu tuần tự từ lâu đã là một chủ đề nền tảng trong AI [Wozengraft và Reiffen, 1961; Viterbi, 1967; Forney, 1972]. Trong bối cảnh NLP, hướng nghiên cứu này có từ những ngày đầu tiên của nhận dạng tiếng nói (*speech recognition*) và dịch máy thống kê (*statistical machine translation*) [Koehn, 2010], khi các nhà nghiên cứu đối mặt với thách thức tìm kiếm hiệu quả trong những không gian giả thuyết khổng lồ để tìm ra chuỗi đầu ra có xác suất cao nhất. Các kỹ thuật như tìm kiếm chùm và nhiều chiến lược cắt tỉa khác nhau đã được phát triển từ thời đó để làm cho việc này khả thi về mặt tính toán. Khi ấy, các mô hình còn tương đối yếu, và phần lớn nghiên cứu tập trung vào phát triển các thuật toán tìm kiếm mạnh để giảm lỗi tìm kiếm. Những ý tưởng nền tảng này tiếp tục ảnh hưởng tới các cách tiếp cận hiện đại.

Khi bước vào kỷ nguyên do các phương pháp học sâu (*deep learning*) thống trị, các mô hình dựa trên mạng nơ-ron sâu đã trở nên cực kỳ mạnh mẽ. Ngay cả với những thuật toán tìm kiếm rất đơn giản, các mô hình này cũng có thể đạt kết quả xuất sắc. Trong bối cảnh đó, suy luận dường như không còn "quan trọng" như trước, và sự chú ý của giới nghiên cứu dần chuyển sang kiến trúc mô hình, phương pháp huấn luyện và mở rộng quy mô mô hình.

Tuy nhiên, xu hướng này gần đây đã thay đổi. Với sự trỗi dậy của LLM, suy luận một lần nữa thu hút sự chú ý đáng kể. Sự quan tâm trở lại này chủ yếu thể hiện ở hai khía cạnh:

- Chi phí suy luận của LLM rất cao. Chẳng hạn, triển khai hiệu quả LLM trong các tình huống có nhiều yêu cầu đồng thời và đòi hỏi độ trễ thấp vẫn là một bài toán khó, khiến hiệu quả suy luận trở nên cực kỳ quan trọng. Trong bối cảnh này, các thiết kế kiến trúc hiệu quả, các thuật toán tìm kiếm được tối ưu và nhiều chiến lược tối ưu suy luận khác nhau có ý nghĩa thực tiễn đáng kể.

- Độ dài chuỗi đầu vào và đầu ra đã tăng đáng kể trong các nhiệm vụ phức tạp. Đặc biệt trong các nhiệm vụ như lập luận toán học, sự tăng trưởng độ dài chuỗi càng làm nổi bật tầm quan trọng của hiệu quả suy luận. Hơn nữa, việc co giãn quá trình suy luận gần đây đã được chứng tỏ là một cách hiệu quả để cải thiện năng lực lập luận của mô hình. Do đó, đạt được co giãn suy luận hiệu quả đang nổi lên như một hướng nghiên cứu đặc biệt hứa hẹn.

Suy luận giờ đây là một chủ đề rộng, bao gồm nhiều kỹ thuật. Nó không chỉ liên quan đến việc phát triển kiến trúc mô hình và thuật toán giải mã, mà ngày càng được định hình bởi những công việc kỹ thuật phức tạp và các tối ưu tinh vi ở cấp hệ thống cần thiết để triển khai LLM một cách hiệu quả và tiết kiệm. Nhiều kỹ thuật trong số này nằm ngoài phạm vi của NLP hay của một lĩnh vực AI cụ thể. Thay vào đó, biên giới của tối ưu suy luận LLM giờ đây đã mở rộng sâu vào những lĩnh vực vốn được coi là cốt lõi của khoa học và kỹ thuật máy tính.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.